1 Contest 1

2 Mathematics 1

3 Data structures 3

4 Numerical 5

5 Number theory 9

6 Combinatorial 11

7 Graph 12

8 Matching 14

9 Geometry 18

10 Strings 22

11 Various 24


# Contest (1)

template.cpp 6 lines

```
ll cdiv(ll a,ll b){return a/b+((a^b)>0&&a%b);}
ll fdiv(ll a,ll b){return a/b-((a^b)<0&&a%b);}
int main(){
  cin.tie(0)->sync_with_stdio(0);
  cin.exceptions(cin.failbit);
}
```

hash.sh 3 lines

```
# Hashes a file, ignoring all whitespace and comments. Use for
# verifying that code was correctly typed.
cpp -dD -P -fpreprocessed | tr -d '[:space:]'| md5sum |cut -c-6
```

## 1.1 Python 速查

```
import sys, math
from collections import deque, Counter, defaultdict
from heapq import heappush, heappop, heapify, nsmallest
from bisect import bisect_left, bisect_right, insort
from itertools import accumulate, combinations, product
from functools import lru_cache, cmp_to_key
from fractions import Fraction
from decimal import Decimal, getcontext
input = sys.stdin.readline  # 快速读入，行尾带换行
sys.setrecursionlimit(1 << 20)  # 递归深的题优先改成循环
n, m = map(int, input().split())
a = list(map(int, input().split()))
rest = sys.stdin.read().split()  # 一次读入剩下全部
print(*a)  # 空格分隔输出
sys.stdout.write("\n".join(map(str, a)) + "\n")
x = 10 ** 100  # 整数没有上限
p = 998244353
y = pow(3, x, p)  # 快速幂取模
z = pow(3, -1, p)  # 逆元，需 3.8 以上
g = math.gcd(12, 18); s = math.isqrt(10 ** 20)
c = math.comb(50, 25); l = math.lcm(4, 6)  # lcm 需 3.9 以上
q = -7 // 2  # 向下取整得 -4，和 C++ 不同，% 也是
f = Fraction(1, 3) + Fraction(1, 6)  # 精确分数
getcontext().prec = 50
d = Decimal(2).sqrt()  # 高精度小数
h = int("ff", 16); b = bin(10)[2:]; k = (10).bit_length()
pc = bin(10).count("1"); t = "abc"[::-1]; o = ord("a")
a.sort(key=lambda v: (-v, v % 3))  # 多关键字排序
a.sort(key=cmp_to_key(lambda u, v: u - v))  # 自定义比较
adj = [[] for _ in range(n)]  # 不要写 [[]] * n
dp = [[0] * m for _ in range(n)]
pq = [5, 1, 4]; heapify(pq); heappush(pq, 2)
mn = heappop(pq)  # 小根堆，大根堆存相反数
dq = deque([1, 2]); dq.appendleft(0); dq.pop()
i = bisect_left([1, 3, 5], 3)  # 同 lower_bound
cnt = Counter("abca"); dd = defaultdict(int)
pre = list(accumulate([1, 2, 3], initial=0))  # 前缀和
@lru_cache(maxsize=None)  # 记忆化搜索
def fib(v): return v if v < 2 else fib(v - 1) + fib(v - 2)
INF = float("inf")
# 字符串，不可修改，要改先转成 list(s)
s = "  Hello,World  "
w = s.strip().split(",")  # 去首尾空白，按逗号切
j = "-".join(w); u = s.upper(); lo = s.lower()
pos = s.find("o"); r = s.replace("l", "L")  # 找不到返回 -1
ok = "123".isdigit(); bg = s.strip().startswith("He")
cs = list("abc"); cs[0] = "z"; s2 = "".join(cs)
fs = f"{3.14159:.3f} {42:05d} {255:x}"  # 格式化输出
# 列表
a = [3, 1, 2]; a.append(5); a.insert(0, 9); a.pop()
a.extend([7, 7]); c7 = a.count(7); i1 = a.index(1)
b = sorted(a, reverse=True); a.reverse()
sq = [v * v for v in a if v % 2]  # 列表推导
mx = max(a, key=lambda v: -v); sm = sum(a)
for i, v in enumerate(a): pass
pairs = list(zip([1, 2], "ab")); rv = a[::-1]; sub = a[1:3]
# 集合 / 字典 / 元组
st = {1, 2}; st.add(3); st.discard(9); st2 = st | {4}
inter = st & {2, 3}; diff = st - {1}; has = 2 in st
d = {"x": 1}; d["y"] = 2; gv = d.get("z", 0)
for k, v in sorted(d.items(), key=lambda t: -t[1]): pass
dl = defaultdict(list); dl[1].append(2)
x1, y1 = (4, 5); x1, y1 = y1, x1  # 解包，交换
mc = Counter("aabbbc").most_common(2)  # 出现最多的两个
small = nsmallest(2, [5, 1, 4]); dq2 = deque([1, 2, 3])
dq2.rotate(1)  # 循环右移一位
```

# Mathematics (2)

## 2.1 Equations

$$ax^2 + bx + c = 0 \Rightarrow x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

The extremum is given by $x = -b/2a$.

$$\begin{aligned} ax + by &= e \\ cx + dy &= f \end{aligned} \Rightarrow \begin{aligned} x &= \frac{ed - bf}{ad - bc} \\ y &= \frac{af - ec}{ad - bc} \end{aligned}$$

In general, given an equation $Ax = b$, the solution to a variable $x_i$ is given by

$$x_i = \frac{\det A_i'}{\det A}$$

where $A_i'$ is $A$ with the $i$'th column replaced by $b$.

## 2.2 Recurrences

If $a_n = c_1 a_{n-1} + \cdots + c_k a_{n-k}$, and $r_1, \ldots, r_k$ are distinct roots of $x^k - c_1 x^{k-1} - \cdots - c_k$, there are $d_1, \ldots, d_k$ s.t.

$$a_n = d_1 r_1^n + \cdots + d_k r_k^n.$$

Non-distinct roots $r$ become polynomial factors, e.g. $a_n = (d_1 n + d_2) r^n$.

## 2.3 Trigonometry

$$\sin(v + w) = \sin v \cos w + \cos v \sin w$$
$$\cos(v + w) = \cos v \cos w - \sin v \sin w$$

$$\tan(v + w) = \frac{\tan v + \tan w}{1 - \tan v \tan w}$$
$$\sin v + \sin w = 2 \sin \frac{v + w}{2} \cos \frac{v - w}{2}$$
$$\cos v + \cos w = 2 \cos \frac{v + w}{2} \cos \frac{v - w}{2}$$
$$(V + W) \tan(v - w)/2 = (V - W) \tan(v + w)/2$$

where $V, W$ are lengths of sides opposite angles $v, w$.

$$a \cos x + b \sin x = r \cos(x - \phi)$$
$$a \sin x + b \cos x = r \sin(x + \phi)$$

where $r = \sqrt{a^2 + b^2}, \phi = \operatorname{atan2}(b, a)$.

## 2.4 Geometry

### 2.4.1 Triangles

Side lengths: $a, b, c$

Semiperimeter: $p = \frac{a + b + c}{2}$

Area: $A = \sqrt{p(p - a)(p - b)(p - c)}$

Circumradius: $R = \frac{abc}{4A}$

Inradius: $r = \frac{A}{p}$

Length of median (divides triangle into two equal-area triangles): $m_a = \frac{1}{2}\sqrt{2b^2 + 2c^2 - a^2}$

Length of bisector (divides angles in two): $s_a = \sqrt{bc\left[1 - \left(\frac{a}{b+c}\right)^2\right]}$

Law of sines: $\frac{\sin\alpha}{a} = \frac{\sin\beta}{b} = \frac{\sin\gamma}{c} = \frac{1}{2R}$

Law of cosines: $a^2 = b^2 + c^2 - 2bc\cos\alpha$

Law of tangents: $\frac{a+b}{a-b} = \frac{\tan\frac{\alpha+\beta}{2}}{\tan\frac{\alpha-\beta}{2}}$

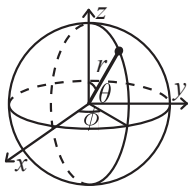
### 2.4.2 Quadrilaterals

With side lengths $a, b, c, d$, diagonals $e, f$, diagonals angle $\theta$, area $A$ and magic flux $F = b^2 + d^2 - a^2 - c^2$:

$$4A = 2ef \cdot \sin\theta = F\tan\theta = \sqrt{4e^2f^2 - F^2}$$

For cyclic quadrilaterals the sum of opposite angles is $180°$, $ef = ac + bd$, and $A = \sqrt{(p-a)(p-b)(p-c)(p-d)}$.

### 2.4.3 Spherical coordinates




$$\begin{array}{cc} x = r\sin\theta\cos\phi & r = \sqrt{x^2+y^2+z^2} \\ y = r\sin\theta\sin\phi & \theta = \operatorname{acos}(z/\sqrt{x^2+y^2+z^2}) \\ z = r\cos\theta & \phi = \operatorname{atan2}(y,x) \end{array}$$

## 2.5 Derivatives/Integrals

$$\begin{array}{cc} \frac{d}{dx}\arcsin x = \frac{1}{\sqrt{1-x^2}} & \frac{d}{dx}\arccos x = -\frac{1}{\sqrt{1-x^2}} \\ \frac{d}{dx}\tan x = 1 + \tan^2 x & \frac{d}{dx}\arctan x = \frac{1}{1+x^2} \\ \int \tan ax = -\frac{\ln|\cos ax|}{a} & \int x\sin ax = \frac{\sin ax - ax\cos ax}{a^2} \\ \int e^{-x^2} = \frac{\sqrt{\pi}}{2}\operatorname{erf}(x) & \int xe^{ax}dx = \frac{e^{ax}}{a^2}(ax-1) \end{array}$$

Integration by parts:

$$\int_a^b f(x)g(x)dx = [F(x)g(x)]_a^b - \int_a^b F(x)g'(x)dx$$

## 2.6 Sums

$$c^a + c^{a+1} + \cdots + c^b = \frac{c^{b+1} - c^a}{c-1}, c \neq 1$$

$$1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2}$$

$$1^2 + 2^2 + 3^2 + \cdots + n^2 = \frac{n(2n+1)(n+1)}{6}$$

$$1^3 + 2^3 + 3^3 + \cdots + n^3 = \frac{n^2(n+1)^2}{4}$$

$$1^4 + 2^4 + 3^4 + \cdots + n^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$$

## 2.7 Series

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \ldots, (-\infty < x < \infty)$$

$$\ln(1+x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \ldots, (-1 < x \leq 1)$$

$$\sqrt{1+x} = 1 + \frac{x}{2} - \frac{x^2}{8} + \frac{2x^3}{32} - \frac{5x^4}{128} + \ldots, (-1 \leq x \leq 1)$$

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \ldots, (-\infty < x < \infty)$$

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \ldots, (-\infty < x < \infty)$$

## 2.8 Probability theory

Let $X$ be a discrete random variable with probability $p_X(x)$ of assuming the value $x$. It will then have an expected value (mean) $\mu = \mathbb{E}(X) = \sum_x xp_X(x)$ and variance $\sigma^2 = V(X) = \mathbb{E}(X^2) - (\mathbb{E}(X))^2 = \sum_x (x - \mathbb{E}(X))^2 p_X(x)$ where $\sigma$ is the standard deviation. If $X$ is instead continuous it will have a probability density function $f_X(x)$ and the sums above will instead be integrals with $p_X(x)$ replaced by $f_X(x)$.

Expectation is linear:

$$\mathbb{E}(aX + bY) = a\mathbb{E}(X) + b\mathbb{E}(Y)$$

For independent $X$ and $Y$,

$$V(aX + bY) = a^2V(X) + b^2V(Y).$$

## 2.9 Markov chains

A *Markov chain* is a discrete random process with the property that the next state depends only on the current state. Let $X_1, X_2, \ldots$ be a sequence of random variables generated by the Markov process. Then there is a transition matrix $\mathbf{P} = (p_{ij})$, with $p_{ij} = \Pr(X_n = i | X_{n-1} = j)$, and $\mathbf{p}^{(n)} = \mathbf{P}^n\mathbf{p}^{(0)}$ is the probability distribution for $X_n$ (i.e., $p_i^{(n)} = \Pr(X_n = i)$), where $\mathbf{p}^{(0)}$ is the initial distribution.

### Stationary distribution

$\pi$ is a stationary distribution if $\pi = \pi\mathbf{P}$. If the Markov chain is *irreducible* (it is possible to get to any state from any state), then $\pi_i = \frac{1}{\mathbb{E}(T_i)}$ where $\mathbb{E}(T_i)$ is the expected time between two visits in state $i$. $\pi_j/\pi_i$ is the expected number of visits in state $j$ between two visits in state $i$.

For a connected, undirected and non-bipartite graph, where the transition probability is uniform among all neighbors, $\pi_i$ is proportional to node $i$'s degree.

### Ergodicity

A Markov chain is *ergodic* if the asymptotic distribution is independent of the initial distribution. A finite Markov chain is ergodic iff it is irreducible and *aperiodic* (i.e., the gcd of cycle lengths is 1). $\lim_{k\to\infty}\mathbf{P}^k = \mathbf{1}\pi$.

### Absorption

A Markov chain is an A-chain if the states can be partitioned into two sets $\mathbf{A}$ and $\mathbf{G}$, such that all states in $\mathbf{A}$ are absorbing ($p_{ii} = 1$), and all states in $\mathbf{G}$ leads to an absorbing state in $\mathbf{A}$. The probability for absorption in state $i \in \mathbf{A}$, when the initial state is $j$,

is $a_{ij} = p_{ij} + \sum_{k\in\mathbf{G}} a_{ik}p_{kj}$. The expected time until absorption, when the initial state is $i$, is $t_i = 1 + \sum_{k\in\mathbf{G}} p_{ki}t_k$.

## 2.10 期望与概率技巧

- **期望线性性**: 不要求独立. 求个数的期望就拆成指示变量, $\mathbb{E}[\text{个数}] = \sum_i \Pr[\text{事件 } i]$.
- **尾和**: $X$ 取非负整数时 $\mathbb{E}[X] = \sum_{k\geq 1}\Pr[X \geq k]$. 独立同分布的最大值: $\Pr[\max \leq k] = F(k)^n$.
- **期望 DP 倒推**: $E_i = c_i + \sum_j p_{ij}E_j$, 终点 $E = 0$, 按拓扑序从后往前算. 只有自环时移项: $E_i = (c_i + \sum_{j\neq i} p_{ij}E_j)/(1-p_{ii})$.
- **有环, 高斯消元**: 把 $E_i - \sum_j p_{ij}E_j = c_i$ 列成方程组, 用 `SolveLinear`, $O(n^3)$, $n$ 约到 500. 取模时把除法换成乘逆元. 吸收概率同理.
- **只有一个未知量**: 设起点期望为 $x$, 其他状态都写成 $a_i + b_i x$ 一路推下去, 最后解 $x = a + bx$, 不用高斯消元.
- **二分答案**: 可以随时重来的题, 答案 $x$ 出现在自己的转移里, 即 $E_i = \min(\text{继续}, x)$. 对猜的 $x$ 做一遍 DP 得到 $f(x)$; 若 $f(x) < x$ 则真答案 $< x$, 否则 $\geq x$. 二分约 100 次.
- **方差**: DP 里同时维护 $\mathbb{E}[X]$ 和 $\mathbb{E}[X^2]$, 如 $\mathbb{E}[(X+1)^2] = \mathbb{E}[X^2] + 2\mathbb{E}[X] + 1$.
- **min-max 容斥**: $\mathbb{E}[\max S] = \sum_{\emptyset\neq T\subseteq S}(-1)^{|T|+1}\mathbb{E}[\min T]$. 用于全部出现的期望时间.
- **常用结论**: 成功率 $p$ 的试验, 期望 $1/p$ 次成功一次. 集齐 $n$ 种卡片: $nH_n$. $n$ 个 $U[0,1]$ 中第 $k$ 小的期望: $k/(n+1)$. 随机排列: 逆序对期望 $n(n-1)/4$, 不动点期望 1, 环数期望 $H_n$.
- **随机游走**: 树上从 $u$ 走到父亲的期望步数是 $2\cdot size(u) - 1$. 0 处反射, 从 0 走到 $n$ 期望 $n^2$ 步. 公平赌徒从 $i$ 出发: 先到 $n$ 的概率 $i/n$, 期望步数 $i(n-i)$.
- **精度**: 期望很大时用 `long double`; 方程太大时可以反复迭代 $E \leftarrow c + PE$ 直到收敛.

# Data structures (3)

### OrderStatisticTree.h

**Description:** 不可重集合 (不是 multiset!), 支持求第 n 个元素和求元素的排名. 要当 map 用, 把 `null_type` 换成值类型.
**Usage:** `Tree<int> t; t.insert(x); int r = t.order_of_key(x);`
`int v = *t.find_by_order(k); // 第 k 小, 从 0 开始`
**Time:** $\mathcal{O}(\log N)$
0ac05e, 17 lines

```
#include <bits/extc++.h>
using namespace __gnu_pbds;

template<class T>
using Tree=tree<T,null_type,less<T>,rb_tree_tag,
    tree_order_statistics_node_update>;

void example(){
  Tree<int>t,t2;t.insert(8);
  auto it=t.insert(10).first;
  assert(it==t.lower_bound(9));
  // t.order_of_key(x) = 严格小于 x 的元素个数
  assert(t.order_of_key(11)==2);
  // t.find_by_order(k) = 第 k 个元素, 从 0 开始
  assert(*t.find_by_order(0)==8);
  t.join(t2); // 需 t 全小于或全大于 t2, 把 t2 并入 t
}
```

### HashMap.h

**Description:** 哈希表, 接口与 unordered_map 基本相同, 但快约 3 倍, 内存约 1.5 倍. 若指定初始容量, 必须是 2 的幂.
**Usage:** `h[key] = val; h.erase(key);`
`auto it = h.find(key); // 不存在时为 h.end()`
d77092, 7 lines

```
#include <bits/extc++.h>
// 让高位也参与哈希, 而不只用低位:
struct chash{ // C 取大奇数
  const uint64_t C=ll(4e18*acos(0))|71;
  ll operator()(ll x)const{return __builtin_bswap64(x*C);}
};
__gnu_pbds::gp_hash_table<ll,int,chash>h({},{},{},{},{1<<16});
```

### LazySegmentTree.h

**Description:** 懒标记线段树, 下标 [0, n), n >= 1. 默认: 区间加, 查区间和与最大值. 换用途只改 T, U, idT, idU, f 和 Seg 里的 app, push, 写法见下方懒标记写法.
**Usage:** `vector<T> a(n, {x, x, 1}); Seg s(a);`
`s.update(l, r, {v}); T q = s.query(l, r); // 都是 [l, r)`
**Time:** $\mathcal{O}(\log N)$
8c7987, 47 lines

```
struct T{ll sum,mx;int len;}; // 节点信息
struct U{ll add;}; // 懒标记
const T idT={0,LLONG_MIN,0};
const U idU={0};
T f(T a,T b){
  return {a.sum+b.sum,max(a.mx,b.mx),a.len+b.len};
}

struct Seg{
  int n;vector<T>t;vector<U>d;
  Seg(const vector<T>&a):n(sz(a)),t(4*n),d(4*n,idU){
    build(1,0,n,a);
  }
  void build(int x,int l,int r,const vector<T>&a){
    if(r-l==1){t[x]=a[l];return;}
    int m=(l+r)/2;
    build(2*x,l,m,a),build(2*x+1,m,r,a);
    t[x]=f(t[2*x],t[2*x+1]);
  }
  void app(int x,U u){ // 整段 x 打上标记 u
    t[x].sum+=u.add*t[x].len,t[x].mx+=u.add;
    d[x].add+=u.add;
  }
  void push(int x){ // 有标记才下传给两个儿子
    if(!d[x].add)return;
    app(2*x,d[x]),app(2*x+1,d[x]),d[x]=idU;
  }
  void update(int L,int R,U u){upd(L,R,u,1,0,n);}
  void upd(int L,int R,U u,int x,int l,int r){
    if(R<=l||r<=L)return;
    if(L<=l&&r<=R)return app(x,u);
    push(x);
    int m=(l+r)/2;
    upd(L,R,u,2*x,l,m);
    upd(L,R,u,2*x+1,m,r);
    t[x]=f(t[2*x],t[2*x+1]);
  }
  T query(int L,int R){return qry(L,R,1,0,n);}
  T qry(int L,int R,int x,int l,int r){
    if(R<=l||r<=L)return idT;
    if(L<=l&&r<=R)return t[x];
    push(x);
    int m=(l+r)/2;
    return f(qry(L,R,2*x,l,m),
             qry(L,R,2*x+1,m,r));
  }
};
```

#### 懒标记写法

换用途时只改 $T, U$, `idT`, `idU`, $f$ 和 `Seg` 里的两个函数. `app(x, u)`: 给整段 $x$ 打上标记 $u$, 要更新 `t[x]`, 并把 $u$ 合并进 `d[x]` (旧标记先做, $u$ 后做). update 覆盖整段时调用它, push 给儿子时也调用它. `push(x)`: 有标记时对两个儿子各 app 一次, 再把 `d[x]` 清成 `idU`. 区间加就是上面的模板.

- **区间赋值, 求和/最大值**: 调用 `update(l, r, {v, 1})`.

```
struct U{ll v;bool has;};
const U idU={0,0};
void app(int x,U u){ // 整段赋成 u.v
  t[x].sum=u.v*t[x].len,t[x].mx=u.v;
  d[x]=u; // 新赋值直接覆盖旧标记
}
void push(int x){
  if(!d[x].has)return;
  app(2*x,d[x]),app(2*x+1,d[x]),d[x]=idU;
}
```

- **赋值和加混用**: 赋值 `{v, 1, 0}`, 加 `{0, 0, v}`.

```
// 标记含义: 若 has 先赋成 s, 再加 add
struct U{ll s;bool has;ll add;};
const U idU={0,0,0};
void app(int x,U u){
  T&a=t[x];U&e=d[x];
  if(u.has)
    a.sum=u.s*a.len,a.mx=u.s,e={u.s,1,0};
  a.sum+=u.add*a.len,a.mx+=u.add,e.add+=u.add;
}
void push(int x){
  if(!d[x].has&&!d[x].add)return;
  app(2*x,d[x]),app(2*x+1,d[x]),d[x]=idU;
}
```

- **区间乘再加, 求和取模**: 调用 `update(l, r, {a, b})`.

```
const ll M=998244353;
struct T{ll sum;int len;};struct U{ll a,b;};
const T idT={0,0};const U idU={1,0};
T f(T x,T y){
  return {(x.sum+y.sum)%M,x.len+y.len};
}
void app(int x,U u){ // 整段每个数 v 变成 a * v + b
  t[x].sum=(u.a*t[x].sum+u.b*t[x].len)%M;
  d[x]={u.a*d[x].a%M,(u.a*d[x].b+u.b)%M};
}
void push(int x){
  if(d[x].a==1&&!d[x].b)return;
  app(2*x,d[x]),app(2*x+1,d[x]),d[x]=idU;
}
```

- **01 序列区间取反, 数 1 的个数**: 调用 `update(l, r, {1})`.

```
struct T{int cnt,len;};struct U{bool r;};
const T idT={0,0};const U idU={0};
T f(T a,T b){return {a.cnt+b.cnt,a.len+b.len};}
void app(int x,U u){ // 整段 0 和 1 互换
  if(u.r)t[x].cnt=t[x].len-t[x].cnt,d[x].r^=1;
}
void push(int x){
  if(!d[x].r)return;
```

```
    app(2*x,d[x]),app(2*x+1,d[x]),d[x]=idU;
  }
```

- **最大子段和**: 只有单点赋值 `update(p, p + 1, {v})`, 不需要下传.

```
const ll NEG=-(ll)1e18;
struct T{ll sum,pre,suf,best;};
struct U{ll v;};
const T idT={0,NEG,NEG,NEG};
const U idU={0};
T f(T a,T b){
  return {a.sum+b.sum,max(a.pre,a.sum+b.pre),
          max(b.suf,b.sum+a.suf),
          max({a.best,b.best,a.suf+b.pre})};
}
void app(int x,U u){ // 只会落在叶子上
  t[x]={u.v,u.v,u.v,u.v};
}
void push(int){}
```

## PersistentSegmentTree.h

**Description:** 可持久化线段树, 下标 [0, n), 单点修改, 区间查询. upd 返回新根, 旧根仍然有效. 根 0 为空树. 按需修改 T, f, unit. kth 要求 f 为加法, 且版本 i 存的是 a[0..i) 中各值的出现次数.

**Usage:** `PST t(n); vi rt = {0};`
`rt.push_back(t.upd(rt.back(), p, v)); // a[p] = f(a[p], v)`
`t.query(rt[i], l, r); // 版本 i 中 [l, r) 的 f 值`
`t.kth(rt[l], rt[r], k); // a[l..r) 第 k 小, k 从 0 起`

**Time:** $\mathcal{O}(\log N)$

b28afc, 36 lines

```
struct PST{
  typedef ll T;
  static constexpr T unit=0;
  T f(T a,T b){return a+b;}
  struct N{T v;int l,r;};
  vector<N>t={{unit,0,0}};int n;
  PST(int n):n(n){}
  int upd(int x,int p,T v,int l=0,int r=-1){
    if(r<0)r=n;
    N c=t[x];
    if(r-l==1)c.v=f(c.v,v);
    else{
      int m=(l+r)/2;
      if(p<m)c.l=upd(c.l,p,v,l,m);
      else c.r=upd(c.r,p,v,m,r);
      c.v=f(t[c.l].v,t[c.r].v);
    }
    t.push_back(c);
    return sz(t)-1;
  }
  T query(int x,int b,int e,int l=0,int r=-1){
    if(r<0)r=n;
    if(!x||e<=l||r<=b)return unit;
    if(b<=l&&r<=e)return t[x].v;
    int m=(l+r)/2;
    return f(query(t[x].l,b,e,l,m),query(t[x].r,b,e,m,r));
  }
  int kth(int a,int b,T k,int l=0,int r=-1){
    if(r<0)r=n;
    if(r-l==1)return l;
    int m=(l+r)/2;
    T c=t[t[b].l].v-t[t[a].l].v;
    if(k<c)return kth(t[a].l,t[b].l,k,l,m);
    return kth(t[a].r,t[b].r,k-c,m,r);
  }
};
```

## UnionFindRollback.h

**Description:** 可撤销并查集 (按大小合并, 无路径压缩). 不需要撤销时可去掉 st, time() 和 rollback().

**Usage:** `RollbackUF uf(n); uf.join(a, b); uf.find(x); uf.size(x);`
`int t = uf.time(); ...; uf.rollback(t);`

**Time:** $\mathcal{O}(\log(N))$

de4ad0, 21 lines

```
struct RollbackUF{
  vi e;vector<pii>st;
  RollbackUF(int n):e(n,-1){}
  int size(int x){return -e[find(x)];}
  int find(int x){return e[x]<0?x:find(e[x]);}
  int time(){return sz(st);}
  void rollback(int t){
    for(int i=time();i-->t;)
      e[st[i].first]=st[i].second;
    st.resize(t);
  }
  bool join(int a,int b){
    a=find(a),b=find(b);
    if(a==b)return false;
    if(e[a]>e[b])swap(a,b);
    st.push_back({a,e[a]});
    st.push_back({b,e[b]});
    e[a]+=e[b];e[b]=a;
    return true;
  }
};
```

## LiChaoTree.h

**Description:** 李超树, 整数 x 取值于 [lo, hi). 节点动态开点, 任意范围都无需离散化. add 插入一条直线 $kx+m$, 或只在 [a, b) 上存在的线段. query 返回 x 处的最大值, 若没有线段覆盖 x 则返回 LLONG_MIN. 求最小值: 加入 $(-k,-m)$ 并对答案取负. 要求 $|kx+m|<9\cdot10^{18}$ 且 $C=hi-lo<9\cdot10^{18}$.

**Usage:** `LiChao lc(-1e9, 1e9 + 1);`
`lc.add({k, m}); lc.add({k, m}, a, b); lc.query(x);`

**Time:** 每条直线或每次查询 $\mathcal{O}(\log C)$, 每条线段 $\mathcal{O}(\log^2 C)$

bf1b50, 40 lines

```
struct LiChao{
  struct L{
    ll k,m;
    ll operator()(ll x)const{return k*x+m;}
  };
  struct N{L f{0,LLONG_MIN};int c[2]{};};
  vector<N>t;ll lo,hi;
  LiChao(ll lo,ll hi):t(1),lo(lo),hi(hi){}
  int ch(int x,int d){
    if(!t[x].c[d])t[x].c[d]=sz(t),t.emplace_back();
    return t[x].c[d];
  }
  void add(L f,ll a=LLONG_MIN,ll b=LLONG_MAX){
    ins(0,lo,hi,a,b,f);
  }
  void ins(int x,ll l,ll r,ll a,ll b,L f){
    if(b<=l||r<=a)return;
    ll m=l+(r-l)/2;
    if(a<=l&&r<=b){
      if(f(m)>t[x].f(m))swap(f,t[x].f);
      if(r-l==1||f.k==t[x].f.k)return;
      int d=f.k>t[x].f.k;
      ins(ch(x,d),d?m:l,d?r:m,a,b,f);
    }else{
      if(a<m)ins(ch(x,0),l,m,a,b,f);
      if(b>m)ins(ch(x,1),m,r,a,b,f);
    }
  }
  ll query(ll v){
    ll res=LLONG_MIN,l=lo,r=hi;
    for(int x=0;;){
      res=max(res,t[x].f(v));
      ll m=l+(r-l)/2;
      int d=v>=m;
      if(!t[x].c[d])return res;
      x=t[x].c[d];
      (d?l:r)=m;
    }
  }
};
```

## FHQTreap.h

**Description:** FHQ treap (split/merge, 无旋转). 节点 0 表示空. 两种用法共用 split/merge: 按值有序的 multiset, 或按位置的序列 (支持懒标记翻转和区间和, 下面的 b 即 [l, r) 段). 不要把翻转和按值操作混用.

**Usage:** `Treap T; T.insert(v); T.erase(v); T.rank(v); T.kth(k);`
`prev(v) = T.kth(T.rank(v) - 1), next(v) = T.kth(T.rank(v + 1));`
`T.rt = T.merge(T.rt, T.nn(v)); // 加到序列末尾`
`int a, b, c; T.split(T.rt, l, a, b); T.split(b, r - l, b, c);`
`T.t[b].rev ^= 1; T.t[b].sum; T.rt = T.merge(T.merge(a, b), c);`

**Time:** 每次操作期望 $\mathcal{O}(\log N)$

ce93b1, 75 lines

```
struct Treap{
  struct N{int l,r,s;unsigned p;ll v,sum;bool rev;};
  vector<N>t{{0,0,0,0,0,0,0}};
  mt19937 rng{12345};
  int rt=0;
  int nn(ll v){ // 新建节点
    t.push_back({0,0,1,(unsigned)rng(),v,v,0});
    return sz(t)-1;
  }
  void pull(int x){
    N&n=t[x];
    n.s=t[n.l].s+t[n.r].s+1;
    n.sum=t[n.l].sum+t[n.r].sum+n.v;
  }
  void push(int x){ // 下传翻转标记
    if(!t[x].rev)return;
    swap(t[x].l,t[x].r);
    t[t[x].l].rev^=1,t[t[x].r].rev^=1,t[x].rev=0;
  }
  void split(int x,int k,int&a,int&b){ // 前 k 个给 a
    if(!x){a=b=0;return;}
    push(x);
    int ls=t[t[x].l].s;
    if(ls<k)split(t[x].r,k-ls-1,t[x].r,b),a=x;
    else split(t[x].l,k,a,t[x].l),b=x;
    pull(x);
  }
  void splitv(int x,ll v,int&a,int&b){ // 值 < v 的给 a
    if(!x){a=b=0;return;}
    push(x);
    if(t[x].v<v)splitv(t[x].r,v,t[x].r,b),a=x;
    else splitv(t[x].l,v,a,t[x].l),b=x;
    pull(x);
  }
  int merge(int a,int b){ // a 的元素都在 b 前面
    if(!a||!b)return a^b;
    if(t[a].p>t[b].p){
      push(a),t[a].r=merge(t[a].r,b),pull(a);
      return a;
    }
    push(b),t[b].l=merge(a,t[b].l),pull(b);
    return b;
  }
  void insert(ll v){
    int n=nn(v),a,b;
    splitv(rt,v,a,b);
    rt=merge(merge(a,n),b);
  }
  void erase(ll v){ // 删除一个 v
```

```
    int a,b,c,d;
    splitv(rt,v,a,b),split(b,1,c,d);
    rt=merge(a,c&&t[c].v==v?d:merge(c,d));
  }
  int rank(ll v){ // 严格小于 v 的个数
    int a,b;
    splitv(rt,v,a,b);
    int r=t[a].s;
    rt=merge(a,b);
    return r;
  }
  ll kth(int k){ // 第 k 小 (k 从 0 开始)
    for(int x=rt;;){
      push(x);
      int ls=t[t[x].l].s;
      if(k==ls)return t[x].v;
      if(k<ls)x=t[x].l;
      else k-=ls+1,x=t[x].r;
    }
  }
  void walk(int x,vector<ll>&o){ // 中序遍历
    if(!x)return;
    push(x),walk(t[x].l,o);
    o.push_back(t[x].v),walk(t[x].r,o);
  }
};
```

## FenwickTree.h

**Description:** 树状数组: 求前缀和 a[0] + a[1] + ... + a[pos - 1], 单点修改 a[i] (传入新值与旧值之差). 区间加, 区间和 (都是 [l, r)) 用 RFT (两个 FT).
**Usage:** `FT t(n); t.update(i, d); t.query(r); t.lower_bound(s);`
`RFT f(n); f.add(l, r, v); f.query(l, r);`
**Time:** 两种操作均为 $\mathcal{O}(\log N)$.

"FenwickTree.h" 2fb67e, 36 lines

```
struct FT{
  vector<ll>s;
  FT(int n):s(n){}
  void update(int pos,ll dif){ // a[pos] += dif
    for(;pos<sz(s);pos|=pos+1)s[pos]+=dif;
  }
  ll query(int pos){ // [0, pos) 的和
    ll res=0;
    for(;pos>0;pos&=pos-1)res+=s[pos-1];
    return res;
  }
  // 最小的 pos 使 [0, pos] 的和不小于 sum
  int lower_bound(ll sum){
    // 全部和都不够则返回 n; sum 不大于 0 返回 -1
    if(sum<=0)return -1;
    int pos=0;
    for(int pw=1<<25;pw;pw>>=1){
      if(pos+pw<=sz(s)&&s[pos+pw-1]<sum)
        pos+=pw,sum-=s[pos-1];
    }
    return pos;
  }
};

struct RFT{ // 区间加, 区间和
  FT a,b;
  RFT(int n):a(n),b(n){}
  void add(int l,int r,ll v){ // [l, r) 都加 v
    a.update(l,v),a.update(r,-v);
    b.update(l,v*l),b.update(r,-v*r);
  }
  ll sum(int r){ // [0, r) 的和
    return a.query(r)*r-b.query(r);
  }
  ll query(int l,int r){return sum(r)-sum(l);}
};
```

## FenwickTree2d.h

**Description:** 二维树状数组: 求所有 $i < I, j < J$ 的 a[i,j] 之和, 单点增加 a[i,j]. 要求事先知道所有要修改的位置 (先对它们调用 fakeUpdate(), 再 init()).
**Usage:** `FT2 f(X); f.fakeUpdate(x, y); // 先登记所有修改点`
`f.init(); f.update(x, y, d); f.query(I, J);`
**Time:** $\mathcal{O}\left(\log^2 N\right)$. (用可持久化线段树可做到 $\mathcal{O}(\log N)$.)

"FenwickTree.h" 157f07, 22 lines

```
struct FT2{
  vector<vi>ys;vector<FT>ft;
  FT2(int limx):ys(limx){}
  void fakeUpdate(int x,int y){
    for(;x<sz(ys);x|=x+1)ys[x].push_back(y);
  }
  void init(){
    for(vi&v:ys)sort(all(v)),ft.emplace_back(sz(v));
  }
  int ind(int x,int y){
    return (int)(lower_bound(all(ys[x]),y)-ys[x].begin());}
  void update(int x,int y,ll dif){
    for(;x<sz(ys);x|=x+1)
      ft[x].update(ind(x,y),dif);
  }
  ll query(int x,int y){
    ll sum=0;
    for(;x;x&=x-1)
      sum+=ft[x-1].query(ind(x-1,y));
    return sum;
  }
};
```

## RMQ.h

**Description:** 数组区间最小值查询 (ST 表). 常数时间返回 min(V[a], V[a + 1], ... V[b - 1]).
**Usage:** `RMQ rmq(values);`
`rmq.query(a, b); // [a, b) 的最小值, 需 a < b`
**Time:** $\mathcal{O}(|V| \log |V| + Q)$

510c32, 16 lines

```
template<class T>
struct RMQ{
  vector<vector<T>>jmp;
  RMQ(const vector<T>&V):jmp(1,V){
    for(int pw=1,k=1;pw*2<=sz(V);pw*=2,++k){
      jmp.emplace_back(sz(V)-pw*2+1);
      rep(j,0,sz(jmp[k]))
        jmp[k][j]=min(jmp[k-1][j],jmp[k-1][j+pw]);
    }
  }
  T query(int a,int b){
    assert(a<b); // 或在 a 等于 b 时返回 inf
    int dep=31-__builtin_clz(b-a);
    return min(jmp[dep][a],jmp[dep][b-(1<<dep)]);
  }
};
```

## MoQueries.h

**Description:** 莫队: 回答区间或树上路径询问, 在端点处加入/删除点, 从一个询问移到下一个. 若值在树边上, 把 `step` 改为加入/删除边 $(a, c)$, 并去掉开头的 `add` 调用 (但保留 `in`).
**Usage:** `vi ans = mo(Q); // Q 为 vector<pii>, 区间 [L, R)`
`vi ans2 = moTree(P, ed, root); // P[i] = {u, v}, ed 为邻接表`
**Time:** $\mathcal{O}\left(N\sqrt{Q}\right)$

ac9594, 49 lines

```
void add(int ind,int end){...} // 加 a[ind] (end 为 0/1)
void del(int ind,int end){...} // 删除 a[ind]
int calc(){...} // 计算当前答案

vi mo(vector<pii>Q){
  int L=1,R=1,blk=350; // ~N/sqrt(Q)
  vi s(sz(Q)),res=s;
#define K(x) pii(x.first/blk, x.second ^ -(x.first/blk & 1))
  iota(all(s),0);
  sort(all(s),[&](int s,int t){return K(Q[s])<K(Q[t]);});
    for(int qi:s){
        auto[x,y]=Q[qi];
        while(L>x)add(--L,0);
        while(R<y)add(R++,1);
        while(L<x)del(L++,0);
        while(R>y)del(--R,1);
        res[qi]=calc();
    }
  return res; // Q 用左闭右开区间 [L, R)
}

vi moTree(vector<array<int,2>>Q,vector<vi>&ed,int root=0){
  int N=sz(ed),pos[2]={root,root},blk=126; // ~N/sqrt(Q)
  vi s(sz(Q)),res=s,I(N),L(N),R(N),in(N),par(N);
  add(root,0),in[root]=1;
  auto dfs=[&](int x,int p,int dep,auto&f)->void{
    par[x]=p;
    L[x]=N;
    if(dep)I[x]=N++;
    for(int y:ed[x])if(y!=p)f(y,x,!dep,f);
    if(!dep)I[x]=N++;
    R[x]=N;
  };
  dfs(root,-1,0,dfs);
#define K(x) pii(I[x[0]] / blk, I[x[1]] ^ -(I[x[0]] / blk & 1))
  iota(all(s),0);
  sort(all(s),[&](int s,int t){return K(Q[s])<K(Q[t]);});
  for(int qi:s)rep(end,0,2){
    int&a=pos[end],b=Q[qi][end],i=0;
#define step(c) { if (in[c]) { del(a, end); in[a] = 0; } \
                  else{add(c,end);in[c]=1;}a=c;}
    while(!(L[b]<=L[a]&&R[a]<=R[b]))
      I[i++]=b,b=par[b];
    while(a!=b)step(par[a]);
    while(i--)step(I[i]);
    if(end)res[qi]=calc();
  }
  return res; // Q 用闭区间 [L, R]
}
```

# Numerical (4)

## 4.1 Polynomials and recurrences

## Polynomial.h

**Description:** 实系数多项式, $a[i]$ 为 $x^i$ 的系数. `p(x)` 用 Horner 法求值; `diff()` 原地求导; `divroot(x0)` 原地除以 $(x - x_0)$ (余数丢弃, $x_0$ 应为根).
**Usage:** `Poly p{{2,-3,1}}; // a[i] 为 x^i 的系数`
`double y = p(0.5); p.divroot(1); // 除以 (x - 1)`
`p.diff(); // 原地求导`

c9b7b0, 17 lines

```
struct Poly{
  vector<double>a;
  double operator()(double x)const{
    double val=0;
    for(int i=sz(a);i--;)(val*=x)+=a[i];
    return val;
  }
  void diff(){
    rep(i,1,sz(a))a[i-1]=i*a[i];
    a.pop_back();
```

```
  }
  void divroot(double x0){
    double b=a.back(),c;a.back()=0;
    for(int i=sz(a)-1;i--;)c=a[i],a[i]=a[i+1]*x0+b,b=c;
    a.pop_back();
  }
};
```

## PolyRoots.h

**Description:** 求多项式在区间 $[xmin, xmax]$ 内的全部实根 (递归求导函数的根, 在单调段上二分).
**Usage:** `polyRoots({{2,-3,1}},-1e9,1e9) // 解 x^2-3x+2 = 0`
**Time:** $\mathcal{O}\left(n^2 \log(1/\epsilon)\right)$

"Polynomial.h" b00bfe, 23 lines

```
vector<double>polyRoots(Poly p,double xmin,double xmax){
  if(sz(p.a)==2){return {-p.a[0]/p.a[1]};}
  vector<double>ret;
  Poly der=p;
  der.diff();
  auto dr=polyRoots(der,xmin,xmax);
  dr.push_back(xmin-1);
  dr.push_back(xmax+1);
  sort(all(dr));
  rep(i,0,sz(dr)-1){
    double l=dr[i],h=dr[i+1];
    bool sign=p(l)>0;
    if(sign^(p(h)>0)){
      rep(it,0,60){ // while (h - l > 1e-8)
        double m=(l+h)/2,f=p(m);
        if((f<=0)^sign)l=m;
        else h=m;
      }
      ret.push_back((l+h)/2);
    }
  }
  return ret;
}
```

## PolyInterpolate.h

**Description:** 给定 $n$ 个点 (x[i], y[i]), 求经过它们的 $n-1$ 次多项式 $p$: $p(x) = a[0] * x^0 + ... + a[n-1] * x^{n-1}$. 为保证数值精度, 可取 $x[k] = c * \cos(k/(n-1) * \pi), k = 0 \dots n-1$.
**Usage:** `vd a = interpolate(x, y, n); // a[i] 为 x^i 的系数`
**Time:** $\mathcal{O}\left(n^2\right)$

08bf48, 13 lines

```
typedef vector<double>vd;
vd interpolate(vd x,vd y,int n){
  vd res(n),temp(n);
  rep(k,0,n-1)rep(i,k+1,n)
    y[i]=(y[i]-y[k])/(x[i]-x[k]);
  double last=0;temp[0]=1;
  rep(k,0,n)rep(i,0,n){
    res[i]+=y[k]*temp[i];
    swap(last,temp[i]);
    temp[i]-=last*x[k];
  }
  return res;
}
```

## BerlekampMassey.h

**Description:** 由数列的前 $2n$ 项恢复出任意 $n$ 阶线性递推关系 (此处在模素数 mod 下计算). 返回 $C$, 满足 $s[i] = \sum_j C[j]s[i-j-1]$. 常用于暴力算出前若干项后猜线性递推式. 理论上适用于任意域, 但用浮点数时不保证数值稳定. 输出长度 $\le n$.
**Usage:** `berlekampMassey({0, 1, 1, 3, 5, 11}) // 得 {1, 2}`
**Time:** $\mathcal{O}\left(N^2\right)$

"../number-theory/ModPow.h" 96548b, 20 lines

```
vector<ll>berlekampMassey(vector<ll>s){
  int n=sz(s),L=0,m=0;
  vector<ll>C(n),B(n),T;
  C[0]=B[0]=1;

  ll b=1;
  rep(i,0,n){++m;
    ll d=s[i]%mod;
    rep(j,1,L+1)d=(d+C[j]*s[i-j])%mod;
    if(!d)continue;
    T=C;ll coef=d*modpow(b,mod-2)%mod;
    rep(j,m,n)C[j]=(C[j]-coef*B[j-m])%mod;
    if(2*L>i)continue;
    L=i+1-L;B=T;b=d;m=0;
  }

  C.resize(L+1);C.erase(C.begin());
  for(ll&x:C)x=(mod-x)%mod;
  return C;
}
```

## LinearRecurrence.h

**Description:** 给定 $S[0 \dots \ge n-1]$ 与 $tr[0 \dots n-1]$, 求 $n$ 阶线性递推 $S[i] = \sum_j S[i-j-1]tr[j]$ 的第 $k$ 项 ($k$ 从 0 开始, 模 mod). 比矩阵快速幂更快. 常与 Berlekamp–Massey 配合使用.
**Usage:** `linearRec({0, 1}, {1, 1}, k) // 第 k 个 Fibonacci 数`
**Time:** $\mathcal{O}\left(n^2 \log k\right)$

47555d, 27 lines

```
typedef vector<ll>Poly;
ll linearRec(Poly S,Poly tr,ll k){
  int n=sz(tr);
  if(n==0)return S[0];

  auto combine=[&](Poly a,Poly b){
    Poly res(n*2+1);
    rep(i,0,n+1)rep(j,0,n+1)
      res[i+j]=(res[i+j]+a[i]*b[j])%mod;
    for(int i=2*n;i>n;--i)rep(j,0,n)
      res[i-1-j]=(res[i-1-j]+res[i]*tr[j])%mod;
    res.resize(n+1);
    return res;
  };

  Poly pol(n+1),e(pol);
  pol[0]=e[1]=1;

  for(++k;k;k/=2){
    if(k%2)pol=combine(pol,e);
    e=combine(e,e);
  }

  ll res=0;
  rep(i,0,n)res=(res+pol[i+1]*S[i])%mod;
  return res;
}
```

# 4.2 Optimization

## GoldenSectionSearch.h

**Description:** 在区间 $[a, b]$ 上求使函数 $f$ 取最小值的自变量, 要求 $f$ 在该区间上单峰, 即只有一个局部极小值且没有局部极大值. 结果的最大误差为 $eps$. 稍改代码即可同样 用于求最大值. 离散版本见 Various 章的 TernarySearch.h.
**Usage:** `double func(double x) { return 4+x+.3*x*x; }`
`double xmin = gss(-1000,1000,func); // 最小值点`
**Time:** $\mathcal{O}\left(\log((b-a)/\epsilon)\right)$

31d45b, 14 lines

```
double gss(double a,double b,double(*f)(double)){
  double r=(sqrt(5)-1)/2,eps=1e-7;
  double x1=b-r*(b-a),x2=a+r*(b-a);
  double f1=f(x1),f2=f(x2);
  while(b-a>eps)
    if(f1<f2){ // 求最大值则改为 >
      b=x2;x2=x1;f2=f1;
      x1=b-r*(b-a);f1=f(x1);
    }else{
      a=x1;x1=x2;f1=f2;
      x2=a+r*(b-a);f2=f(x2);
    }
  return a;
}
```

## IntegrateAdaptive.h

**Description:** 用自适应 Simpson 法快速计算数值积分.
**Usage:** `// 三重积分求单位球体积`

```
double sphereVolume = quad(-1, 1, [](double x) {
return quad(-1, 1, [&](double y) {
return quad(-1, 1, [&](double z) {
return x*x + y*y + z*z < 1; });});});
```

92dd79, 15 lines

```
typedef double d;
#define S(a,b) (f(a) + 4*f((a+b) / 2) + f(b)) * (b-a) / 6

template<class F>
d rec(F&f,d a,d b,d eps,d S){
  d c=(a+b)/2;
  d S1=S(a,c),S2=S(c,b),T=S1+S2;
  if(abs(T-S)<=15*eps||b-a<1e-10)
    return T+(T-S)/15;
  return rec(f,a,c,eps/2,S1)+rec(f,c,b,eps/2,S2);
}
template<class F>
d quad(d a,d b,F f,d eps=1e-8){
  return rec(f,a,b,eps,S(a,b));
}
```

## Simplex.h

**Description:** 求解一般线性规划 (最大化): 在 $Ax \le b$, $x \ge 0$ 约束下最大化 $c^T x$. 无解返回 -inf, 目标可任意大 (无界) 返回 inf, 否则返回 $c^T x$ 的最大值. 传入的向量 x 会被设为一个最优解 (无界时为任意一个满足约束的解). 不保证数值稳定. 为提高效率, 应让 $x = 0$ 是可行解.
**Usage:** `vvd A = {{1,-1}, {-1,1}, {-1,-2}};`
`vd b = {1,1,-4}, c = {-1,-1}, x;`
`T val = LPSolver(A, b, c).solve(x); // x 存最优解`
**Time:** $\mathcal{O}(NM * \#pivots)$, 一次 pivot 可类比一次边松弛. 最坏情况 $\mathcal{O}(2^n)$.

aa8530, 68 lines

```
typedef double T; // 或 long double, Rational, double+mod<P>
typedef vector<T>vd;
typedef vector<vd>vvd;

const T eps=1e-8,inf=1/.0;
#define MP make_pair
#define ltj(X) if(s == -1 || MP(X[j],N[j]) < MP(X[s],N[s])) s=j

struct LPSolver{
  int m,n;
  vi N,B;
  vvd D;

  LPSolver(const vvd&A,const vd&b,const vd&c):
    m(sz(b)),n(sz(c)),N(n+1),B(m),D(m+2,vd(n+2)){
      rep(i,0,m)rep(j,0,n)D[i][j]=A[i][j];
      rep(i,0,m){B[i]=n+i;D[i][n]=-1;D[i][n+1]=b[i];}
      rep(j,0,n){N[j]=j;D[m][j]=-c[j];}
      N[n]=-1;D[m+1][n]=1;
    }

  void pivot(int r,int s){
    T*a=D[r].data(),inv=1/a[s];
```

```
    rep(i,0,m+2)if(i!=r&&abs(D[i][s])>eps){
      T*b=D[i].data(),inv2=b[s]*inv;
      rep(j,0,n+2)b[j]-=a[j]*inv2;
      b[s]=a[s]*inv2;
    }
    rep(j,0,n+2)if(j!=s)D[r][j]*=inv;
    rep(i,0,m+2)if(i!=r)D[i][s]*=-inv;
    D[r][s]=inv;
    swap(B[r],N[s]);
  }

  bool simplex(int phase){
    int x=m+phase-1;
    for(;;){
      int s=-1;
      rep(j,0,n+1)if(N[j]!=-phase)ltj(D[x]);
      if(D[x][s]>=-eps)return true;
      int r=-1;
      rep(i,0,m){
        if(D[i][s]<=eps)continue;
        if(r==-1||MP(D[i][n+1]/D[i][s],B[i])
                    <MP(D[r][n+1]/D[r][s],B[r]))r=i;
      }
      if(r==-1)return false;
      pivot(r,s);
    }
  }

  T solve(vd&x){
    int r=0;
    rep(i,1,m)if(D[i][n+1]<D[r][n+1])r=i;
    if(D[r][n+1]<-eps){
      pivot(r,n);
      if(!simplex(2)||D[m+1][n+1]<-eps)return -inf;
      rep(i,0,m)if(B[i]==-1){
        int s=0;
        rep(j,1,n+1)ltj(D[i]);
        pivot(i,s);
      }
    }
    bool ok=simplex(1);x=vd(n);
    rep(i,0,m)if(B[i]<n)x[B[i]]=D[i][n+1];
    return ok?D[m][n+1]:inf;
  }
};
```

## 4.3 Matrices

### Determinant.h

**Description:** 计算实矩阵的行列式 (列主元高斯消元). 会破坏原矩阵.
**Usage:** `vector<vector<double>> a(n, vector<double>(n));`
`double d = det(a); // a 会被修改`
**Time:** $\mathcal{O}\left(N^3\right)$ bd5cec, 15 lines

```
double det(vector<vector<double>>&a){
  int n=sz(a);double res=1;
  rep(i,0,n){
    int b=i;
    rep(j,i+1,n)if(fabs(a[j][i])>fabs(a[b][i]))b=j;
    if(i!=b)swap(a[i],a[b]),res*=-1;
    res*=a[i][i];
    if(res==0)return 0;
    rep(j,i+1,n){
      double v=a[j][i]/a[i][i];
      if(v!=0)rep(k,i+1,n)a[j][k]-=v*a[i][k];
    }
  }
  return res;
}
```

### IntDeterminant.h

**Description:** 在模 `mod` 意义下计算整数矩阵的行列式 (辗转相除消元, 模数不必为素数). 去掉取模即得纯整数版本. 会破坏原矩阵.
**Usage:** `vector<vector<ll>> a(n, vector<ll>(n));`
`ll d = det(a); // a 会被修改`
**Time:** $\mathcal{O}\left(N^3\right)$ 3313dc, 18 lines

```
const ll mod=12345;
ll det(vector<vector<ll>>&a){
  int n=sz(a);ll ans=1;
  rep(i,0,n){
    rep(j,i+1,n){
      while(a[j][i]!=0){ // 辗转相除
        ll t=a[i][i]/a[j][i];
        if(t)rep(k,i,n)
          a[i][k]=(a[i][k]-a[j][k]*t)%mod;
        swap(a[i],a[j]);
        ans*=-1;
      }
    }
    ans=ans*a[i][i]%mod;
    if(!ans)return 0;
  }
  return (ans+mod)%mod;
}
```

### SolveLinear.h

**Description:** 求解 $A*x=b$. 若有多解, 返回任意一个. 返回秩, 无解返回 -1. $A$ 与 $b$ 的数据会被破坏.
**Usage:** `vector<vd> A(n, vd(m)); vd b(n), x(m); // x 须预设长度 m`
`int r = solveLinear(A, b, x); // 无解返回 -1`
**Time:** $\mathcal{O}\left(n^2m\right)$ 44c9ab, 38 lines

```
typedef vector<double>vd;
const double eps=1e-12;

int solveLinear(vector<vd>&A,vd&b,vd&x){
  int n=sz(A),m=sz(x),rank=0,br,bc;
  if(n)assert(sz(A[0])==m);
  vi col(m);iota(all(col),0);

  rep(i,0,n){
    double v,bv=0;
    rep(r,i,n)rep(c,i,m)
      if((v=fabs(A[r][c]))>bv)
        br=r,bc=c,bv=v;
    if(bv<=eps){
      rep(j,i,n)if(fabs(b[j])>eps)return -1;
      break;
    }
    swap(A[i],A[br]);
    swap(b[i],b[br]);
    swap(col[i],col[bc]);
    rep(j,0,n)swap(A[j][i],A[j][bc]);
    bv=1/A[i][i];
    rep(j,i+1,n){
      double fac=A[j][i]*bv;
      b[j]-=fac*b[i];
      rep(k,i+1,m)A[j][k]-=fac*A[i][k];
    }
    rank++;
  }

  x.assign(m,0);
  for(int i=rank;i--;){
    b[i]/=A[i][i];
    x[col[i]]=b[i];
    rep(j,0,i)b[j]-=A[j][i]*b[i];
  }
  return rank; // rank < m 时有多解
}
```

### SolveLinearBinary.h

**Description:** 在 $\mathbb{F}_2$ 上求解 $Ax=b$, $m$ 为未知数个数. 若有多解, 任意返回一个. 返回秩, 无解返回 -1. 会破坏 $A$ 与 $b$.
**Usage:** `vector<bs> A(n); vi b(n); bs x;`
`int r = solveLinear(A, b, x, m); // m 为未知数个数`
**Time:** $\mathcal{O}\left(n^2m\right)$ fa2d7a, 34 lines

```
typedef bitset<1000>bs;

int solveLinear(vector<bs>&A,vi&b,bs&x,int m){
  int n=sz(A),rank=0,br;
  assert(m<=sz(x));
  vi col(m);iota(all(col),0);
  rep(i,0,n){
    for(br=i;br<n;++br)if(A[br].any())break;
    if(br==n){
      rep(j,i,n)if(b[j])return -1;
      break;
    }
    int bc=(int)A[br]._Find_next(i-1);
    swap(A[i],A[br]);
    swap(b[i],b[br]);
    swap(col[i],col[bc]);
    rep(j,0,n)if(A[j][i]!=A[j][bc]){
      A[j].flip(i);A[j].flip(bc);
    }
    rep(j,i+1,n)if(A[j][i]){
      b[j]^=b[i];
      A[j]^=A[i];
    }
    rank++;
  }

  x=bs();
  for(int i=rank;i--;){
    if(!b[i])continue;
    x[col[i]]=1;
    rep(j,0,i)b[j]^=A[j][i];
  }
  return rank; // rank < m 时有多解
}
```

### MatrixInverse.h

**Description:** 求矩阵 $A$ 的逆. 返回秩; 除非 $A$ 奇异 (秩 $< n$), 结果存回 $A$. 易推广到素数模; 对素数幂模, 以 A 模 p 的逆作为 $A^{-1}$ 的初值, 反复令 $A^{-1}=A^{-1}(2I-AA^{-1})\pmod{p^k}$, 每步 k 翻倍.
**Usage:** `vector<vector<double>> A(n, vector<double>(n));`
`int r = matInv(A); // r 等于 n 时 A 变为逆矩阵`
**Time:** $\mathcal{O}\left(n^3\right)$ ebfff6, 35 lines

```
int matInv(vector<vector<double>>&A){
  int n=sz(A);vi col(n);
  vector<vector<double>>tmp(n,vector<double>(n));
  rep(i,0,n)tmp[i][i]=1,col[i]=i;

  rep(i,0,n){
    int r=i,c=i;
    rep(j,i,n)rep(k,i,n)
      if(fabs(A[j][k])>fabs(A[r][c]))
        r=j,c=k;
    if(fabs(A[r][c])<1e-12)return i;
    A[i].swap(A[r]);tmp[i].swap(tmp[r]);
    rep(j,0,n)
      swap(A[j][i],A[j][c]),swap(tmp[j][i],tmp[j][c]);
```

```
    swap(col[i],col[c]);
    double v=A[i][i];
    rep(j,i+1,n){
      double f=A[j][i]/v;
      A[j][i]=0;
      rep(k,i+1,n)A[j][k]-=f*A[i][k];
      rep(k,0,n)tmp[j][k]-=f*tmp[i][k];
    }
    rep(j,i+1,n)A[i][j]/=v;
    rep(j,0,n)tmp[i][j]/=v;
    A[i][i]=1;
  }

  for(int i=n-1;i>0;--i)rep(j,0,i){
    double v=A[j][i];
    rep(k,0,n)tmp[j][k]-=v*tmp[i][k];
  }

  rep(i,0,n)rep(j,0,n)A[col[i]][col[j]]=tmp[i][j];
  return n;
}
```

## Tridiagonal.h

**Description:** $x = \text{tridiagonal}(d, p, q, b)$ 求解方程组

$$\begin{pmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_{n-1} \end{pmatrix} = \begin{pmatrix} d_0 & p_0 & 0 & 0 & \cdots & 0 \\ q_0 & d_1 & p_1 & 0 & \cdots & 0 \\ 0 & q_1 & d_2 & p_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & q_{n-3} & d_{n-2} & p_{n-2} \\ 0 & 0 & \cdots & 0 & q_{n-2} & d_{n-1} \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{n-1} \end{pmatrix}.$$

可用于求解如下形式的问题

$$a_i = b_i a_{i-1} + c_i a_{i+1} + d_i,\ 1 \le i \le n,$$

其中 $a_0$, $a_{n+1}$, $b_i$, $c_i$ 与 $d_i$ 已知. 则 $a$ 可由下式求得

$$\{a_i\} = \text{tridiagonal}(\{1,-1,-1,...,-1,1\}, \{0,c_1,c_2,\ldots,c_n\}, \{b_1,b_2,\ldots,b_n,0\}, \{a_0,d_1,d_2,\ldots,d_n,a_{n+1}\}).$$

解不唯一时会失败.
若对所有 $i$ 有 $|d_i| > |p_i| + |q_{i-1}|$, 或 $|d_i| > |p_{i-1}| + |q_i|$, 或矩阵正定, 则算法数值稳定, 不需要 `tr` 以及对 `diag[i] == 0` 的检查.
**Usage:** `vector<T> x = tridiagonal(d, p, q, b); // 记号同上`
**Time:** $\mathcal{O}(N)$

8f9fa8, 26 lines

```
typedef double T;
vector<T>tridiagonal(vector<T>diag,const vector<T>&super,
    const vector<T>&sub,vector<T>b){
  int n=sz(b);vi tr(n);
  rep(i,0,n-1){
    if(abs(diag[i])<1e-9*abs(super[i])){ // diag[i] == 0
      b[i+1]-=b[i]*diag[i+1]/super[i];
      if(i+2<n)b[i+2]-=b[i]*sub[i+1]/super[i];
      diag[i+1]=sub[i];tr[++i]=1;
    }else{
      diag[i+1]-=super[i]*sub[i]/diag[i];
      b[i+1]-=b[i]*sub[i]/diag[i];
    }
  }
  for(int i=n;i--;){
    if(tr[i]){
      swap(b[i],b[i-1]);
      diag[i-1]=diag[i];
      b[i]/=super[i-1];
    }else{
      b[i]/=diag[i];
      if(i)b[i-1]-=b[i]*super[i-1];
    }
  }
  return b;
}
```

# 4.4 Fourier transforms

## FastFourierTransform.h

**Description:** fft(a) 对所有 $k$ 计算 $\hat{f}(k) = \sum_x a[x]\exp(2\pi i \cdot kx/N)$. N 必须是 2 的幂. 用于卷积: `conv(a, b) = c`, 其中 $c[x] = \sum a[i]b[x-i]$. 对复数或多于两个向量做卷积: 先 FFT, 逐点相乘, 除以 n, reverse(start+1, end), 再 FFT 回来. 若 $(\sum a_i^2 + \sum b_i^2)\log_2 N < 9 \cdot 10^{14}$ 则取整是安全的 (实际约 $10^{16}$; 随机输入可更高). 否则用 NTT/FFTMod.
**Usage:** `vd c = conv(a, b); // 实数卷积, 长度 sz(a)+sz(b)-1`
`vector<C> v(n); fft(v); // n 须为 2 的幂`
**Time:** $\mathcal{O}(N \log N)$, 其中 $N = |A| + |B|$ ($N = 2^{22}$ 时约 $\sim 1s$)

00ced6, 35 lines

```
typedef complex<double>C;
typedef vector<double>vd;
void fft(vector<C>&a){
  int n=sz(a),L=31-__builtin_clz(n);
  static vector<complex<long double>>R(2,1);
  static vector<C>rt(2,1); // (^ 用 double 快 10%)
  for(static int k=2;k<n;k*=2){
    R.resize(n);rt.resize(n);
    auto x=polar(1.0L,acos(-1.0L)/k);
    rep(i,k,2*k)rt[i]=R[i]=i&1?R[i/2]*x:R[i/2];
  }
  vi rev(n);
  rep(i,0,n)rev[i]=(rev[i/2]|(i&1)<<L)/2;
  rep(i,0,n)if(i<rev[i])swap(a[i],a[rev[i]]);
  for(int k=1;k<n;k*=2)
    for(int i=0;i<n;i+=2*k)rep(j,0,k){
      C z = rt[j+k] * a[i+j+k]; // 手写展开快 25%
      a[i+j+k]=a[i+j]-z;
      a[i+j]+=z;
    }
}
vd conv(const vd&a,const vd&b){
  if(a.empty()||b.empty())return {};
  vd res(sz(a)+sz(b)-1);
  int L=32-__builtin_clz(sz(res)),n=1<<L;
  vector<C>in(n),out(n);
  copy(all(a),begin(in));
  rep(i,0,sz(b))in[i].imag(b[i]);
  fft(in);
  for(C&x:in)x*=x;
  rep(i,0,n)out[i]=in[-i&(n-1)]-conj(in[i]);
  fft(out);
  rep(i,0,sz(res))res[i]=imag(out[i])/(4*n);
  return res;
}
```

## FastFourierTransformMod.h

**Description:** 更高精度的 FFT, 可用于模任意整数的卷积, 只要 $N\log_2 N \cdot \text{mod} < 8.6 \cdot 10^{14}$ (实际可到 $10^{16}$ 或更高). 输入须在 $[0, \text{mod})$ 内.
**Usage:** `vl c = convMod<1000000007>(a, b); // a, b 为 vl`
**Time:** $\mathcal{O}(N \log N)$, 其中 $N = |A| + |B|$ (比 NTT 或 FFT 慢一倍)

"FastFourierTransform.h" b82773, 22 lines

```
typedef vector<ll>vl;
template<int M>vl convMod(const vl&a,const vl&b){
  if(a.empty()||b.empty())return {};
  vl res(sz(a)+sz(b)-1);
  int B=32-__builtin_clz(sz(res)),n=1<<B,cut=int(sqrt(M));
  vector<C>L(n),R(n),outs(n),outl(n);
  rep(i,0,sz(a))L[i]=C((int)a[i]/cut,(int)a[i]%cut);
  rep(i,0,sz(b))R[i]=C((int)b[i]/cut,(int)b[i]%cut);
  fft(L),fft(R);
  rep(i,0,n){
    int j=-i&(n-1);
    outl[j]=(L[i]+conj(L[j]))*R[i]/(2.0*n);
    outs[j]=(L[i]-conj(L[j]))*R[i]/(2.0*n)/1i;
  }
  fft(outl),fft(outs);
  rep(i,0,sz(res)){
    ll av=ll(real(outl[i])+.5),cv=ll(imag(outs[i])+.5);
    ll bv=ll(imag(outl[i])+.5)+ll(real(outs[i])+.5);
    res[i]=((av%M*cut+bv)%M*cut+cv)%M;
  }
  return res;
}
```

## NumberTheoreticTransform.h

**Description:** ntt(a) 对所有 $k$ 计算 $\hat{f}(k) = \sum_x a[x]g^{xk}$, 其中 $g = \text{root}^{(mod-1)/N}$. N 必须是 2 的幂. 用于模形如 $2^a b + 1$ 的特定素数下的卷积, 要求卷积结果长度不超过 $2^a$. 任意模数见 FFTMod. `conv(a, b) = c`, 其中 $c[x] = \sum a[i]b[x-i]$. 手动卷积: 对输入做 NTT, 逐点相乘, 除以 n, reverse(start+1, end), 再 NTT 回来. 输入须在 [0, mod) 内.
**Usage:** `vl c = conv(a, b); // 模 998244353 卷积`
**Time:** $\mathcal{O}(N \log N)$

"../number-theory/ModPow.h" ced03d, 35 lines

```
const ll mod=(119<<23)+1,root=62; // = 998244353
// p < 2^30 时还可用 5 << 25, 7 << 26, 479 << 21
// 以及 483 << 21 (root 相同), 后两个大于 10^9
typedef vector<ll>vl;
void ntt(vl&a){
  int n=sz(a),L=31-__builtin_clz(n);
  static vl rt(2,1);
  for(static int k=2,s=2;k<n;k*=2,s++){
    rt.resize(n);
    ll z[]={1,modpow(root,mod>>s)};
    rep(i,k,2*k)rt[i]=rt[i/2]*z[i&1]%mod;
  }
  vi rev(n);
  rep(i,0,n)rev[i]=(rev[i/2]|(i&1)<<L)/2;
  rep(i,0,n)if(i<rev[i])swap(a[i],a[rev[i]]);
  for(int k=1;k<n;k*=2)
    for(int i=0;i<n;i+=2*k)rep(j,0,k){
      ll z=rt[j+k]*a[i+j+k]%mod,&ai=a[i+j];
      a[i+j+k]=ai-z+(z>ai?mod:0);
      ai+=(ai+z>=mod?z-mod:z);
    }
}
vl conv(const vl&a,const vl&b){
  if(a.empty()||b.empty())return {};
  int s=sz(a)+sz(b)-1,B=32-__builtin_clz(s),
      n=1<<B;
  int inv=modpow(n,mod-2);
  vl L(a),R(b),out(n);
  L.resize(n),R.resize(n);
  ntt(L),ntt(R);
  rep(i,0,n)
    out[-i&(n-1)]=(ll)L[i]*R[i]%mod*inv%mod;
  ntt(out);
  return {out.begin(),out.begin()+s};
}
```

## FastSubsetTransform.h

**Description:** 变换到一组基, 使得形如 $c[z] = \sum_{z=x\oplus y} a[x]\cdot b[y]$ 的卷积可快速计算, 其中 $\oplus$ 为 AND, OR, XOR 之一 (默认 XOR, 其余换用注释中的行). $a$ 的长度必须是 2 的幂. point_add / point_eval 在变换后的数组上对原数组单点加 / 单点求值.
**Usage:** `vl c = conv(a, b); // a, b 等长, 长度为 2 的幂`
`FST(a, 0); point_add(a, i, v); ll x = point_eval(a, i);`
**Time:** $\mathcal{O}(N \log N)$, 单点操作 $\mathcal{O}(N)$

3de923, 35 lines

```
using vl=vector<ll>;
void FST(vl&a,bool inv){
    for(int n=sz(a),step=1;step<n;step*=2){
        for(int i=0;i<n;i+=2*step)rep(j,i,i+step){
            ll&u=a[j],&v=a[j+step];tie(u,v)=
                inv ? pair(v - u, u) : pair(v, u + v); // AND
                inv ? pair(v, u - v) : pair(u + v, u); // OR
                pair(u+v,u-v); // XOR
        }
    }
    if(inv)for(ll&x:a)x/=sz(a); // XOR
}
vl conv(vl a,vl b){
    FST(a,0);FST(b,0);
    rep(i,0,sz(a))a[i]*=b[i];
    FST(a,1);return a;
}
int psgn(int x){return __builtin_popcount(x)&1?-1:1;}
// 原数组 a[i] += val, 此处 a 已变换
void point_add(vl&a,int i,ll val){
    rep(j,0,sz(a))
        a[j] += val * ((i|j) == sz(a)-1); // AND
        a[j] += val * ((i&j) == 0); // OR
        a[j]+=val*psgn(i&j); // XOR
}
// 返回原数组 a[i], 此处 a 已变换
ll point_eval(const vl&a,int i){
    ll res=0;
    rep(j,0,sz(a))
        res += a[j] * ((i&j) == 0) * psgn(sz(a)-1-(i|j)); //
            AND
        res += a[j] * ((i|j) == sz(a)-1) * psgn(i&j); // OR
        res+=a[j]*psgn(i&j); // XOR
    res/=sz(a); // XOR
    return res;
}
```

### MaxPlusConvolution.h

**Description:** 给定数组 $a$, $b$, 其中 $b$ 是凹的, 返回数组 $c_k = \max_{i+j=k}(a_i + b_j)$, 长度与 $a$ 相同. $b$ 的开头隐式插入一个 0.
**Usage:** `vll c = maxPlusConv(a, b); // b 须为凹数组`
**Time:** $\mathcal{O}(N \log N)$

709c52, 44 lines

```
using vll=vector<ll>;
vll maxPlusConv(const vll&a,const vll&b){
  if(b.empty())return a;
  using Seg=array<ll,3>;
  auto eval=[&](Seg&seg,ll x){
    ll offset=x-seg[0];
    assert(0<=seg[1]+offset&&seg[1]+offset<sz(b));
    return b[seg[1]+offset]+seg[2];
  };
  auto endIndex=[&](Seg&seg){
    return seg[0]+sz(b)-seg[1];
  };
  vector<Seg>stk;
  rep(i,0,sz(a)){
    Seg seg{i+1,0,a[i]};
    while(!stk.empty()){
      ll idx=stk.back()[0];
      if(idx<seg[0]||eval(stk.back(),idx)>eval(seg,idx))
        break;
      stk.pop_back();
    }
    if(!stk.empty()&&endIndex(stk.back())>seg[0]
        &&eval(stk.back(),seg[0])>eval(seg,seg[0])){
      ll lo=seg[0];
      ll hi=min(endIndex(stk.back()),endIndex(seg));
      while(lo+1<hi){
        ll mid=midpoint(lo,hi);
        if(eval(stk.back(),mid)>eval(seg,mid))lo=mid;
        else hi=mid;
      }
      seg[1]=hi-seg[0];
      seg[0]=hi;
    }
    stk.push_back(seg);
  }
  vll ans=a;
  auto it=stk.begin();
  rep(i,1,sz(ans)){
    while(next(it)!=stk.end()&&(*next(it))[0]<=i)
      ++it;
    ans[i]=max(ans[i],eval(*it,i));
  }
  return ans;
}
```

# Number theory (5)

## 5.1 Modular arithmetic

### ModularArithmetic.h

**Description:** 模运算。假设模数 $MOD$ 为质数。
**Usage:** `using mi = mint<MOD>; // MOD = 1e9+7`
`mi a = MOD+5; inv(a); // 400000003`

0db02d, 27 lines

```
template<int M>struct mint{
    static const int mod=M;
    int v;
    explicit operator int()const{return v;}
    mint():v(0){}
    mint(ll _v):v(_v%M){v+=(v<0)*M;}
    mint&operator+=(mint o){
        if((v+=o.v)>=M)v-=M;
        return *this;}
    mint&operator-=(mint o){
        if((v-=o.v)<0)v+=M;
        return *this;}
    mint&operator*=(mint o){
        v=(ll)v*o.v%M;return *this;}
    friend mint pow(mint a,ll p){assert(p>=0);
        return p==0?1:pow(a*a,p/2)*(p&1?a:1);}
    friend mint inv(mint a){assert(a.v!=0);
        return pow(a,M-2);}
    friend mint operator+(mint a,mint b){return a+=b;}
    friend mint operator-(mint a,mint b){return a-=b;}
    friend mint operator*(mint a,mint b){return a*=b;}
    friend istream&operator>>(istream&is,mint&a){ // 改
        return is>>a.v;} // 值在 [0,mod)
    friend ostream&operator<<(ostream&os,mint a){ // 无 改
        return os<<a.v;}
};
using mi=mint<0>; // 改为实际模数
```

### ModInverse.h

**Description:** 预处理模逆元。要求 LIM $\leq$ mod 且 mod 为质数。
**Usage:** `// 下面的代码需放在 main 等函数内`
`inv[i]; // 1 到 LIM-1 中 i 模 mod 的逆元`

6f684f, 3 lines

```
const ll mod = 1000000007, LIM = 200000;
ll*inv=new ll[LIM]-1;inv[1]=1;
rep(i,2,LIM)inv[i]=mod-(mod/i)*inv[mod%i]%mod;
```

### ModPow.h

**Description:** 快速幂，计算 $b^e \bmod \text{mod}$。
**Usage:** `ll r = modpow(b, e); // b 的 e 次方模 mod`
**Time:** $\mathcal{O}(\log e)$

b83e45, 8 lines

```
const ll mod=1000000007; // 设为 const 更快

ll modpow(ll b,ll e){
  ll ans=1;
  for(;e;b=b*b%mod,e/=2)
    if(e&1)ans=ans*b%mod;
  return ans;
}
```

### ModLog.h

**Description:** 返回满足 $a^x = b \pmod m$ 的最小 $x > 0$, 若不存在则返回 $-1$。modLog(a,1,m) 可用于 求 $a$ 的阶。
**Usage:** `ll x = modLog(a, b, m); // 无解时为 -1`
**Time:** $\mathcal{O}\left(\sqrt{m}\right)$

c040b8, 11 lines

```
ll modLog(ll a,ll b,ll m){
  ll n=(ll)sqrt(m)+1,e=1,f=1,j=1;
  unordered_map<ll,ll>A;
  while(j<=n&&(e=f=e*a%m)!=b%m)
    A[e*b%m]=j++;
  if(e==b%m)return j;
  if(__gcd(m,e)==__gcd(m,b))
    rep(i,2,n+2)if(A.count(e=e*f%m))
      return n*i-A[e];
  return -1;
}
```

### ModSum.h

**Description:** 等差数列取模后的求和。
`modsum(to, c, k, m)` $= \sum_{i=0}^{\text{to}-1} (ki + c)\%m$。 divsum 类似，但求的是向下取整除法之和。
**Usage:** `ll s = modsum(to, c, k, m); // 对 (k*i+c) mod m 求和`
`ull t = divsum(to, c, k, m); // 对 (k*i+c)/m 下取整求和`
**Time:** $\log(m)$，常数较大。

5c5bc5, 16 lines

```
typedef unsigned long long ull;
ull sumsq(ull to){return to/2*((to-1)|1);}

ull divsum(ull to,ull c,ull k,ull m){
  ull res=k/m*sumsq(to)+c/m*to;
  k%=m;c%=m;
  if(!k)return res;
  ull to2=(to*k+c)/m;
  return res+(to-1)*to2-divsum(to2,m-1-c,m,k);
}

ll modsum(ull to,ll c,ll k,ll m){
  c=((c%m)+m)%m;
  k=((k%m)+m)%m;
  return to*c+k*sumsq(to)-m*divsum(to,c,k,m);
}
```

### ModMulLL.h

**Description:** 计算 $a \cdot b \bmod c$（或 $a^b \bmod c$），要求 $0 \leq a, b \leq c \leq 7.2 \cdot 10^{18}$。
**Usage:** `ull x = modmul(a, b, M); // a*b mod M`
`ull y = modpow(b, e, M); // b 的 e 次方 mod M`
**Time:** modmul 为 $\mathcal{O}(1)$，modpow 为 $\mathcal{O}(\log b)$

bbbd8f, 11 lines

```
typedef unsigned long long ull;
ull modmul(ull a,ull b,ull M){
  ll ret=a*b-M*ull(1.L/M*a*b);
  return ret+M*(ret<0)-M*(ret>=(ll)M);
}
ull modpow(ull b,ull e,ull mod){
```

```
  ull ans=1;
  for(;e;b=modmul(b,b,mod),e/=2)
    if(e&1)ans=modmul(ans,b,mod);
  return ans;
}
```

ModSqrt.h

**Description:** 用 Tonelli-Shanks 算法求模平方根。求 $x$ 使 $x^2 = a \pmod p$（$-x$ 为另一个解）。
**Usage:** `ll x = sqrt(a, p); // p 为质数 , 无解时 assert 失败`
**Time:** 最坏 $\mathcal{O}\left(\log^2 p\right)$，对大多数 $p$ 为 $\mathcal{O}\left(\log p\right)$

"ModPow.h" 19a793, 24 lines

```
ll sqrt(ll a,ll p){
  a%=p;if(a<0)a+=p;
  if(a==0)return 0;
  assert(modpow(a,(p-1)/2,p)==1); // 否则无解
  if(p%4==3)return modpow(a,(p+1)/4,p);
  // 若 p % 8 == 5，可用 a^(n+3)/8 或 2^(n+3)/8 * 2^(n-1)/4
  ll s=p-1,n=2;
  int r=0,m;
  while(s%2==0)
    ++r,s/=2;
  while(modpow(n,(p-1)/2,p)!=p-1)++n;
  ll x=modpow(a,(s+1)/2,p);
  ll b=modpow(a,s,p),g=modpow(n,s,p);
  for(;;r=m){
    ll t=b;
    for(m=0;m<r&&t!=1;++m)
      t=t*t%p;
    if(m==0)return x;
    ll gs=modpow(g,1LL<<(r-m-1),p);
    g=gs*gs%p;
    x=x*gs%p;
    b=b*g%p;
  }
}
```

## 5.2 Primality

FastEratosthenes.h

**Description:** 质数筛，生成所有小于 LIM 的质数。
**Usage:** `vi pr = eratosthenes(); // 同时填好 isPrime`
**Time:** LIM=1e9 时 $\approx$ 1.5s

6b2912, 20 lines

```
const int LIM=1e6;
bitset<LIM>isPrime;
vi eratosthenes(){
  const int S=(int)round(sqrt(LIM)),R=LIM/2;
  vi pr={2},sieve(S+1);pr.reserve(int(LIM/log(LIM)*1.1));
  vector<pii>cp;
  for(int i=3;i<=S;i+=2)if(!sieve[i]){
    cp.push_back({i,i*i/2});
    for(int j=i*i;j<=S;j+=2*i)sieve[j]=1;
  }
  for(int L=1;L<=R;L+=S){
    array<bool,S>block{};
    for(auto&[p,idx]:cp)
      for(int i=idx;i<S+L;idx=(i+=p))block[i-L]=1;
    rep(i,0,min(S,R-L))
      if(!block[i])pr.push_back((L+i)*2+1);
  }
  for(int i:pr)isPrime[i]=1;
  return pr;
}
```

MillerRabin.h

**Description:** 确定性 Miller-Rabin 素性测试。保证对不超过 $7 \cdot 10^{18}$ 的数正确；更大的数请用 Python，并随机扩充 A。
**Usage:** `bool p = isPrime(n); // n 为 ull`
**Time:** $a^b \mod c$ 复杂度的 7 倍。

"ModMulLL.h" 60dcd1, 12 lines

```
bool isPrime(ull n){
  if(n<2||n%6%4!=1)return (n|1)==3;
  ull A[]={2,325,9375,28178,450775,9780504,1795265022},
      s=__builtin_ctzll(n-1),d=n>>s;
  for(ull a:A){ // ^末尾 0 的个数
    ull p=modpow(a%n,d,n),i=s;
    while(p!=1&&p!=n-1&&a%n&&i--)
      p=modmul(p,p,n);
    if(p!=n-1&&i!=s)return 0;
  }
  return 1;
}
```

Factor.h

**Description:** Pollard-rho 随机化质因数分解。返回一个数的 所有质因子，顺序任意（如 2299 -> {11, 19, 11}）。
**Usage:** `vector<ull> f = factor(n); // 含重复 , 无序`
**Time:** $\mathcal{O}\left(n^{1/4}\right)$，有小因子时更快。

"ModMulLL.h", "MillerRabin.h" d8d98d, 18 lines

```
ull pollard(ull n){
  ull x=0,y=0,t=30,prd=2,i=1,q;
  auto f=[&](ull x){return modmul(x,x,n)+i;};
  while(t++%40||__gcd(prd,n)==1){
    if(x==y)x=++i,y=f(x);
    if((q=modmul(prd,max(x,y)-min(x,y),n)))prd=q;
    x=f(x),y=f(f(y));
  }
  return __gcd(prd,n);
}
vector<ull>factor(ull n){
  if(n==1)return {};
  if(isPrime(n))return {n};
  ull x=pollard(n);
  auto l=factor(x),r=factor(n/x);
  l.insert(l.end(),all(r));
  return l;
}
```

## 5.3 Divisibility

SqrtIntervals.h

**Description:** 将 1..n 按顺序划分为若干闭区间，使每个区间内 $\lfloor \frac{n}{i} \rfloor$（或 $\lceil \frac{n}{i} \rceil$）不变。
**Usage:** `for (auto [l, r] : fdivSplit(n)) // 段内 n/i 下取整相同 cdivSplit(n) 同理 , 对应上取整`
**Time:** $\mathcal{O}\left(\sqrt{N}\right)$

6d93f2, 13 lines

```
auto fdivSplit=[](int n){
    vector<pii>segs;
    for(int q,r,l=1;l<=n;l=r+1){
        q=n/l,r=n/q;
        segs.pb({l,r});
    }
    return segs;
};
auto cdivSplit=[](int n){
    auto segs=fdivSplit(n-1);
    segs.pb({n,n});
    return segs;
};
```

euclid.h

**Description:** 求两个整数 $x$ 和 $y$，使 $ax + by = \gcd(a, b)$。若 只需要 gcd，请直接用内置的 __gcd。若 $a$ 与 $b$ 互质，则 $x$ 是 $a$ 的逆元 (mod $b$)。
**Usage:** `ll x, y, g = euclid(a, b, x, y); // g = gcd(a, b)`

33ba8f, 5 lines

```
ll euclid(ll a,ll b,ll&x,ll&y){
  if(!b)return x=1,y=0,a;
  ll d=euclid(b,a%b,y,x);
  return y-=a/b*x,d;
}
```

CRT.h

**Description:** 中国剩余定理。
crt(a, m, b, n) 求 $x$ 使 $x \equiv a \pmod m$，$x \equiv b \pmod n$。若 $|a| < m$ 且 $|b| < n$，则 $x$ 满足 $0 \le x < \text{lcm}(m, n)$。要求 $mn < 2^{62}$。
**Usage:** `ll x = crt(a, m, b, n); // 无解时 assert 失败`
**Time:** $\log(n)$

"euclid.h" 04d93a, 7 lines

```
ll crt(ll a,ll m,ll b,ll n){
  if(n>m)swap(a,b),swap(m,n);
  ll x,y,g=euclid(m,n,x,y);
  assert((a-b)%g==0); // 否则无解
  x=(b-a)%n*x%n/g*m+a;
  return x<0?x+m*n/g:x;
}
```

### 5.3.1 Bézout's identity

For $a \neq$, $b \neq 0$, then $d = gcd(a, b)$ is the smallest positive integer for which there are integer solutions to

$$ax + by = d$$

If $(x, y)$ is one solution, then all solutions are given by

$$\left(x + \frac{kb}{\gcd(a,b)}, y - \frac{ka}{\gcd(a,b)}\right), \quad k \in \mathbb{Z}$$

phiFunction.h

**Description:** 欧拉 $\phi$ 函数定义为 $\phi(n) := \#$（$\le n$ 且与 $n$ 互质的正整数）。$\phi(1) = 1$；$p$ 为质数 $\Rightarrow \phi(p^k) = (p-1)p^{k-1}$；$m, n$ 互质 $\Rightarrow \phi(mn) = \phi(m)\phi(n)$。若 $n = p_1^{k_1} p_2^{k_2} ... p_r^{k_r}$，则 $\phi(n) = (p_1 - 1)p_1^{k_1 - 1} ... (p_r - 1)p_r^{k_r - 1}$。$\phi(n) = n \cdot \prod_{p|n}(1 - 1/p)$。
$\sum_{d|n} \phi(d) = n$，$\sum_{1 \le k \le n, \gcd(k,n)=1} k = n\phi(n)/2, n > 1$
**欧拉定理**：$a, n$ 互质 $\Rightarrow a^{\phi(n)} \equiv 1 \pmod n$。
**费马小定理**：$p$ 为质数 $\Rightarrow a^{p-1} \equiv 1 \pmod p$ $\forall a$。
**Usage:** `calculatePhi(); // 之后 phi[i] 即 i 的欧拉函数 (i < LIM)`

cf7d6d, 8 lines

```
const int LIM=5000000;
int phi[LIM];

void calculatePhi(){
  rep(i,0,LIM)phi[i]=i&1?i:i/2;
  for(int i=3;i<LIM;i+=2)if(phi[i]==i)
    for(int j=i;j<LIM;j+=i)phi[j]-=phi[j]/i;
}
```

## 5.4 Fractions

Fractions.h

**Description:** 分数类（自动约分）。乘积可能溢出，需注意。
**Usage:** `frac a(1, 2), b(1, 3); frac c = a + b; // 5/6`
5563ee, 28 lines

```
struct frac{
    int p,q;
    frac(int p=0,int q=1):p(p),q(q){norm();}
    void norm(){assert(q);
        if(q<0)p=-p,q=-q;
        int g=gcd(p,q);
        p/=g,q/=g;}
    frac&operator+=(frac o){
        tie(p,q)=pair(p*o.q+q*o.p,q*o.q);
        norm();return *this;}
    frac&operator-=(frac o){
        tie(p,q)=pair(p*o.q-q*o.p,q*o.q);
        norm();return *this;}
    frac&operator*=(frac o){
        tie(p,q)=pair(p*o.p,q*o.q);
        norm();return *this;}
    frac&operator/=(frac o){
        tie(p,q)=pair(p*o.q,q*o.p);
        norm();return *this;}
    friend frac operator+(frac a,frac b){return a+=b;}
    friend frac operator-(frac a,frac b){return a-=b;}
    friend frac operator*(frac a,frac b){return a*=b;}
    friend frac operator/(frac a,frac b){return a/=b;}
    friend bool operator==(frac a,frac b)=default;
    friend bool operator<(frac a,frac b){return a.p*b.q<a.q*b.p
        ;}
    friend ostream&operator<<(ostream&os,frac a){
        return os<<a.p<<'/'<<a.q;}
};
```

ContinuedFractions.h
**Description:** 给定 $N$ 和实数 $x \geq 0$，求满足 $p, q \leq N$ 的最接近 $x$ 的有理数 $p/q$。结果满足 $|p/q - x| \leq 1/qN$。
相邻的渐近分数满足 $p_{k+1}q_k - q_{k+1}p_k = (-1)^k$。（$p_k/q_k$ 交替地 $> x$ 与 $< x$。）
若 $x$ 为有理数，$y$ 最终变为 $\infty$；若 $x$ 是 2 次多项式的根，则 $a$ 序列最终循环。
**Usage:** `auto [p, q] = approximate(x, N); // pair<ll, ll>`
**Time:** $\mathcal{O}(\log N)$
dd6c5e, 21 lines

```
typedef double d; // N 约 1e7 时可用 ; N 约 1e9 用 long double
pair<ll,ll>approximate(d x,ll N){
  ll LP=0,LQ=1,P=1,Q=0,inf=LLONG_MAX;d y=x;
  for(;;){
    ll lim=min(P?(N-LP)/P:inf,Q?(N-LQ)/Q:inf),
       a=(ll)floor(y),b=min(a,lim),
       NP=b*P+LP,NQ=b*Q+LQ;
    if(a>b){
      // 若 b 大于 a/2，存在更优的半渐近分数
      // 若 b = a/2，则可能存在
      // 可在此直接返回 {P, Q} 以得到更规范的逼近
      return (abs(x-(d)NP/(d)NQ)<abs(x-(d)P/(d)Q))?
        make_pair(NP,NQ):make_pair(P,Q);
    }
    if(abs(y=1/(y-(d)a))>3*N){
      return {NP,NQ};
    }
    LP=P;P=NP;
    LQ=Q;Q=NQ;
  }
}
```

FracBinarySearch.h
**Description:** 给定 $f$ 和 $N$，求使 $f(p/q)$ 为真的最小分数 $p/q \in [0, 1]$，其中 $p, q \leq N$。若 $f$ 找到精确解，可以在 $f$ 中抛出异常，此时可以去掉 $N$ 的限制。
**Usage:** `Frac r = fracBS([](Frac f) { return 3*f.p >= f.q; }, 10);`
`// r = {1, 3}, 即 1/3`
**Time:** $\mathcal{O}(\log(N))$
27ab3e, 25 lines

```
struct Frac{ll p,q;};

template<class F>
Frac fracBS(F f,ll N){
  bool dir=1,A=1,B=1;
  Frac lo{0,1},hi{1,1}; // 把 hi 设为 1/0 可搜索 (0, N]
  if(f(lo))return lo;
  assert(f(hi));
  while(A||B){
    ll adv=0,step=1; // dir 为真移动 hi，否则移动 lo
    for(int si=0;step;(step*=2)>>=si){
      adv+=step;
      Frac mid{lo.p*adv+hi.p,lo.q*adv+hi.q};
      if(abs(mid.p)>N||mid.q>N||dir==!f(mid)){
        adv-=step;si=2;
      }
    }
    hi.p+=lo.p*adv;
    hi.q+=lo.q*adv;
    dir=!dir;
    swap(lo,hi);
    A=B;B=!!adv;
  }
  return dir?hi:lo;
}
```

## 5.5 Pythagorean Triples

The Pythagorean triples are uniquely generated by

$$a = k \cdot (m^2 - n^2),\ \ b = k \cdot (2mn),\ \ c = k \cdot (m^2 + n^2),$$

with $m > n > 0$, $k > 0$, $m \perp n$, and either $m$ or $n$ even.

## 5.6 Primes

$p = 962592769$ is such that $2^{21} \mid p - 1$, which may be useful. For hashing use 970592641 (31-bit number), 31443539979727 (45-bit), 3006703054056749 (52-bit). There are 78498 primes less than 1 000 000.

Primitive roots exist modulo any prime power $p^a$, except for $p = 2, a > 2$, and there are $\phi(\phi(p^a))$ many. For $p = 2, a > 2$, the group $\mathbb{Z}_{2^a}^{\times}$ is instead isomorphic to $\mathbb{Z}_2 \times \mathbb{Z}_{2^{a-2}}$.

## 5.7 Estimates

$\sum_{d|n} d = O(n \log \log n)$.

The number of divisors of $n$ is at most around 100 for $n < 5e4$, 500 for $n < 1e7$, 2000 for $n < 1e10$, 200 000 for $n < 1e19$.

## 5.8 Mobius Function

$$\mu(n) = \begin{cases} 0 & n \text{ is not square free} \\ 1 & n \text{ has even number of prime factors} \\ -1 & n \text{ has odd number of prime factors} \end{cases}$$

Mobius Inversion:

$$g(n) = \sum_{d|n} f(d) \Leftrightarrow f(n) = \sum_{d|n} \mu(d) g(n/d)$$

Other useful formulas/forms:

$\sum_{d|n} \mu(d) = [n = 1]$ (very useful)

$g(n) = \sum_{n|d} f(d) \Leftrightarrow f(n) = \sum_{n|d} \mu(d/n) g(d)$

$g(n) = \sum_{1 \leq m \leq n} f(\lfloor \frac{n}{m} \rfloor) \Leftrightarrow f(n) = \sum_{1 \leq m \leq n} \mu(m) g(\lfloor \frac{n}{m} \rfloor)$

# Combinatorial (6)

## 6.1 Permutations

### 6.1.1 Factorial

| $n$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|
| $n!$ | 1 | 2 | 6 | 24 | 120 | 720 | 5040 | 40320 | 362880 | 3628800 |
| $n$ | 11 | 12 | 13 | 14 | 15 | 16 | 17 | | | |
| $n!$ | 4.0e7 | 4.8e8 | 6.2e9 | 8.7e10 | 1.3e12 | 2.1e13 | 3.6e14 | | | |
| $n$ | 20 | 25 | 30 | 40 | 50 | 100 | 150 | 171 | | |
| $n!$ | 2e18 | 2e25 | 3e32 | 8e47 | 3e64 | 9e157 | 6e262 | >DBL_MAX | | |

IntPerm.h
**Description:** 把排列转成整数 (不保持字典序). 整数转排列可以查表.
**Usage:** `vi v = {2, 0, 1}; int r = permToInt(v);`
`// v 为 0..n-1 的排列, r 属于 [0, n!), n 不超过 12`
**Time:** $\mathcal{O}(n)$
044568, 6 lines

```
int permToInt(vi&v){
  int use=0,i=0,r=0;
  for(int x:v)r=r*++i+__builtin_popcount(use&-(1<<x)),
    use|=1<<x; // 注意是负号，非 ~
  return r;
}
```

### 6.1.2 Cycles

Let $g_S(n)$ be the number of $n$-permutations whose cycle lengths all belong to the set $S$. Then

$$\sum_{n=0}^{\infty} g_S(n) \frac{x^n}{n!} = \exp\left(\sum_{n \in S} \frac{x^n}{n}\right)$$

### 6.1.3 Derangements

Permutations of a set such that none of the elements appear in their original position.

$$D(n) = (n-1)(D(n-1)+D(n-2)) = nD(n-1)+(-1)^n = \left\lfloor \frac{n!}{e} \right\rceil$$

### 6.1.4 Burnside's lemma

Given a group $G$ of symmetries and a set $X$, the number of elements of $X$ *up to symmetry* equals

$$\frac{1}{|G|}\sum_{g\in G}|X^g|,$$

where $X^g$ are the elements fixed by $g$ ($g.x = x$).

If $f(n)$ counts "configurations" (of some sort) of length $n$, we can ignore rotational symmetry using $G = \mathbb{Z}_n$ to get

$$g(n) = \frac{1}{n}\sum_{k=0}^{n-1} f(\gcd(n,k)) = \frac{1}{n}\sum_{k|n} f(k)\phi(n/k).$$

## 6.2 Partitions and subsets

### 6.2.1 Partition function

Number of ways of writing $n$ as a sum of positive integers, disregarding the order of the summands.

$$p(0) = 1,\ p(n) = \sum_{k\in\mathbb{Z}\setminus\{0\}} (-1)^{k+1} p(n-k(3k-1)/2)$$

$$p(n) \sim 0.145/n \cdot \exp(2.56\sqrt{n})$$

| $n$ | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 20 | 50 | 100 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| $p(n)$ | 1 | 1 | 2 | 3 | 5 | 7 | 11 | 15 | 22 | 30 | 627 | ∼2e5 | ∼2e8 |

### 6.2.2 Lucas' Theorem

Let $n, m$ be non-negative integers and $p$ a prime. Write $n = n_k p^k + ... + n_1 p + n_0$ and $m = m_k p^k + ... + m_1 p + m_0$. Then $\binom{n}{m} \equiv \prod_{i=0}^{k} \binom{n_i}{m_i} \pmod p$.

### 6.2.3 Binomials

multinomial.h

**Description:** 计算多项式系数 $\binom{k_1+\cdots+k_n}{k_1,k_2,\ldots,k_n} = \frac{(\sum k_i)!}{k_1!k_2!\ldots k_n!}$.

**Usage:** `ll c = multinomial(v); // v = {k_1, ..., k_n}`

a0a312, 5 lines

```
ll multinomial(vi&v){
  ll c=1,m=v.empty()?1:v[0];
  rep(i,1,sz(v))rep(j,0,v[i])c=c*++m/(j+1);
  return c;
}
```

## 6.3 General purpose numbers

### 6.3.1 Bernoulli numbers

EGF of Bernoulli numbers is $B(t) = \frac{t}{e^t-1}$ (FFT-able). $B[0,\ldots] = [1, -\frac{1}{2}, \frac{1}{6}, 0, -\frac{1}{30}, 0, \frac{1}{42}, \ldots]$

Sums of powers:

$$\sum_{i=1}^{n} n^m = \frac{1}{m+1}\sum_{k=0}^{m}\binom{m+1}{k} B_k \cdot (n+1)^{m+1-k}$$

Euler-Maclaurin formula for infinite sums:

$$\sum_{i=m}^{\infty} f(i) = \int_m^\infty f(x)dx - \sum_{k=1}^{\infty}\frac{B_k}{k!}f^{(k-1)}(m)$$

$$\approx \int_m^\infty f(x)dx + \frac{f(m)}{2} - \frac{f'(m)}{12} + \frac{f'''(m)}{720} + O(f^{(5)}(m))$$

### 6.3.2 Stirling numbers of the first kind

Number of permutations on $n$ items with $k$ cycles.

$$c(n,k) = c(n-1,k-1) + (n-1)c(n-1,k),\ c(0,0) = 1$$

$$\textstyle\sum_{k=0}^{n} c(n,k)x^k = x(x+1)\ldots(x+n-1)$$

$c(8,k) = 8, 0, 5040, 13068, 13132, 6769, 1960, 322, 28, 1$
$c(n,2) = 0, 0, 1, 3, 11, 50, 274, 1764, 13068, 109584, \ldots$

### 6.3.3 Eulerian numbers

Number of permutations $\pi \in S_n$ in which exactly $k$ elements are greater than the previous element. $k$ $j$:s s.t. $\pi(j) > \pi(j+1)$, $k+1$ $j$:s s.t. $\pi(j) \geq j$, $k$ $j$:s s.t. $\pi(j) > j$.

$$E(n,k) = (n-k)E(n-1,k-1) + (k+1)E(n-1,k)$$

$$E(n,0) = E(n,n-1) = 1$$

$$E(n,k) = \sum_{j=0}^{k}(-1)^j\binom{n+1}{j}(k+1-j)^n$$

### 6.3.4 Stirling numbers of the second kind

Partitions of $n$ distinct elements into exactly $k$ groups.

$$S(n,k) = S(n-1,k-1) + kS(n-1,k)$$

$$S(n,1) = S(n,n) = 1$$

$$S(n,k) = \frac{1}{k!}\sum_{j=0}^{k}(-1)^{k-j}\binom{k}{j}j^n$$

### 6.3.5 Bell numbers

Total number of partitions of $n$ distinct elements. $B(n) = 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, \ldots$. For $p$ prime,

$$B(p^m + n) \equiv mB(n) + B(n+1) \pmod p$$

### 6.3.6 Labeled unrooted trees

# on $n$ vertices: $n^{n-2}$
# on $k$ existing trees of size $n_i$: $n_1 n_2 \cdots n_k n^{k-2}$
# with degrees $d_i$: $(n-2)!/((d_1-1)!\cdots(d_n-1)!)$

### 6.3.7 Catalan numbers

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1} = \frac{(2n)!}{(n+1)!n!}$$

$$C_0 = 1,\ C_{n+1} = \frac{2(2n+1)}{n+2}C_n,\ C_{n+1} = \sum C_i C_{n-i}$$

$C_n = 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, \ldots$

- sub-diagonal monotone paths in an $n \times n$ grid.
- strings with $n$ pairs of parenthesis, correctly nested.
- binary trees with with $n+1$ leaves (0 or 2 children).
- ordered trees with $n+1$ vertices.
- ways a convex polygon with $n+2$ sides can be cut into triangles by connecting vertices with straight lines.
- permutations of $[n]$ with no 3-term increasing subseq.

# Graph (7)

## 7.1 Fundamentals

BellmanFord.h

**Description:** 求从 $s$ 出发的单源最短路, 允许负边权. 不可达的点 dist = inf; 能经过负环到达的点 dist = -inf. 要求 $V^2 \max|w_i| < \sim 2^{63}$.

**Usage:** `vector<Node> nodes(n); vector<Ed> eds{{a, b, w}}; bellmanFord(nodes, eds, s); // nodes[v].dist, nodes[v].prev`

**Time:** $\mathcal{O}(VE)$

830a8f, 23 lines

```
const ll inf=LLONG_MAX;
struct Ed{int a,b,w,s(){return a<b?a:-a;}};
struct Node{ll dist=inf;int prev=-1;};

void bellmanFord(vector<Node>&nodes,vector<Ed>&eds,int s){
  nodes[s].dist=0;
  sort(all(eds),[](Ed a,Ed b){return a.s()<b.s();});

  int lim=sz(nodes)/2+2; // 顶点随机打乱时可用 /3+100
  rep(i,0,lim)for(Ed ed:eds){
    Node cur=nodes[ed.a],&dest=nodes[ed.b];
    if(abs(cur.dist)==inf)continue;
    ll d=cur.dist+ed.w;
    if(d<dest.dist){
      dest.prev=ed.a;
      dest.dist=(i<lim-1?d:-inf);
    }
```

```
  }
  rep(i,0,lim)for(Ed e:eds){
    if(nodes[e.a].dist==-inf)
      nodes[e.b].dist=-inf;
  }
}
```

### FloydWarshall.h

**Description:** 有向图全源最短路, 允许负边权. 输入距离矩阵 $m$, 若 $i$ 与 $j$ 不相邻则 $m[i][j] = \texttt{inf}$. 结束后 $m[i][j]$ 为 $i$ 到 $j$ 的最短距离, 无路径时为 inf, 路径经过负环时为 -inf.
**Usage:** `vector<vector<ll>> m(n, vector<ll>(n, inf));`
`m[a][b] = min(m[a][b], w); floydWarshall(m);`
**Time:** $\mathcal{O}\left(N^3\right)$

531245, 12 lines

```
const ll inf=1LL<<62;
void floydWarshall(vector<vector<ll>>&m){
  int n=sz(m);
  rep(i,0,n)m[i][i]=min(m[i][i],0LL);
  rep(k,0,n)rep(i,0,n)rep(j,0,n)
    if(m[i][k]!=inf&&m[k][j]!=inf){
      auto newDist=max(m[i][k]+m[k][j],-inf);
      m[i][j]=min(m[i][j],newDist);
    }
  rep(k,0,n)if(m[k][k]<0)rep(i,0,n)rep(j,0,n)
    if(m[i][k]!=inf&&m[k][j]!=inf)m[i][j]=-inf;
}
```

### TopoSort.h

**Description:** 拓扑排序. 输入有向图的邻接表. 输出一个顶点序列, 使所有边都从左指向右. 若有环, 返回序列的长度小于 $n$: 从环出发可达的点都不会出现.
**Usage:** `vector<vi> gr(n); gr[a].push_back(b); // 边 a 到 b`
`vi ord = topoSort(gr); // sz(ord) < n 表示有环`
**Time:** $\mathcal{O}\left(|V| + |E|\right)$

d678d8, 8 lines

```
vi topoSort(const vector<vi>&gr){
  vi indeg(sz(gr)),q;
  for(auto&li:gr)for(int x:li)indeg[x]++;
  rep(i,0,sz(gr))if(indeg[i]==0)q.push_back(i);
  rep(j,0,sz(q))for(int x:gr[q[j]])
    if(--indeg[x]==0)q.push_back(x);
  return q;
}
```

### DynamicConnectivity.h

**Description:** 离线动态图连通性 (线段树分治 + 可撤销并查集). 每条边在时间区间 $[l, r]$ 内存在, 在 add_query 的时刻求答案; 默认答案为连通块个数 (点编号 $1..n$, 并查集大小 $n+1$). 易错点: 若 $u > v$ 先交换 $u, v$ (方便匹配同一条边); 边在时刻 $t$ 被删除则结束时间为 $t-1$; 别忘了处理从未删除的边.
**Usage:** `DynConn dc(T); dc.add_edge(l, r, {u, v});`
`dc.add_query(t); RollbackUF uf(n + 1);`
`dc.solve(uf); // 答案在 dc.ans[t]`

"UnionFindRollback.h" 8732a8, 43 lines

```
struct DynConn{
    int T; // DynConn(T) 的时间范围为 [0..T]
    vector<vector<pii>>tree;
    vector<int>queries; // 按需修改询问类型
    vi ans; // 默认是连通块数 ; -1 表示此时无询问
    DynConn(int t):T(t+2),tree(4*T),queries(4*T,-1),
        ans(T,-1){}
    // [ql, qr] 为边的开始和结束时刻
    void add_edge(int ql,int qr,pii edge){
        if(ql>qr)return;
        auto f=[&](auto f,int node,int l,int r)->void{
            if(ql>r||qr<l)return;
            if(ql<=l&&r<=qr){
                tree[node].push_back(edge);
                return;
            }
            int mid=(l+r)/2;
            f(f,2*node,l,mid);
            f(f,2*node+1,mid+1,r);
        };
        f(f,1,0,T);
    }
    void add_query(int time){ // 按需加参数
        queries[time]=0;
    }
    void solve(auto&dsu){
        auto f=[&](auto f,int node,int l,int r)->void{
            int t=dsu.time();
            for(auto[a,b]:tree[node])dsu.join(a,b);
            if(l==r){
                if(queries[l]!=-1){ // 按需修改
                    ans[l]=sz(dsu.e)-l-dsu.time()/2;
                }
            }else{
                int mid=(l+r)/2;
                f(f,2*node,l,mid);
                f(f,2*node+1,mid+1,r);
            }
            dsu.rollback(t);
        };
        f(f,1,0,T);
    }
};
```

## 7.2 Network flow

### MinCostMaxFlow.h

**Description:** 最小费用最大流, 要求无负环. 可以构造负边权使第一次 Dijkstra 很慢, 但实践中没出过问题. 之后每轮 (加势能后) 边权都 $\geq 0$. 想得到原边上的流量, 在 ae 时记下边的编号.
**Usage:** `MCMF<ll, ll> mf(n); mf.ae(u, v, cap, cost);`
`auto [fl, co] = mf.calc(s, t); // 最大流 与 最小费用`
`mf.edges[2 * i].flow // 第 i 次 ae 的边上的流量`
**Time:** 不计第一次 Dijkstra, 若容量为整数且 $F$ 为最大流, 则为 $\mathcal{O}\left(FM \log M\right)$.

69c7a2, 72 lines

```
template<class F,class C> // F 为流量类型 ; C 为费用类型
struct MCMF{
    struct Edge{int to;F flow,cap;C cost;};

    int n;
    vector<C>p,dist;
    vector<int>pre;
    vector<Edge>edges;
    vector<vector<int>>adj;

    const C INF=numeric_limits<C>::max()/4;

    MCMF(int n):n(n),p(n),dist(n),pre(n),adj(n){}

    void ae(int u,int v,F cap,C cost){
        assert(cap>=0);
        adj[u].push_back(sz(edges));
        edges.push_back({v,0,cap,cost});
        adj[v].push_back(sz(edges));
        edges.push_back({u,0,0,-cost});
    }

    bool path(int s,int t){
        for(int i=0;i<n;i++)
            dist[i]=INF;
        using T=pair<C,int>;
        priority_queue<T,vector<T>,greater<>>todo;
        todo.push({dist[s]=0,s});
        while(!todo.empty()){
            T x=todo.top();
            todo.pop();
            if(x.first>dist[x.second])
                continue;
            for(auto&eid:adj[x.second]){
                const Edge&e=edges[eid];
                if(e.flow<e.cap&&dist[e.to]
              >x.first+e.cost+p[x.second]-p[e.to]){
                    dist[e.to]=x.first+e.cost+p[x.second]-p[e.
                        to];
                    pre[e.to]=eid;
                    todo.push({dist[e.to],e.to});
                }
            }
        }
        return dist[t]!=INF;
    }

    pair<F,C>calc(int s,int t){
        assert(s!=t);
        for(int unused=0;unused<n;unused++)
            for(int eid=0;eid<(int)edges.size();eid++){
                const Edge&e=edges[eid];
                if(e.cap)
                    p[e.to]=min(p[e.to],p[edges[eid^1].to]+e.
                        cost);
            }
        F totflow=0;
        C totcost=0;
        while(path(s,t)){
            for(int i=0;i<n;i++)
                p[i]=min(p[i]+dist[i],INF);
            F df=numeric_limits<F>::max()/4;
            for(int x=t;x!=s;x=edges[pre[x]^1].to){
                const Edge&e=edges[pre[x]];
                df=min(df,e.cap-e.flow);
            }
            totflow+=df;
            totcost+=(p[t]-p[s])*df;
            for(int x=t;x!=s;x=edges[pre[x]^1].to)
                edges[pre[x]].flow+=df,edges[pre[x]^1].flow-=df
                    ;
        }
        return {totflow,totcost};
    }
};
```

### Dinic.h

**Description:** 最大流 Dinic 算法 (带容量缩放), 复杂度 $O(VE \log U)$, 其中 $U = \max|\text{cap}|$. 若 $U = 1$ 为 $O(\min(E^{1/2}, V^{2/3})E)$; 二分图匹配为 $O(\sqrt{V}E)$.
**Usage:** `Dinic d(n); d.addEdge(a, b, c); // 第 4 个参数是反向容量`
`ll f = d.calc(s, t); bool L = d.leftOfMinCut(v);`

d7f0f1, 42 lines

```
struct Dinic{
  struct Edge{
    int to,rev;
    ll c,oc;
    ll flow(){return max(oc-c,0LL);} // 需要流量时用
  };
  vi lvl,ptr,q;
  vector<vector<Edge>>adj;
  Dinic(int n):lvl(n),ptr(n),q(n),adj(n){}
  void addEdge(int a,int b,ll c,ll rcap=0){
    adj[a].push_back({b,sz(adj[b]),c,c});
    adj[b].push_back({a,sz(adj[a])-1,rcap,rcap});
  }
  ll dfs(int v,int t,ll f){
    if(v==t||!f)return f;
```

```
    for(int&i=ptr[v];i<sz(adj[v]);i++){
      Edge&e=adj[v][i];
      if(lvl[e.to]==lvl[v]+1)
        if(ll p=dfs(e.to,t,min(f,e.c))){
          e.c-=p,adj[e.to][e.rev].c+=p;
          return p;
        }
    }
    return 0;
  }
  ll calc(int s,int t){
    ll flow=0;q[0]=s;
    rep(L,0,31)do{ // 随机数据下 'int L=30' 可能更快
      lvl=ptr=vi(sz(q));
      int qi=0,qe=lvl[s]=1;
      while(qi<qe&&!lvl[t]){
        int v=q[qi++];
        for(Edge e:adj[v])
          if(!lvl[e.to]&&e.c>>(30-L))
            q[qe++]=e.to,lvl[e.to]=lvl[v]+1;
      }
      while(ll p=dfs(s,t,LLONG_MAX))flow+=p;
    }while(lvl[t]);
    return flow;
  }
  bool leftOfMinCut(int a){return lvl[a]!=0;}
};
```

## MinCut.h

**Description:** 跑完最大流后, $s$-$t$ 最小割的 $s$ 侧就是从 $s$ 出发, 只走剩余容量为正的边所能到达的所有点.
**Usage:** `d.calc(s, t); // 先用 Dinic 等跑最大流`
`d.leftOfMinCut(v) // 为真则 v 在 s 侧`
割边 = 所有从 s 侧指向 t 侧的原边 (u, v)

## GlobalMinCut.h

**Description:** Stoer-Wagner 算法: 求无向图的全局最小割, 图以邻接矩阵给出. 返回 (割的权值, 割的一侧点集).
**Usage:** `vector<vi> mat(n, vi(n)); // mat[a][b] 为对称的边权`
`auto [w, side] = globalMinCut(mat); // 割值 w 与一侧点集`
**Time:** $\mathcal{O}\left(V^3\right)$ 8b0e19, 21 lines

```
pair<int,vi>globalMinCut(vector<vi>mat){
  pair<int,vi>best={INT_MAX,{}};
  int n=sz(mat);
  vector<vi>co(n);
  rep(i,0,n)co[i]={i};
  rep(ph,1,n){
    vi w=mat[0];
    size_t s=0,t=0;
    rep(it,0,n-ph){ // 用优先队列可优化为 O(E log V)
      w[t]=INT_MIN;
      s=t,t=max_element(all(w))-w.begin();
      rep(i,0,n)w[i]+=mat[t][i];
    }
    best=min(best,{w[t]-mat[t][t],co[t]});
    co[s].insert(co[s].end(),all(co[t]));
    rep(i,0,n)mat[s][i]+=mat[t][i];
    rep(i,0,n)mat[i][s]=mat[s][i];
    mat[0][t]=INT_MIN;
  }
  return best;
}
```

# Matching (8)

## HopcroftKarp.h

**Description:** 快速二分图匹配. $g$ 为左部各点的邻接表 (存右部点编号), $r$ 为长度等于右部点数且全为 $-1$ 的数组. 返回匹配数. $r[i]$ 为右部点 $i$ 匹配的左部点, 未匹配为 $-1$. Usage 中的 $l[i]$ 为左部点 $i$ 匹配的右部点.
**Usage:** `vi r(m, -1); hopcroftKarp(g, r);`
`vi l(sz(g), -1); rep(j,0,m) if (r[j] != -1) l[r[j]] = j;`
**Time:** $\mathcal{O}\left(E\sqrt{V}\right)$ 731cfb, 20 lines

```
int hopcroftKarp(vector<vi>&g,vi&r){
  int n=sz(g),res=0;
  vi l(n,-1),q(n),d(n);
  auto dfs=[&](auto f,int u)->bool{
    int t=exchange(d[u],0)+1;
    for(int v:g[u])
      if(r[v]==-1||(d[r[v]]==t&&f(f,r[v])))
        return l[u]=v,r[v]=u,1;
    return 0;
  };
  for(int t=0,f=0;;t=f=0,d.assign(n,0)){
    rep(i,0,n)if(l[i]==-1)q[t++]=i,d[i]=1;
    rep(i,0,t)for(int v:g[q[i]]){
      if(r[v]==-1)f=1;
      else if(!d[r[v]])d[r[v]]=d[q[i]]+1,q[t++]=r[v];
    }
    if(!f)return res;
    rep(i,0,n)if(l[i]==-1)res+=dfs(dfs,i);
  }
}
```

## MinimumVertexCover.h

**Description:** 求二分图 (A, B) 的最小点覆盖, $|A| = n$, $|B| = m$. $g$ 为左部各点的邻接表. 覆盖的大小等于最大匹配数, 其补集是最大独立集.
**Usage:** `vi c = cover(g, n, m); // 右部点 j 记为 n + j`
"HopcroftKarp.h" bfb654, 20 lines

```
vi cover(vector<vi>&g,int n,int m){
  vi match(m,-1);
  int res=hopcroftKarp(g,match);
  vector<bool>lfound(n,true),seen(m);
  for(int it:match)if(it!=-1)lfound[it]=false;
  vi q,cover;
  rep(i,0,n)if(lfound[i])q.push_back(i);
  while(!q.empty()){
    int i=q.back();q.pop_back();
    lfound[i]=1;
    for(int e:g[i])if(!seen[e]&&match[e]!=-1){
      seen[e]=true;
      q.push_back(match[e]);
    }
  }
  rep(i,0,n)if(!lfound[i])cover.push_back(i);
  rep(i,0,m)if(seen[i])cover.push_back(n+i);
  assert(sz(cover)==res);
  return cover;
}
```

## WeightedMatching.h

**Description:** 带权二分图匹配 (匈牙利算法): 给左部每个点匹配一个不同的 右部点, 使边权和最小. 输入 cost[N][M], cost[i][j] 为 L[i] 匹配 R[j] 的代价; 返回 (最小代价, match), L[i] 与 R[match[i]] 匹配. 求最大权时把代价取负. 要求 $N \le M$.
**Usage:** `vector<vector<ll>> cost(N, vector<ll>(M));`
`auto [c, mt] = hungarian(cost); // L[i] 匹配 R[mt[i]]`
**Time:** $\mathcal{O}\left(N^2M\right)$ f79c17, 35 lines

```
template<class T>
pair<T,vector<T>>hungarian(const vector<vector<T>>&a){
  if(a.empty())return {0,{}};
  int n=sz(a)+1,m=sz(a[0])+1;
  vector<T>u(n),v(m),ans(n-1);
  vi p(m);
  rep(i,1,n){
    p[0]=i;
    int j0=0; // 加入虚拟点 0
    vector<T>dist(m,1e18); // 按需修改上界
    vi pre(m,-1);
    vector<bool>done(m+1);
    do{ // dijkstra
      done[j0]=true;
      int i0=p[j0],j1;
      T delta=1e18; // 按需修改上界
      rep(j,1,m)if(!done[j]){
          T cur=a[i0-1][j-1]-u[i0]-v[j];
          if(cur<dist[j])dist[j]=cur,pre[j]=j0;
          if(dist[j]<delta)delta=dist[j],j1=j;
        }
      rep(j,0,m){
        if(done[j])u[p[j]]+=delta,v[j]-=delta;
        else dist[j]-=delta;
      }
      j0=j1;
    }while(p[j0]);
    while(j0){ // 更新交错路
      int j1=pre[j0];
      p[j0]=p[j1],j0=j1;
    }
  }
  rep(j,1,m)if(p[j])ans[p[j]-1]=j-1;
  return {-v[0],ans}; // {最小代价 , 匹配数组 }
}
```

## GeneralMatching.h

**Description:** 一般图最大匹配 (随机化, 基于 Tutte 矩阵求逆). 失败概率为 $N/mod$.
**Usage:** `vector<pii> ed{{a, b}}; // 点编号 0..N-1 的无向边`
`vector<pii> m = generalMatching(N, ed); // 匹配边`
**Time:** $\mathcal{O}\left(N^3\right)$
"../numerical/MatrixInverse-mod.h" cb1912, 40 lines

```
vector<pii>generalMatching(int N,vector<pii>&ed){
  vector<vector<ll>>mat(N,vector<ll>(N)),A;
  for(pii pa:ed){
    int a=pa.first,b=pa.second,r=rand()%mod;
    mat[a][b]=r,mat[b][a]=(mod-r)%mod;
  }

  int r=matInv(A=mat),M=2*N-r,fi,fj;
  assert(r%2==0);

  if(M!=N)do{
    mat.resize(M,vector<ll>(M));
    rep(i,0,N){
      mat[i].resize(M);
      rep(j,N,M){
        int r=rand()%mod;
        mat[i][j]=r,mat[j][i]=(mod-r)%mod;
      }
    }
  }while(matInv(A=mat)!=M);

  vi has(M,1);vector<pii>ret;
  rep(it,0,M/2){
    rep(i,0,M)if(has[i])
      rep(j,i+1,M)if(A[i][j]&&mat[i][j]){
        fi=i;fj=j;goto done;
```

```
    }assert(0);done:
    if(fj<N)ret.emplace_back(fi,fj);
    has[fi]=has[fj]=0;
    rep(sw,0,2){
      ll a=modpow(A[fi][fj],mod-2);
      rep(i,0,M)if(has[i]&&A[i][fj]){
        ll b=A[i][fj]*a%mod;
        rep(j,0,M)A[i][j]=(A[i][j]-A[fi][j]*b)%mod;
      }
      swap(fi,fj);
    }
  }
  return ret;
}
```

## 8.1 DFS algorithms

### AllRoots.h

**Description:** 换根 DP (0 起编号), 要求 + 满足结合律和交换律. operator+ 把多个儿子合并为一个值. promote() 求一个儿子对父亲的贡献.
**Usage:** `vector<pii> el; // n - 1 条树边 (a, b), 第 i 条边编号 i`
`vector<ART> res = AllRoots<ART>::calc(el); // 各点为根的答案`
**Time:** $\mathcal{O}(N)$.

a8ea30, 47 lines

```
struct ART{
  // 默认构造必须是单位元
  int ans=0; // 到其他点的最远距离
  ART operator+(const ART&o)const{
    return {max(ans,o.ans)};
  }
  ART promote(int x,int e)const{
    if(e==-1)return *this;
    return {ans+1}; // x 为点编号 ; e 为边编号
  }
};
ART base{}; // 单点的值 , 不一定是单位元
template<class T>
struct AllRoots{
  int n;vector<vector<pii>>adj;
  vector<vector<T>>suf;vector<T>res;
  AllRoots(vector<pii>&el):n(sz(el)+1),adj(n),suf(n),res(n){
    for(int ei=-1;auto[a,b]:el){ei++;
      adj[a].pb({b,ei});
      adj[b].pb({a,ei});
    }
    for(auto&s:suf)s.pb(base); // 点的初始值
  }
  T down(int x,int p=-1){
    for(auto&c:adj[x])
      if(c.F==p)swap(c,adj[x][0]);
    for(int i=sz(adj[x])-1;i>-(p<0);--i)
      suf[x].pb(down(adj[x][i].F,x)+suf[x].back());
    return (p==-1?T{}:suf[x].back().promote(x,adj[x][0].S));
  }
  void up(int x,int p=-1){
    rep(i,p!=-1,sz(adj[x])){
      auto[y,ei]=adj[x][i];
      suf[x].pop_back();
      res[y]=(res[x]+suf[x].back()).promote(x,ei);
      res[x]=res[x]+suf[y].back().promote(y,ei);
      up(y,x);
    }
  }
  static vector<T>calc(vector<pii>&el){
    AllRoots ar(el);
    ar.down(0);
    ar.up(0);
    rep(i,0,ar.n)ar.res[i]=ar.res[i].promote(i,-1);
    return ar.res;
  }
};
```

### SCC.h

**Description:** 求有向图的强连通分量. 若 $u, v$ 在同一分量中, 则 $u$ 可达 $v$ 且 $v$ 可达 $u$.
**Usage:** `scc(graph, [&](vi& v) { ... }); // 按逆拓扑序访问各分量`
`comp[i] 为点 i 所在分量的编号 ; 分量只会连向`
`编号更小的分量 . ncomps 为分量个数`
**Time:** $\mathcal{O}(E+V)$

76b5c9, 24 lines

```
vi val,comp,z,cont;
int Time,ncomps;
template<class G,class F>int dfs(int j,G&g,F&f){
  int low=val[j]=++Time,x;z.push_back(j);
  for(auto e:g[j])if(comp[e]<0)
    low=min(low,val[e]?:dfs(e,g,f));

  if(low==val[j]){
    do{
      x=z.back();z.pop_back();
      comp[x]=ncomps;
      cont.push_back(x);
    }while(x!=j);
    f(cont);cont.clear();
    ncomps++;
  }
  return val[j]=low;
}
template<class G,class F>void scc(G&g,F f){
  int n=sz(g);
  val.assign(n,0);comp.assign(n,-1);
  Time=ncomps=0;
  rep(i,0,n)if(comp[i]<0)dfs(i,g,f);
}
```

### BiconnectedComponents.h

**Description:** 求无向图的所有点双连通分量, 并对每个分量的边集调用回调. 同一点双内任意两点之间至少有两条内部不相交的路径 (即存在经过它们的环). 一个点可以属于多个分量. 不属于任何分量的边是桥, 即不在任何环上.
**Usage:** `int eid = 0; ed.resize(N); for (auto &x: ed) x.clear();`
`for (auto [a, b] : edges) { // 对每条边`
`ed[a].emplace_back(b, eid);`
`ed[b].emplace_back(a, eid++); }`
`bicomps([&](const vi& edgelist) {...}); // 分量的边编号`
**Time:** $\mathcal{O}(E+V)$

c6b7c7, 32 lines

```
vi num,st;
vector<vector<pii>>ed;
int Time;
template<class F>
int dfs(int at,int par,F&f){
  int me=num[at]=++Time,top=me;
  for(auto[y,e]:ed[at])if(e!=par){
    if(num[y]){
      top=min(top,num[y]);
      if(num[y]<me)
        st.push_back(e);
    }else{
      int si=sz(st);
      int up=dfs(y,e,f);
      top=min(top,up);
      if(up==me){
        st.push_back(e);
        f(vi(st.begin()+si,st.end()));
        st.resize(si);
      }
      else if(up<me)st.push_back(e);
      else{ /* e 是桥 */ }
    }
  }
  return top;
}

template<class F>
void bicomps(F f){
  num.assign(sz(ed),0);
  rep(i,0,sz(ed))if(!num[i])dfs(i,-1,f);
}
```

### 2sat.h

**Description:** 求解 2-SAT: 给布尔变量 a, b, c, ... 赋值, 使形如 $(a||b)\&\&(!a||c)\&\&(d||!b)\&\&\ldots$ 的表达式为真, 或报告无解. 取反的变量用按位取反表示 (~x).
**Usage:** `TwoSat ts(n); // n 为布尔变量个数`
`ts.either(0, ~3); // 变量 0 为真或变量 3 为假`
`ts.setValue(2); // 变量 2 为真`
`ts.atMostOne({0,~1,2}); // 0 , ~1 , 2 中至多一个为真`
`ts.solve(); // 有解时返回 true`
`ts.values[0..N-1] 为各变量的赋值`
**Time:** $\mathcal{O}(N+E)$, 其中 N 为布尔变量个数, E 为子句个数.

5f9706, 56 lines

```
struct TwoSat{
  int N;
  vector<vi>gr;
  vi values; // 0 为假 ; 1 为真

  TwoSat(int n=0):N(n),gr(2*n){}

  int addVar(){ // 可选
    gr.emplace_back();
    gr.emplace_back();
    return N++;
  }

  void either(int f,int j){
    f=max(2*f,-1-2*f);
    j=max(2*j,-1-2*j);
    gr[f].push_back(j^1);
    gr[j].push_back(f^1);
  }
  void setValue(int x){either(x,x);}

  void atMostOne(const vi&li){ // 可选
    if(sz(li)<=1)return;
    int cur=~li[0];
    rep(i,2,sz(li)){
      int next=addVar();
      either(cur,~li[i]);
      either(cur,next);
      either(~li[i],next);
      cur=~next;
    }
    either(cur,~li[1]);
  }

  vi val,comp,z;int time=0;
  int dfs(int i){
    int low=val[i]=++time,x;z.push_back(i);
    for(int e:gr[i])if(!comp[e])
      low=min(low,val[e]?:dfs(e));
    if(low==val[i])do{
      x=z.back();z.pop_back();
      comp[x]=low;
      if(values[x>>1]==-1)
        values[x>>1]=x&1;
    }while(x!=i);
```

```
    return val[i]=low;
  }

  bool solve(){
    values.assign(N,-1);
    val.assign(2*N,0);comp=val;
    rep(i,0,2*N)if(!comp[i])dfs(i);
    rep(i,0,N)if(comp[2*i]==comp[2*i+1])return 0;
    return 1;
  }
};
```

### EulerWalk.h

**Description:** 无向/有向图的欧拉路径/回路. 输入为 (终点, 全局边编号) 的邻接表, 无向图中正反两条边编号相同. 返回欧拉路径/回路上的点序列 (从 src 开始; 回路时首尾都是 src), 不存在时返回空. 若要得到边编号, 在 s 和 ret 中额外存 .second.
**Usage:** `vector<vector<pii>> gr(n); gr[a].push_back({b, i}); gr[b].push_back({a, i}); // 仅无向图需要`
`vi path = eulerWalk(gr, nedges, src); // 空表示不存在`
**Time:** $\mathcal{O}(V+E)$

780b64, 15 lines

```
vi eulerWalk(vector<vector<pii>>&gr,int nedges,int src=0){
  int n=sz(gr);
  vi D(n),its(n),eu(nedges),ret,s={src};
  D[src]++; // 允许欧拉路径而不只是回路
  while(!s.empty()){
    int x=s.back(),y,e,&it=its[x],end=sz(gr[x]);
    if(it==end){ret.push_back(x);s.pop_back();continue;}
    tie(y,e)=gr[x][it++];
    if(!eu[e]){
      D[x]--,D[y]++;
      eu[e]=1;s.push_back(y);
    }}
  for(int x:D)if(x<0||sz(ret)!=nedges+1)return {};
  return {ret.rbegin(),ret.rend()};
}
```

### FunctionalGraph.h

**Description:** 函数图 (每个点 $i$ 恰有一条出边指向 $a[i]$, $0 \le a[i] < n$) 的分解. cycles[id] 为第 id 个环上的点 (沿出边顺序). root_of[v] = {环编号, 环中位置}, 对应的点 root 是从 $v$ 出发沿出边到达的第一个环上点. childs 为不在环上的边反向后构成的森林 (childs[$a[v]$] 含 $v$).
**Usage:** `auto [root_of, cycles, childs] = func_graph(a);`
`auto [cyc_id, cyc_pos] = root_of[v];`
`int root = cycles[cyc_id][cyc_pos]; bool on_cyc = (v == root);`
**Time:** $\mathcal{O}(N)$

fa0484, 26 lines

```
struct func_graph{
  vector<pii>root_of;
  vector<vi>cycles,childs;
  func_graph(const vi&a):
    root_of(sz(a)),childs(sz(a)){
    vi vis(sz(a));
    rep(i,0,sz(a)){
      if(!vis[i]){
        int u=i;
        for(;!vis[u];u=a[u])vis[u]=1;
        if(vis[u]==1)
          for(cycles.pb({});vis[u]==1;u=a[u]){
            root_of[u]={sz(cycles)-1,
                        sz(cycles.back())};
            cycles.back().pb(u);
            vis[u]=2;
          }
        for(int v=i;vis[v]==1;v=a[v]){
          root_of[v]=root_of[u];
          childs[a[v]].pb(v);
          vis[v]=2;
        }
      }
    }
  }
};
```

## 8.2 Coloring

### EdgeColoring.h

**Description:** 给定最大度为 $D$ 的简单无向图, 求边的 $(D+1)$ 染色, 使相邻的边颜色不同. ($D$ 染色是 NP-hard 的, 但二分图可以通过 反复对最大度的点求匹配做到.)
**Usage:** `vector<pii> eds; // 边 (u, v), 点编号 0..N-1`
`vi col = edgeColoring(N, eds); // col[i] 为边 i 的颜色`
**Time:** $\mathcal{O}(NM)$

e210e2, 31 lines

```
vi edgeColoring(int N,vector<pii>eds){
  vi cc(N+1),ret(sz(eds)),fan(N),free(N),loc;
  for(pii e:eds)++cc[e.first],++cc[e.second];
  int u,v,ncols=*max_element(all(cc))+1;
  vector<vi>adj(N,vi(ncols,-1));
  for(pii e:eds){
    tie(u,v)=e;
    fan[0]=v;
    loc.assign(ncols,0);
    int at=u,end=u,d,c=free[u],ind=0,i=0;
    while(d=free[v],!loc[d]&&(v=adj[u][d])!=-1)
      loc[d]=++ind,cc[ind]=d,fan[ind]=v;
    cc[loc[d]]=c;
    for(int cd=d;at!=-1;cd^=c^d,at=adj[at][cd])
      swap(adj[at][cd],adj[end=at][cd^c^d]);
    while(adj[fan[i]][d]!=-1){
      int left=fan[i],right=fan[++i],e=cc[i];
      adj[u][e]=left;
      adj[left][e]=u;
      adj[right][e]=-1;
      free[right]=e;
    }
    adj[u][d]=fan[i];
    adj[fan[i]][d]=u;
    for(int y:{fan[0],u,end})
      for(int&z=free[y]=0;adj[y][z]!=-1;z++);
  }
  rep(i,0,sz(eds))
    for(tie(u,v)=eds[i];adj[u][ret[i]]!=v;)++ret[i];
  return ret;
}
```

## 8.3 Heuristics

### MaximalCliques.h

**Description:** 对图中每个极大团调用回调 (图以对称的 bitset 邻接矩阵给出, 不允许自环). 回调的参数是表示该极大团的 bitset.
**Usage:** `vector<B> eds(n); // eds[i][j] = 1 表示有边`
`cliques(eds, [&](B r) { ... }); // r 为一个极大团`
**Time:** $\mathcal{O}\left(3^{n/3}\right)$, 稀疏图上快得多

b0d5b1, 12 lines

```
typedef bitset<128>B;
template<class F>
void cliques(vector<B>&eds,F f,B P=~B(),B X={},B R={}){
  if(!P.any()){if(!X.any())f(R);return;}
  auto q=(P|X)._Find_first();
  auto cands=P&~eds[q];
  rep(i,0,sz(eds))if(cands[i]){
    R[i]=1;
    cliques(eds,f,P&eds[i],X&eds[i],R);
    R[i]=P[i]=0;X[i]=1;
  }
}
```

### MaximumClique.h

**Description:** 快速求图的最大团 (图以对称的 bitset 矩阵给出, 不允许自环). 对补图求最大团即可得到最大独立集.
**Usage:** `vb g(n); // g[i][j] = 1 表示有边 , 需对称`
`vi c = Maxclique(g).maxClique(); // 最大团的点`
**Time:** 对 n=155 的最坏随机图 (p=.90) 约 1s. 稀疏图上更快.

f7c0bc, 49 lines

```
typedef vector<bitset<200>>vb;
struct Maxclique{
  double limit=0.025,pk=0;
  struct Vertex{int i,d=0;};
  typedef vector<Vertex>vv;
  vb e;
  vv V;
  vector<vi>C;
  vi qmax,q,S,old;
  void init(vv&r){
    for(auto&v:r)v.d=0;
    for(auto&v:r)for(auto j:r)v.d+=e[v.i][j.i];
    sort(all(r),[](auto a,auto b){return a.d>b.d;});
    int mxD=r[0].d;
    rep(i,0,sz(r))r[i].d=min(i,mxD)+1;
  }
  void expand(vv&R,int lev=1){
    S[lev]+=S[lev-1]-old[lev];
    old[lev]=S[lev-1];
    while(sz(R)){
      if(sz(q)+R.back().d<=sz(qmax))return;
      q.push_back(R.back().i);
      vv T;
      for(auto v:R)if(e[R.back().i][v.i])T.push_back({v.i});
      if(sz(T)){
        if(S[lev]++/++pk<limit)init(T);
        int j=0,mxk=1,mnk=max(sz(qmax)-sz(q)+1,1);
        C[1].clear(),C[2].clear();
        for(auto v:T){
          int k=1;
          auto f=[&](int i){return e[v.i][i];};
          while(any_of(all(C[k]),f))k++;
          if(k>mxk)mxk=k,C[mxk+1].clear();
          if(k<mnk)T[j++].i=v.i;
          C[k].push_back(v.i);
        }
        if(j>0)T[j-1].d=0;
        rep(k,mnk,mxk+1)for(int i:C[k])
          T[j].i=i,T[j++].d=k;
        expand(T,lev+1);
      }else if(sz(q)>sz(qmax))qmax=q;
      q.pop_back(),R.pop_back();
    }
  }
  vi maxClique(){init(V),expand(V);return qmax;}
  Maxclique(vb conn):e(conn),C(sz(e)+1),S(sz(C)),old(S){
    rep(i,0,sz(e))V.push_back({i});
  }
};
```

### MaximumIndependentSet.h

**Description:** 求一般图的最大独立集: 对补图求最大团 (见 MaximumClique). 若图是二分图, 见 MinimumVertexCover.
**Usage:** `vb h(n); // 补图`
`rep(i,0,n) rep(j,0,n) h[i][j] = i != j && !g[i][j];`
`vi s = Maxclique(h).maxClique(); // 即最大独立集`

## 8.4 Trees

### BinaryLifting.h

**Description:** 预处理树上 2 的幂次跳跃表 (倍增), 支持快速向上跳和求 LCA. 要求根的父亲是它自己.
**Usage:** `vi P(n); // P[v] 为父亲 , P[root] = root`
`vector<vi> tbl = treeJump(P); int u = jmp(tbl, v, k);`
`int w = lca(tbl, depth, a, b); // depth 为深度数组 vi`
**Time:** 预处理 $\mathcal{O}(N \log N)$, 查询 $\mathcal{O}(\log N)$

bfce85, 25 lines

```
vector<vi>treeJump(vi&P){
  int on=1,d=1;
  while(on<sz(P))on*=2,d++;
  vector<vi>jmp(d,P);
  rep(i,1,d)rep(j,0,sz(P))
    jmp[i][j]=jmp[i-1][jmp[i-1][j]];
  return jmp;
}

int jmp(vector<vi>&tbl,int nod,int steps){
  rep(i,0,sz(tbl))
    if(steps&(1<<i))nod=tbl[i][nod];
  return nod;
}

int lca(vector<vi>&tbl,vi&depth,int a,int b){
  if(depth[a]<depth[b])swap(a,b);
  a=jmp(tbl,a,depth[a]-depth[b]);
  if(a==b)return a;
  for(int i=sz(tbl);i--;){
    int c=tbl[i][a],d=tbl[i][b];
    if(c!=d)a=c,b=d;
  }
  return tbl[0][a];
}
```

### LCA.h

**Description:** 求树上最近公共祖先 (LCA) 的数据结构 (以 'rt' 为根). C 为树的邻接表, 有向或无向均可.
**Usage:** `LCA l(C, rt); int w = l.lca(a, b);`
**Time:** $\mathcal{O}(N \log N + Q)$

"../data-structures/RMQ.h" 1153c0, 21 lines

```
struct LCA{
  int T=0;
  vi time,path,ret;
  RMQ<int>rmq;

  LCA(vector<vi>&C,int rt):time(sz(C)),rmq((dfs(C,rt,-1),ret)){
      }
  void dfs(vector<vi>&C,int v,int par){
    time[v]=T++;
    for(int y:C[v])if(y!=par){
      path.push_back(v),ret.push_back(time[v]);
      dfs(C,y,v);
    }
  }

  int lca(int a,int b){
    if(a==b)return a;
    tie(a,b)=minmax(time[a],time[b]);
    return path[rmq.query(a,b)];
  }
  //dist(a,b){return depth[a] + depth[b] - 2*depth[lca(a,b)];}
};
```

### CompressTree.h

**Description:** 虚树: 给定有根树和点集 S, 加入所有两两的 LCA (至多 $|S|-1$ 个) 并压缩边, 得到包含 S 中所有点的最小子树. 返回 (par, orig_index) 的列表, 表示一棵以 0 为根的树, 其中 par 是该列表中的下标, 根指向自己.
**Usage:** `LCA lca(C, rt); vpi t = compressTree(lca, S);`
`// t[i] = (par, orig) , par 为 t 中的下标 , t[0] 为根`
**Time:** $\mathcal{O}(|S| \log |S|)$

"LCA.h" 9775a0, 21 lines

```
typedef vector<pair<int,int>>vpi;
vpi compressTree(LCA&lca,const vi&subset){
  static vi rev;rev.resize(sz(lca.time));
  vi li=subset,&T=lca.time;
  auto cmp=[&](int a,int b){return T[a]<T[b];};
  sort(all(li),cmp);
  int m=sz(li)-1;
  rep(i,0,m){
    int a=li[i],b=li[i+1];
    li.push_back(lca.lca(a,b));
  }
  sort(all(li),cmp);
  li.erase(unique(all(li)),li.end());
  rep(i,0,sz(li))rev[li[i]]=i;
  vpi ret={pii(0,li[0])};
  rep(i,0,sz(li)-1){
    int a=li[i],b=li[i+1];
    ret.emplace_back(rev[lca.lca(a,b)],b);
  }
  return ret;
}
```

### HLD.h

**Description:** 树链剖分: 把树分解为点不相交的重链和轻边, 使任意叶子到根的路径上 至多有 log(n) 条轻边. 输入邻接表 (若邻接表中没有指向父亲的边, 注释掉 dfsSz 中的 (*) 行), 初值数组 $v$ 和根. VALS_EDGES 为真表示值存在边上而不是点上.
**Usage:** `HLD<false> h(adj, vals, root); h.modifyPath(u, v, val);`
`T x = h.queryPath(u, v), y = h.querySubtree(v);`
**Time:** $\mathcal{O}\left((\log N)^2\right)$

"../data-structures/LazySegmentTree.h" ba77fc, 55 lines

```
template<bool VALS_EDGES>struct HLD{
    int N,tim=0;
    vector<vi>adj;
    vi par,siz,rt,pos,tail;
    lztree tree;
    HLD(vector<vi>&adj,vector<T>v,int root)
        :N(sz(adj)),adj(adj),par(N,-1),siz(N,1),
          rt(N,root),pos(N),tail(N,-1),tree(N){
        dfsSz(root);dfsHld(root);
        rep(i,root,sz(v))tree.set(pos[i],v[i]); // 初始化值
    }
  void dfsSz(int v){
    for(int&u:adj[v]){
      adj[u].erase(find(all(adj[u]),v)); // (*)
      par[u]=v;
      dfsSz(u);
      siz[v]+=siz[u];
      if(siz[u]>siz[adj[v][0]])swap(u,adj[v][0]);
    }
  }
    void dfsHld(int v){
        pos[v]=tim++,tail[v]=v;
        for(bool fst=1;int u:adj[v]){
            rt[u]=(u==adj[v][0]?rt[v]:u);
            dfsHld(u);
            if(fst)tail[v]=tail[u];
            fst=0;
        }
    }
  template<class B>void process(int u,int v,B op){
    for(;;v=par[rt[v]]){
      if(pos[u]>pos[v])swap(u,v);
      if(rt[u]==rt[v])break;
      op(pos[rt[v]],pos[v]+1);
    }
    op(pos[u]+VALS_EDGES,pos[v]+1);
  }
    T get(int u){ // 单点修改类似
        return tree.get(pos[u]);
    }
  void modifyPath(int u,int v,U val){
    process(u,v,[&](int l,int r){tree.update(l,r,val);});
  }
  // 查整条链用 tree.query(pos[rt[v]], pos[tail[v]] + 1)
    T queryPath(int u,int v){ // 按题意修改
        T res=idT; // 单位元
        process(u,v,[&](int l,int r){ // 合并区间答案
                res=f(res,tree.query(l,r));
        });
        return res;
    }
    T querySubtree(int v){ // modifySubtree 类似
        return tree.query(pos[v]+VALS_EDGES,pos[v]+siz[v]);
    }
};
```

### LinkCutTree.h

**Description:** 维护无根树构成的森林. 可以加边和删边 (要求结果仍是森林), 并查询两点是否在同一棵树中.
**Usage:** `LinkCut lc(N); lc.link(u, v); lc.cut(u, v);`
`bool b = lc.connected(u, v);`
**Time:** 所有操作均摊 $\mathcal{O}(\log N)$.

0fb462, 90 lines

```
struct Node{ // Splay 树 . 根的 pp 存原树中的父亲
  Node*p=0,*pp=0,*c[2];
  bool flip=0;
  Node(){c[0]=c[1]=0;fix();}
  void fix(){
    if(c[0])c[0]->p=this;
    if(c[1])c[1]->p=this;
    // 需要时在此更新子树和等信息
  }
  void pushFlip(){
    if(!flip)return;
    flip=0;swap(c[0],c[1]);
    if(c[0])c[0]->flip^=1;
    if(c[1])c[1]->flip^=1;
  }
  int up(){return p?p->c[1]==this:-1;}
  void rot(int i,int b){
    int h=i^b;
    Node*x=c[i],*y=b==2?x:x->c[h],*z=b?y:x;
    if((y->p=p))p->c[up()]=y;
    c[i]=z->c[i^1];
    if(b<2){
      x->c[h]=y->c[h^1];
      y->c[h^1]=x;
    }
    z->c[i^1]=this;
    fix();x->fix();y->fix();
    if(p)p->fix();
    swap(pp,y->pp);
  }
  void splay(){
    for(pushFlip();p;){
      if(p->p)p->p->pushFlip();
      p->pushFlip();pushFlip();
      int c1=up(),c2=p->up();
      if(c2==-1)p->rot(c1,2);
      else p->p->rot(c2,c1!=c2);
    }
  }
```

```
  Node*first(){
    pushFlip();
    return c[0]?c[0]->first():(splay(),this);
  }
};

struct LinkCut{
  vector<Node>node;
  LinkCut(int N):node(N){}

  void link(int u,int v){ // 加边 (u, v)
    assert(!connected(u,v));
    makeRoot(&node[u]);
    node[u].pp=&node[v];
  }
  void cut(int u,int v){ // 删边 (u, v)
    Node*x=&node[u],*top=&node[v];
    makeRoot(top);x->splay();
    assert(top==(x->pp?:x->c[0]));
    if(x->pp)x->pp=0;
    else{
      x->c[0]=top->p=0;
      x->fix();
    }
  }
  bool connected(int u,int v){ // u , v 是否在同一棵树
    Node*nu=access(&node[u])->first();
    return nu==access(&node[v])->first();
  }
  void makeRoot(Node*u){
    access(u);
    u->splay();
    if(u->c[0]){
      u->c[0]->p=0;
      u->c[0]->flip^=1;
      u->c[0]->pp=u;
      u->c[0]=0;
      u->fix();
    }
  }
  Node*access(Node*u){
    u->splay();
    while(Node*pp=u->pp){
      pp->splay();u->pp=0;
      if(pp->c[1]){
        pp->c[1]->p=0;pp->c[1]->pp=pp;}
      pp->c[1]=u;pp->fix();u=pp;
    }
    return u;
  }
};
```

DirectedMST.h

**Description:** 给定根, 求有向图的最小树形图 (有向 MST). 若不存在则返回 -1.

**Usage:** `vector<Edge> g{{a, b, w}}; // 从 a 到 b 的边`
`auto [cost, par] = dmst(n, r, g); // 无解时 cost = -1`
`par[v] 为 v 的父亲 , par[r] = -1`

**Time:** $\mathcal{O}(E \log V)$

"../data-structures/UnionFindRollback.h" 39e620, 60 lines

```
struct Edge{int a,b;ll w;};
struct Node{
  Edge key;
  Node*l,*r;
  ll delta;
  void prop(){
    key.w+=delta;
    if(l)l->delta+=delta;
    if(r)r->delta+=delta;
    delta=0;
  }
  Edge top(){prop();return key;}
};
Node*merge(Node*a,Node*b){
  if(!a||!b)return a?:b;
  a->prop(),b->prop();
  if(a->key.w>b->key.w)swap(a,b);
  swap(a->l,(a->r=merge(b,a->r)));
  return a;
}
void pop(Node*&a){a->prop();a=merge(a->l,a->r);}

pair<ll,vi>dmst(int n,int r,vector<Edge>&g){
  RollbackUF uf(n);
  vector<Node*>heap(n);
  for(Edge e:g)heap[e.b]=merge(heap[e.b],new Node{e});
  ll res=0;
  vi seen(n,-1),path(n),par(n);
  seen[r]=r;
  vector<Edge>Q(n),in(n,{-1,-1}),comp;
  deque<tuple<int,int,vector<Edge>>>cycs;
  rep(s,0,n){
    int u=s,qi=0,w;
    while(seen[u]<0){
      if(!heap[u])return {-1,{}};
      Edge e=heap[u]->top();
      heap[u]->delta-=e.w,pop(heap[u]);
      Q[qi]=e,path[qi++]=u,seen[u]=s;
      res+=e.w,u=uf.find(e.a);
      if(seen[u]==s){
        Node*cyc=0;
        int end=qi,time=uf.time();
        do cyc=merge(cyc,heap[w=path[--qi]]);
        while(uf.join(u,w));
        u=uf.find(u),heap[u]=cyc,seen[u]=-1;
        cycs.push_front({u,time,{&Q[qi],&Q[end]}});
      }
    }
    rep(i,0,qi)in[uf.find(Q[i].b)]=Q[i];
  }

  for(auto&[u,t,comp]:cycs){ // 还原方案 , 可选
    uf.rollback(t);
    Edge inEdge=in[u];
    for(auto&e:comp)in[uf.find(e.b)]=e;
    in[uf.find(inEdge.b)]=inEdge;
  }
  rep(i,0,n)par[i]=in[i].a;
  return {res,par};
}
```

## 8.5 Math

### 8.5.1 Number of Spanning Trees

Create an $N \times N$ matrix `mat`, and for each edge $a \to b \in G$, do `mat[a][b]--`, `mat[b][b]++` (and `mat[b][a]--`, `mat[a][a]++` if $G$ is undirected). Remove the $i$th row and column and take the determinant; this yields the number of directed spanning trees rooted at $i$ (if $G$ is undirected, remove any row/-column).

### 8.5.2 Erdős–Gallai theorem

A simple graph with node degrees $d_1 \geq \cdots \geq d_n$ exists iff $d_1 + \cdots + d_n$ is even and for every $k = 1 \ldots n$,

$$\sum_{i=1}^{k} d_i \leq k(k-1) + \sum_{i=k+1}^{n} \min(d_i, k).$$

# Geometry (9)

## 9.1 Geometric primitives

Point.h

**Description:** 平面点类. T 可取 double 或 long long (避免用 int).

**Usage:** `typedef Point<double> P; P a(1, 2), b(3, 4);`
`double d = (b-a).dist(), c = a.cross(b); // 叉积`

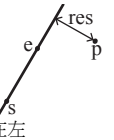
47ec0a, 28 lines

```
template<class T>int sgn(T x){return (x>0)-(x<0);}
template<class T>
struct Point{
  typedef Point P;
  T x,y;
  explicit Point(T x=0,T y=0):x(x),y(y){}
  bool operator<(P p)const{return tie(x,y)<tie(p.x,p.y);}
  bool operator==(P p)const{return tie(x,y)==tie(p.x,p.y);}
  P operator+(P p)const{return P(x+p.x,y+p.y);}
  P operator-(P p)const{return P(x-p.x,y-p.y);}
  P operator*(T d)const{return P(x*d,y*d);}
  P operator/(T d)const{return P(x/d,y/d);}
  T dot(P p)const{return x*p.x+y*p.y;}
  T cross(P p)const{return x*p.y-y*p.x;}
  T cross(P a,P b)const{return (a-*this).cross(b-*this);}
  T dist2()const{return x*x+y*y;}
  double dist()const{return sqrt((double)dist2());}
  // 与 x 轴的夹角取值 [-pi, pi]
  double angle()const{return atan2(y,x);}
  P unit()const{return *this/dist();} // 长度变为 1
  P perp()const{return P(-y,x);} // 逆时针转 90 度
  P normal()const{return perp().unit();}
  // 绕原点逆时针旋转 a 弧度
  P rotate(double a)const{
    return P(x*cos(a)-y*sin(a),x*sin(a)+y*cos(a));}
  friend ostream&operator<<(ostream&os,P p){
    return os<<"("<<p.x<<","<<p.y<<")";}
};
```

lineDistance.h

**Description:**

返回点 p 到直线 ab 的有向距离. 从 a 看向 b, 左侧为正, 右侧为负; a==b 时得 nan. P 可为 Point<T> 或 Point3D<T>, T 如 double 或 long long. 中间有乘积, 用 int 或 long long 时注意溢出. Point3D 总得非负距离; 对 Point3D 需对叉积结果调用 .dist.

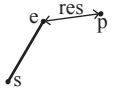



**Usage:** `double d = lineDist(a, b, p); // 正数表示 p 在左侧`

"Point.h" f6bf6b, 4 lines

```
template<class P>
double lineDist(const P&a,const P&b,const P&p){
  return (double)(b-a).cross(p-a)/(b-a).dist();
}
```

## SegmentDistance.h




**Description:**

返回点 p 到线段 se 的最短距离.

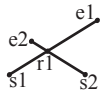
**Usage:** `Point<double> a, b(2,2), p(1,1);`
`bool onSegment = segDist(a,b,p) < 1e-10; // p 在线段上`

"Point.h" 5c88f4, 6 lines

```
typedef Point<double>P;
double segDist(P&s,P&e,P&p){
  if(s==e)return (p-s).dist();
  auto d=(e-s).dist2(),t=min(d,max(.0,(p-s).dot(e-s)));
  return ((p-s)*d-(e-s)*t).dist()/d;
}
```

## SegmentIntersection.h

**Description:**




若线段 s1-e1 与 s2-e2 有唯一交点, 返回只含该点的 vector; 无交点返回空 vector; 有无穷多交点时返回 2 个元素, 即公共线段的两个端点. 若 P 为 Point<ll> 而交点不是整点, 结果错误. 中间有三个坐标的乘积, 用 int 或 long long 时注意溢出.

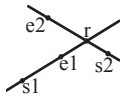
**Usage:** `vector<P> inter = segInter(s1,e1,s2,e2);`
`if (sz(inter)==1) // 唯一交点`
`cout << inter[0] << endl;`

"Point.h", "OnSegment.h" 9d57f2, 13 lines

```
template<class P>vector<P>segInter(P a,P b,P c,P d){
  auto oa=c.cross(d,a),ob=c.cross(d,b),
       oc=a.cross(b,c),od=a.cross(b,d);
  // 交点唯一且不是端点
  if(sgn(oa)*sgn(ob)<0&&sgn(oc)*sgn(od)<0)
    return {(a*ob-b*oa)/(ob-oa)};
  set<P>s;
  if(onSegment(c,d,a))s.insert(a);
  if(onSegment(c,d,b))s.insert(b);
  if(onSegment(a,b,c))s.insert(c);
  if(onSegment(a,b,d))s.insert(d);
  return {all(s)};
}
```

## lineIntersection.h

**Description:**




若过 s1,e1 的直线与过 s2,e2 的直线有唯一交点, 返回 {1, 交点}; 无交点 (平行) 返回 {0, (0,0)}; 有无穷多交点 (重合) 返回 {-1, (0,0)}. 若 P 为 Point<ll> 而交点不是整点, 结果错误. 中间有三个坐标的乘积, 用 int 或 ll 时注意溢出.

**Usage:** `auto res = lineInter(s1,e1,s2,e2);`
`if (res.first == 1) // 交点为 res.second`
`cout << res.second << endl;`

"Point.h" a01f81, 8 lines

```
template<class P>
pair<int,P>lineInter(P s1,P e1,P s2,P e2){
  auto d=(e1-s1).cross(e2-s2);
  if(d==0) // 平行
    return {-(s1.cross(e1,s2)==0),P(0,0)};
  auto p=s2.cross(e1,e2),q=s2.cross(e2,s1);
  return {1,(s1*p+e1*q)/d};
}
```

## sideOf.h

**Description:** 返回从 $s$ 看向 $e$ 时 $p$ 所在的位置. 1/0/-1 ⇔ 左侧/在直线上/右侧. 若给出可选参数 $eps$, 则 $p$ 到直线距离不超过 $eps$ 时返回 0. P 应为 Point<T>, T 如 double 或 long long. 中间有乘积, 用 int 或 long long 时注意溢出.

**Usage:** `bool left = sideOf(p1,p2,q)==1; // q 在 p1 到 p2 的左侧`

"Point.h" 3af81c, 9 lines

```
template<class P>
int sideOf(P s,P e,P p){return sgn(s.cross(e,p));}

template<class P>
int sideOf(const P&s,const P&e,const P&p,double eps){
  auto a=(e-s).cross(p-s);
  double l=(e-s).dist()*eps;
  return (a>l)-(a<-l);
}
```

## OnSegment.h

**Description:** 当且仅当 p 在线段 se 上时返回 true. 用 Point<double> 时改用 (segDist(s,e,p)<=epsilon).

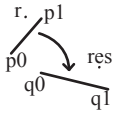
**Usage:** `bool on = onSegment(s, e, p);`

"Point.h" c597e8, 3 lines

```
template<class P>bool onSegment(P s,P e,P p){
  return p.cross(s,e)==0&&(s-p).dot(e-p)<=0;
}
```

## linearTransformation.h

**Description:**




对点 r 施加把直线 p0-p1 变到直线 q0-q1 的线性变换 (平移, 旋转和缩放).

**Usage:** `P r2 = linearTransformation(p0, p1, q0, q1, r);`

"Point.h" 03a306, 6 lines

```
typedef Point<double>P;
P linearTransformation(const P&p0,const P&p1,
    const P&q0,const P&q1,const P&r){
  P dp=p1-p0,dq=q1-q0,num(dp.cross(dq),dp.dot(dq));
  return q0+P((r-p0).cross(num),(r-p0).dot(num))/dp.dist2();
}
```

## Angle.h

**Description:** 用于给角度排序的类 (角度用 int 点加上绕原点的圈数表示). 适用于极角扫描. 有时也用来表示点或向量.

**Usage:** `vector<Angle> v = {w[0], w[0].t360() ...};`
`sort(all(v));`
`int j = 0; rep(i,0,n) { while (v[j] < v[i].t180()) ++j; }`
`// 扫描 j, 使 j-i 等于以 0 和 i 为顶点的正向三角形个数`

0f0602, 35 lines

```
struct Angle{
  int x,y;
  int t;
  Angle(int x,int y,int t=0):x(x),y(y),t(t){}
  Angle operator-(Angle b)const{return {x-b.x,y-b.y,t};}
  int half()const{ // 角在 [pi, 2pi) 内时为真
    assert(x||y); // 可按需改哪一半算较大
    return y<0||(y==0&&x<0);
  }
  Angle t90()const{return {-y,x,t+(half()&&x>=0)};}
  Angle t180()const{return {-x,-y,t+half()};}
  Angle t360()const{return {x,y,t+1};}
};
bool operator<(Angle a,Angle b){ // 比较 [0, 2pi) 内的角
  // 加入 a.dist2() 和 b.dist2() 可同时比较距离
  return make_tuple(a.t,a.half(),a.y*(ll)b.x)<
         make_tuple(b.t,b.half(),a.x*(ll)b.y);
}

// 给定两点求它们之间的最小角
// 即覆盖这条线段的角
pair<Angle,Angle>segmentAngles(Angle a,Angle b){
  if(b<a)swap(a,b);
  return (b<a.t180()?
          make_pair(a,b):make_pair(b,a.t360()));
}
Angle operator+(Angle a,Angle b){ // 点 a 加向量 b
  Angle r(a.x+b.x,a.y+b.y,a.t);
  if(a.t180()<r)r.t--;
  return r.t180()<a?r.t360():r;
}
Angle angleDiff(Angle a,Angle b){ // 角 b 减去角 a
  int tu=b.t-a.t;a.t=b.t;
  return {a.x*b.x+a.y*b.y,a.x*b.y-a.y*b.x,tu-(b<a)};
}
```

# 9.2 Circles

## CircleIntersection.h

**Description:** 求两圆的两个交点, 写入 out. 无交点时返回 false. 两圆同心时要求 r1 != r2 (有 assert).

**Usage:** `pair<P, P> out; bool ok = circleInter(a, b, r1, r2, &out);`

"Point.h" 84d6d3, 11 lines

```
typedef Point<double>P;
bool circleInter(P a,P b,double r1,double r2,pair<P,P>*out){
  if(a==b){assert(r1!=r2);return false;}
  P vec=b-a;
  double d2=vec.dist2(),sum=r1+r2,dif=r1-r2,
         p=(d2+r1*r1-r2*r2)/(d2*2),h2=r1*r1-p*p*d2;
  if(sum*sum<d2||dif*dif>d2)return false;
  P mid=a+vec*p,per=vec.perp()*sqrt(fmax(0,h2)/d2);
  *out={mid+per,mid-per};
  return true;
}
```

## CircleTangents.h

**Description:** 求两圆的外公切线; 把 r2 取负则求内公切线. 可能返回 0, 1 或 2 条切线 – 0 条: 一圆包含另一圆 (求内公切线时还包括两圆相交), 或两圆相同; 1 条: 两圆相切 (此时 .first = .second, 切线垂直于两圆心连线). .first 和 .second 分别是圆 1 和圆 2 上的切点. 求点到圆的切线时令 r2 = 0.

**Usage:** `vector<pair<P, P>> v = tangents(c1, r1, c2, r2);`
`// 圆 1 上切点 v[i].first, 圆 2 上切点 v[i].second`

"Point.h" b0153d, 13 lines

```
template<class P>
vector<pair<P,P>>tangents(P c1,double r1,P c2,double r2){
  P d=c2-c1;
  double dr=r1-r2,d2=d.dist2(),h2=d2-dr*dr;
  if(d2==0||h2<0)return {};
  vector<pair<P,P>>out;
  for(double sign:{-1,1}){
    P v=(d*dr+d.perp()*sqrt(h2)*sign)/d2;
    out.push_back({c1+v*r1,c2+v*r2});
  }
  if(h2==0)out.pop_back();
  return out;
}
```

## CirclePolygonIntersection.h

**Description:** 求圆与逆时针 (ccw) 多边形的交集面积.

**Usage:** `double a = circlePoly(c, r, ps); // ps 为逆时针多边形`

**Time:** $\mathcal{O}(n)$

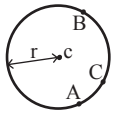
"../../content/geometry/Point.h" 19add1, 19 lines

```
typedef Point<double>P;
#define arg(p, q) atan2(p.cross(q), p.dot(q))
double circlePoly(P c,double r,vector<P>ps){
  auto tri=[&](P p,P q){
    auto r2=r*r/2;
    P d=q-p;
    auto a=d.dot(p)/d.dist2(),b=(p.dist2()-r*r)/d.dist2();
    auto det=a*a-b;
```

```
    if(det<=0)return arg(p,q)*r2;
    auto s=max(0.,-a-sqrt(det)),t=min(1.,-a+sqrt(det));
    if(t<0||1<=s)return arg(p,q)*r2;
    P u=p+d*s,v=q+d*(t-1);
    return arg(p,u)*r2+u.cross(v)/2+arg(v,q)*r2;
  };
  auto sum=0.0;
  rep(i,0,sz(ps))
    sum+=tri(ps[i]-c,ps[(i+1)%sz(ps)]-c);
  return sum;
}
```

### circumcircle.h

**Description:**




三角形的外接圆是经过三个顶点的圆. ccRadius 返回过 A, B, C 三点的圆的半径, ccCenter 返回该圆的圆心.

**Usage:** `double r = ccRadius(A, B, C); P o = ccCenter(A, B, C);`

"Point.h" 1caa3a, 9 lines

```
typedef Point<double>P;
double ccRadius(const P&A,const P&B,const P&C){
  return (B-A).dist()*(C-B).dist()*(A-C).dist()/
      abs((B-A).cross(C-A))/2;
}
P ccCenter(const P&A,const P&B,const P&C){
  P b=C-A,c=B-A;
  return A+(b*c.dist2()-c*b.dist2()).perp()/b.cross(c)/2;
}
```

### MinimumEnclosingCircle.h

**Description:** 求覆盖点集的最小圆.
**Usage:** `auto [o, r] = mec(ps); // 圆心 o, 半径 r`
**Time:** 期望 $\mathcal{O}(n)$

"circumcircle.h" 09dd0a, 17 lines

```
pair<P,double>mec(vector<P>ps){
  shuffle(all(ps),mt19937(time(0)));
  P o=ps[0];
  double r=0,EPS=1+1e-8;
  rep(i,0,sz(ps))if((o-ps[i]).dist()>r*EPS){
    o=ps[i],r=0;
    rep(j,0,i)if((o-ps[j]).dist()>r*EPS){
      o=(ps[i]+ps[j])/2;
      r=(o-ps[i]).dist();
      rep(k,0,j)if((o-ps[k]).dist()>r*EPS){
        o=ccCenter(ps[i],ps[j],ps[k]);
        r=(o-ps[i]).dist();
      }
    }
  }
  return {o,r};
}
```

## 9.3 Polygons

### InsidePolygon.h

**Description:** 若 p 在多边形内则返回 true. strict 为 true 时, 边界上的点返回 false. 中间有乘积, 注意溢出.
**Usage:** `vector<P> v = {P{4,4}, P{1,2}, P{2,1}}; bool in = inPolygon(v, P{3, 3}, false); // 含边界`
**Time:** $\mathcal{O}(n)$

"Point.h", "OnSegment.h", "SegmentDistance.h" 2bf504, 12 lines

```
template<class P> // 顺时针或逆时针多边形均可
bool inPolygon(vector<P>&p,P a,bool strict=true){
  int cnt=0,n=sz(p);
  rep(i,0,n){
    P q=p[(i+1)%n];
    if(onSegment(p[i],q,a))return !strict;
    // 浮点数时可改用下一行
    //if (segDist(p[i], q, a) <= eps) return !strict;
    cnt^=((a.y<p[i].y)-(a.y<q.y))*a.cross(p[i],q)>0;
  }
  return cnt;
}
```

### PolygonArea.h

**Description:** 返回多边形有向面积的两倍. 顶点按顺时针给出时面积为负. T 为 int 时注意溢出!
**Usage:** `vector<Point<ll>> v = {...}; // 多边形顶点`
`ll a2 = polygonArea2(v); // 有向面积的 2 倍`

"Point.h" 95fc8f, 7 lines

```
template<class T>
T polygonArea2(vector<Point<T>>&v){
  assert(!v.empty()); // 或返回 0
  T a=v.back().cross(v[0]);
  rep(i,0,sz(v)-1)a+=v[i].cross(v[i+1]);
  return a;
}
```

### PolygonCenter.h

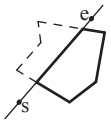
**Description:** 返回多边形的重心 (质心).
**Usage:** `P c = polygonCenter(v);`
**Time:** $\mathcal{O}(n)$

"Point.h" 9706dc, 9 lines

```
typedef Point<double>P;
P polygonCenter(const vector<P>&v){
  P res(0,0);double A=0;
  for(int i=0,j=sz(v)-1;i<sz(v);j=i++){
    res=res+(v[i]+v[j])*v[j].cross(v[i]);
    A+=v[j].cross(v[i]);
  }
  return res/A/3;
}
```

### PolygonCut.h

**Description:**
用从 s 到 e 的直线切割多边形, 切掉直线左侧的部分, 返回剩余多边形的顶点.




**Usage:** `vector<P> p = ...; p = polygonCut(p, P(0,0), P(1,0)); // 切掉 x 轴上方`

"Point.h" d07181, 13 lines

```
typedef Point<double>P;
vector<P>polygonCut(const vector<P>&poly,P s,P e){
  vector<P>res;
  rep(i,0,sz(poly)){
    P cur=poly[i],prev=i?poly[i-1]:poly.back();
    auto a=s.cross(e,cur),b=s.cross(e,prev);
    if((a<0)!=(b<0))
      res.push_back(cur+(prev-cur)*(a/(a-b)));
    if(a<0)
      res.push_back(cur);
  }
  return res;
}
```

### PolygonUnion.h

**Description:** 求 $n$ 个多边形 (不一定凸) 的并的面积. 每个多边形的顶点须按逆时针 (CCW) 给出. (可在 sideOf/sgn 中加 eps 判断, 但通常不需要.)
**Usage:** `double a = polyUnion(poly); // poly 为 vector<vector<P>>`
**Time:** $\mathcal{O}(N^2)$, 其中 $N$ 为总点数

"Point.h", "sideOf.h" 3931c6, 33 lines

```
typedef Point<double>P;
double rat(P a,P b){return sgn(b.x)?a.x/b.x:a.y/b.y;}
double polyUnion(vector<vector<P>>&poly){
  double ret=0;
  rep(i,0,sz(poly))rep(v,0,sz(poly[i])){
    P A=poly[i][v],B=poly[i][(v+1)%sz(poly[i])];
    vector<pair<double,int>>segs={{0,0},{1,0}};
    rep(j,0,sz(poly))if(i!=j){
      rep(u,0,sz(poly[j])){
        P C=poly[j][u],D=poly[j][(u+1)%sz(poly[j])];
        int sc=sideOf(A,B,C),sd=sideOf(A,B,D);
        if(sc!=sd){
          double sa=C.cross(D,A),sb=C.cross(D,B);
          if(min(sc,sd)<0)
            segs.emplace_back(sa/(sa-sb),sgn(sc-sd));
        }else if(!sc&&!sd&&j<i&&sgn((B-A).dot(D-C))>0){
          segs.emplace_back(rat(C-A,B-A),1);
          segs.emplace_back(rat(D-A,B-A),-1);
        }
      }
    }
    sort(all(segs));
    for(auto&s:segs)s.first=min(max(s.first,0.0),1.0);
    double sum=0;
    int cnt=segs[0].second;
    rep(j,1,sz(segs)){
      if(!cnt)sum+=segs[j].first-segs[j-1].first;
      cnt+=segs[j].second;
    }
    ret+=A.cross(B)*sum;
  }
  return ret/2;
}
```

### HalfPlane.h

**Description:** 求一组半平面的交. 输入的每条有向直线表示它左侧的半平面. 输出为表示交集的凸多边形, 其中的点可能重复或共线. 若 eps 大于 sqrt(2) 乘以直线求交误差, 不会严重出错. 若保证不会有 3 个半平面交于同一点, 则 eps 取更多值时也很可能正确.
**Usage:** `vector<Line> vs; vs.push_back({P(0,0), P(1,0)}); // 取左侧`
`vector<P> poly = halfPlaneIntersection(vs); // 可能为空`
**Time:** $\mathcal{O}(n \log n)$

"Point.h", "sideOf.h", "lineIntersection.h" eda44b, 31 lines

```
typedef Point<double>P;
typedef array<P,2>Line;
#define sp(a) a[0], a[1]
#define ang(a) (a[1] - a[0]).angle()

int angDiff(Line a,Line b){return sgn(ang(a)-ang(b));}
bool cmp(Line a,Line b){
  int s=angDiff(a,b);
  return (s?s:sideOf(sp(a),b[0]))<0;
}
vector<P>halfPlaneIntersection(vector<Line>vs){
  const double EPS=sqrt(2)*1e-8;
  sort(all(vs),cmp);
  vector<Line>deq(sz(vs)+5);
  vector<P>ans(sz(vs)+5);
  deq[0]=vs[0];
  int ah=0,at=0,n=sz(vs);
  rep(i,1,n+1){
    if(i==n)vs.push_back(deq[ah]);
    if(angDiff(vs[i],vs[i-1])==0)continue;
    while(ah<at&&sideOf(sp(vs[i]),ans[at-1],EPS)<0)
      at--;
```

```
    while(i!=n&&ah<at&&sideOf(sp(vs[i]),ans[ah],ans[at],EPS)<0)
      ah++;
    auto res=lineInter(sp(vs[i]),sp(deq[at]));
    if(res.first!=1)continue;
    ans[at++]=res.second,deq[at]=vs[i];
  }
  if(at-ah<=2)return {};
  return {ans.begin()+ah,ans.begin()+at};
}
```

## ConvexHull.h



**Description:**
按逆时针顺序返回凸包上的点. 位于凸包边上 (在另外两点之间) 的点不算在凸包内.
**Usage:** `vector<P> hull = convexHull(pts);`
**Time:** $\mathcal{O}(n \log n)$

"Point.h" 310954, 13 lines

```
typedef Point<ll>P;
vector<P>convexHull(vector<P>pts){
  if(sz(pts)<=1)return pts;
  sort(all(pts));
  vector<P>h(sz(pts)+1);
  int s=0,t=0;
  for(int it=2;it--;s=--t,reverse(all(pts)))
    for(P p:pts){
      while(t>=s+2&&h[t-2].cross(h[t-1],p)<=0)t--;
      h[t++]=p;
    }
  return {h.begin(),h.begin()+t-(t==2&&h[0]==h[1])};
}
```

## HullDiameter.h

**Description:** 返回凸包 (逆时针, 无重复点和共线点) 上距离最远的两点.
**Usage:** `array<P, 2> d = hullDiameter(hull);`
`ll D2 = (d[0]-d[1]).dist2(); // 直径的平方`
**Time:** $\mathcal{O}(n)$

"Point.h" c571b8, 12 lines

```
typedef Point<ll>P;
array<P,2>hullDiameter(vector<P>S){
  int n=sz(S),j=n<2?0:1;
  pair<ll,array<P,2>>res({0,{S[0],S[0]}});
  rep(i,0,j)
    for(;;j=(j+1)%n){
      res=max(res,{(S[i]-S[j]).dist2(),{S[i],S[j]}});
      if((S[(j+1)%n]-S[j]).cross(S[i+1]-S[i])>=0)
        break;
    }
  return res.second;
}
```

## PointInsideHull.h

**Description:** 判断点 p 是否在凸包 (逆时针, 无共线点) 内, 在内则返回 true. strict 为 true 时边界上的点不算在内.
**Usage:** `bool in = inHull(hull, p, false); // 含边界`
**Time:** $\mathcal{O}(\log N)$

"Point.h", "sideOf.h", "OnSegment.h" 71446b, 14 lines

```
typedef Point<ll>P;

bool inHull(const vector<P>&l,P p,bool strict=true){
  int a=1,b=sz(l)-1,r=!strict;
  if(sz(l)<3)return r&&onSegment(l[0],l.back(),p);
  if(sideOf(l[0],l[a],l[b])>0)swap(a,b);
  if(sideOf(l[0],l[a],p)>=r||sideOf(l[0],l[b],p)<=-r)
    return false;
  while(abs(a-b)>1){
    int c=(a+b)/2;
    (sideOf(l[0],l[c],p)>0?b:a)=c;
  }
  return sgn(l[a].cross(l[b],p))<r;
}
```

## LineHullIntersection.h

**Description:** 直线与凸多边形求交. 多边形须为逆时针且无共线点. lineHull(a, b, poly) 返回一对下标, 描述直线 ab 与多边形的相交情况: • $(-1,-1)$ 不相交, • $(i,-1)$ 只碰到顶点 $i$, • $(i,i)$ 沿着边 $(i,i+1)$, • $(i,j)$ 穿过边 $(i,i+1)$ 和 $(j,j+1)$. 最后一种情形中若穿过顶点 $i$, 视为发生在边 $(i,i+1)$ 上. 返回顺序与直线依次碰到多边形的顺序一致. extrVertex 返回凸包上在方向 dir 上投影最大的顶点下标.
**Usage:** `array<int, 2> r = lineHull(a, b, poly);`
`int i = extrVertex(poly, dir); // dir 方向上最远的顶点`
**Time:** $\mathcal{O}(\log n)$

"Point.h" 7cf45b, 39 lines

```
#define cmp(i,j) sgn(dir.perp().cross(poly[(i)%n]-poly[(j)%n]))
#define extr(i) cmp(i + 1, i) >= 0 && cmp(i, i - 1 + n) < 0
template<class P>int extrVertex(vector<P>&poly,P dir){
  int n=sz(poly),lo=0,hi=n;
  if(extr(0))return 0;
  while(lo+1<hi){
    int m=(lo+hi)/2;
    if(extr(m))return m;
    int ls=cmp(lo+1,lo),ms=cmp(m+1,m);
    (ls<ms||(ls==ms&&ls==cmp(lo,m))?hi:lo)=m;
  }
  return lo;
}

#define cmpL(i) sgn(a.cross(poly[i], b))
template<class P>
array<int,2>lineHull(P a,P b,vector<P>&poly){
  int endA=extrVertex(poly,(a-b).perp());
  int endB=extrVertex(poly,(b-a).perp());
  if(cmpL(endA)<0||cmpL(endB)>0)
    return {-1,-1};
  array<int,2>res;
  rep(i,0,2){
    int lo=endB,hi=endA,n=sz(poly);
    while((lo+1)%n!=hi){
      int m=((lo+hi+(lo<hi?0:n))/2)%n;
      (cmpL(m)==cmpL(endB)?lo:hi)=m;
    }
    res[i]=(lo+!cmpL(hi))%n;
    swap(endA,endB);
  }
  if(res[0]==res[1])return {res[0],-1};
  if(!cmpL(res[0])&&!cmpL(res[1]))
    switch((res[0]-res[1]+sz(poly)+1)%sz(poly)){
      case 0:return {res[0],res[0]};
      case 2:return {res[1],res[1]};
    }
  return res;
}
```

# 9.4 Misc. Point Set Problems

## ClosestPair.h

**Description:** 求平面最近点对.
**Usage:** `pair<P, P> pq = closest(v); // 要求 sz(v) > 1`
**Time:** $\mathcal{O}(n \log n)$

"Point.h" ac41a6, 17 lines

```
typedef Point<ll>P;
pair<P,P>closest(vector<P>v){
  assert(sz(v)>1);
  set<P>S;
  sort(all(v),[](P a,P b){return a.y<b.y;});
  pair<ll,pair<P,P>>ret{LLONG_MAX,{P(),P()}};
  int j=0;
  for(P p:v){
    P d{1+(ll)sqrt(ret.first),0};
    while(v[j].y<=p.y-d.x)S.erase(v[j++]);
    auto lo=S.lower_bound(p-d),hi=S.upper_bound(p+d);
    for(;lo!=hi;++lo)
      ret=min(ret,{(*lo-p).dist2(),{*lo,p}});
    S.insert(p);
  }
  return ret.second;
}
```

## kdTree.h

**Description:** KD 树 (二维, 可扩展到三维), 求最近点.
**Usage:** `KDTree kd(pts); pair<T, P> r = kd.nearest(p);`
`// 最近点为 r.second, 平方距离为 r.first`

"Point.h" bac5b0, 62 lines

```
typedef long long T;
typedef Point<T>P;
const T INF=numeric_limits<T>::max();

bool on_x(const P&a,const P&b){return a.x<b.x;}
bool on_y(const P&a,const P&b){return a.y<b.y;}

struct Node{
  P pt; // 若为叶子则存其唯一的点
  T x0=INF,x1=-INF,y0=INF,y1=-INF; // 包围盒
  Node*first=0,*second=0;

  T distance(const P&p){ // 到点 p 的最小平方距离
    T x=(p.x<x0?x0:p.x>x1?x1:p.x);
    T y=(p.y<y0?y0:p.y>y1?y1:p.y);
    return (P(x,y)-p).dist2();
  }

  Node(vector<P>&&vp):pt(vp[0]){
    for(P p:vp){
      x0=min(x0,p.x);x1=max(x1,p.x);
      y0=min(y0,p.y);y1=max(y1,p.y);
    }
    if(vp.size()>1){
      // 宽不小于高时按 x 划分
      sort(all(vp),x1-x0>=y1-y0?on_x:on_y);
      // 两个儿子各取一半中间重复点多时较慢
      int half=sz(vp)/2;
      first=new Node({vp.begin(),vp.begin()+half});
      second=new Node({vp.begin()+half,vp.end()});
    }
  }
};

struct KDTree{
  Node*root;
  KDTree(const vector<P>&vp):root(new Node({all(vp)})){}

  pair<T,P>search(Node*node,const P&p){
    if(!node->first){
      // 若不应找到 p 自身则取消下一行注释
      // if (p == node->pt) return {INF, P()};
      return make_pair((p-node->pt).dist2(),node->pt);
    }

    Node*f=node->first,*s=node->second;
    T bfirst=f->distance(p),bsec=s->distance(p);
    if(bfirst>bsec)swap(bsec,bfirst),swap(f,s);

    // 先搜近的一侧必要时再搜另一侧
```

```
    auto best=search(f,p);
    if(bsec<best.first)
      best=min(best,search(s,p));
    return best;
  }

  // 求离 p 最近的点及其平方距离
  // 需要 Point 定义某种 operator<
  pair<T,P>nearest(const P&p){
    return search(root,p);
  }
};
```

## FastDelaunay.h

**Description:** 快速 Delaunay 三角剖分. 每个三角形的外接圆内都不含任何输入点. 不能有重复点. 若所有点共线, 不返回任何三角形. 也可用于 double, 但 circ 中可能有精度问题. 三角形按 {t[0][0], t[0][1], t[0][2], t[1][0], ...} 的顺序返回, 均为逆时针.

**Usage:** vector<P> t = triangulate(pts);
// 第 i 个三角形为 t[3*i], t[3*i+1], t[3*i+2]

**Time:** $\mathcal{O}(n \log n)$

"Point.h" eefdf5, 88 lines

```
typedef Point<ll>P;
typedef struct Quad*Q;
typedef __int128_t lll; // 坐标小于 2e4 时可用 ll
P arb(LLONG_MAX,LLONG_MAX); // 不等于任何其他点

struct Quad{
  Q rot,o;P p=arb;bool mark;
  P&F(){return r()->p;}
  Q&r(){return rot->rot;}
  Q prev(){return rot->o->rot;}
  Q next(){return r()->prev();}
}*H;

bool circ(P p,P a,P b,P c){ // p 是否在外接圆内
  lll p2=p.dist2(),A=a.dist2()-p2,
      B=b.dist2()-p2,C=c.dist2()-p2;
  return p.cross(a,b)*C+p.cross(b,c)*A+p.cross(c,a)*B>0;
}
Q makeEdge(P orig,P dest){
  Q r=H?H:new Quad{new Quad{new Quad{new Quad{0}}}};
  H=r->o;r->r()->r()=r;
  rep(i,0,4)r=r->rot,r->p=arb,r->o=i&1?r:r->r();
  r->p=orig;r->F()=dest;
  return r;
}
void splice(Q a,Q b){
  swap(a->o->rot->o,b->o->rot->o);swap(a->o,b->o);
}
Q connect(Q a,Q b){
  Q q=makeEdge(a->F(),b->p);
  splice(q,a->next());
  splice(q->r(),b);
  return q;
}

pair<Q,Q>rec(const vector<P>&s){
  if(sz(s)<=3){
    Q a=makeEdge(s[0],s[1]),b=makeEdge(s[1],s.back());
    if(sz(s)==2)return {a,a->r()};
    splice(a->r(),b);
    auto side=s[0].cross(s[1],s[2]);
    Q c=side?connect(b,a):0;
    return {side<0?c->r():a,side<0?c:b->r()};
  }

#define H(e) e->F(), e->p
#define valid(e) (e->F().cross(H(base)) > 0)
  Q A,B,ra,rb;
  int half=sz(s)/2;
  tie(ra,A)=rec({all(s)-half});
  tie(B,rb)=rec({sz(s)-half+all(s)});
  while((B->p.cross(H(A))<0&&(A=A->next()))||
        (A->p.cross(H(B))>0&&(B=B->r()->o)));
  Q base=connect(B->r(),A);
  if(A->p==ra->p)ra=base->r();
  if(B->p==rb->p)rb=base;

#define DEL(e, init, dir) Q e = init->dir; if (valid(e)) \
    while (circ(e->dir->F(), H(base), e->F())) { \
      Q t = e->dir; \
      splice(e, e->prev()); \
      splice(e->r(), e->r()->prev()); \
      e->o = H; H = e; e = t; \
    }
  for(;;){
    DEL(LC,base->r(),o);DEL(RC,base,prev());
    if(!valid(LC)&&!valid(RC))break;
    if(!valid(LC)||(valid(RC)&&circ(H(RC),H(LC))))
      base=connect(RC,base->r());
    else
      base=connect(base->r(),LC->r());
  }
  return {ra,rb};
}

vector<P>triangulate(vector<P>pts){
  sort(all(pts));assert(unique(all(pts))==pts.end());
  if(sz(pts)<2)return {};
  Q e=rec(pts).first;
  vector<Q>q={e};
  int qi=0;
  while(e->o->F().cross(e->F(),e->p)<0)e=e->o;
#define ADD { Q c = e; do { c->mark = 1; pts.push_back(c->p); \
  q.push_back(c->r());c=c->next();}while(c!=e);}
  ADD;pts.clear();
  while(qi<sz(q))if(!(e=q[qi++])->mark)ADD;
  return pts;
}
```

# 9.5 3D

## PolyhedronVolume.h

**Description:** 求多面体 (有向) 体积的神奇公式. 各面须朝外. trilist 的元素需有下标成员 a, b, c, 如 3dHull 的 F.

**Usage:** double vol = signedPolyVolume(pts, hull3d(pts));

3058c3, 6 lines

```
template<class V,class L>
double signedPolyVolume(const V&p,const L&trilist){
  double v=0;
  for(auto i:trilist)v+=p[i.a].cross(p[i.b]).dot(p[i.c]);
  return v/6;
}
```

## Point3D.h

**Description:** 三维空间点类. T 可取 double 或 long long.

**Usage:** typedef Point3D<double> P3; P3 a(1, 0, 0), b(0, 1, 0); P3 n = a.cross(b); double d = a.dot(b), l = n.dist();

8058ae, 32 lines

```
template<class T>struct Point3D{
  typedef Point3D P;
  typedef const P&R;
  T x,y,z;
  explicit Point3D(T x=0,T y=0,T z=0):x(x),y(y),z(z){}
  bool operator<(R p)const{
    return tie(x,y,z)<tie(p.x,p.y,p.z);}
  bool operator==(R p)const{
    return tie(x,y,z)==tie(p.x,p.y,p.z);}
  P operator+(R p)const{return P(x+p.x,y+p.y,z+p.z);}
  P operator-(R p)const{return P(x-p.x,y-p.y,z-p.z);}
  P operator*(T d)const{return P(x*d,y*d,z*d);}
  P operator/(T d)const{return P(x/d,y/d,z/d);}
  T dot(R p)const{return x*p.x+y*p.y+z*p.z;}
  P cross(R p)const{
    return P(y*p.z-z*p.y,z*p.x-x*p.z,x*p.y-y*p.x);
  }
  T dist2()const{return x*x+y*y+z*z;}
  double dist()const{return sqrt((double)dist2());}
  // 方位角即经度与 x 轴夹角取值 [-pi, pi]
  double phi()const{return atan2(y,x);}
  // 天顶角即纬度与 z 轴夹角取值 [0, pi]
  double theta()const{return atan2(sqrt(x*x+y*y),z);}
  P unit()const{return *this/(T)dist();} // 长度变为 1
  // 与 *this 和 p 都垂直的单位向量
  P normal(P p)const{return cross(p).unit();}
  // 绕 axis 逆时针旋转 angle 弧度
  P rotate(double angle,P axis)const{
    double s=sin(angle),c=cos(angle);P u=axis.unit();
    return u*dot(u)*(1-c)+(*this)*c-cross(u)*s;
  }
};
```

## 3dHull.h

**Description:** 求点集三维凸包的所有面. *不能有四点共面*, 否则结果随机. 所有面都朝外.

**Usage:** vector<F> fs = hull3d(pts);
// 面 f 的顶点下标 f.a, f.b, f.c, 外法向 f.q

**Time:** $\mathcal{O}(n^2)$

"Point3D.h" 5b45fc, 49 lines

```
typedef Point3D<double>P3;

struct PR{
  void ins(int x){(a==-1?a:b)=x;}
  void rem(int x){(a==x?a:b)=-1;}
  int cnt(){return (a!=-1)+(b!=-1);}
  int a,b;
};

struct F{P3 q;int a,b,c;};

vector<F>hull3d(const vector<P3>&A){
  assert(sz(A)>=4);
  vector<vector<PR>>E(sz(A),vector<PR>(sz(A),{-1,-1}));
#define E(x,y) E[f.x][f.y]
  vector<F>FS;
  auto mf=[&](int i,int j,int k,int l){
    P3 q=(A[j]-A[i]).cross((A[k]-A[i]));
    if(q.dot(A[l])>q.dot(A[i]))
      q=q*-1;
    F f{q,i,j,k};
    E(a,b).ins(k);E(a,c).ins(j);E(b,c).ins(i);
    FS.push_back(f);
  };
  rep(i,0,4)rep(j,i+1,4)rep(k,j+1,4)
    mf(i,j,k,6-i-j-k);

  rep(i,4,sz(A)){
    rep(j,0,sz(FS)){
      F f=FS[j];
      if(f.q.dot(A[i])>f.q.dot(A[f.a])){
        E(a,b).rem(f.c);
```

```
        E(a,c).rem(f.b);
        E(b,c).rem(f.a);
        swap(FS[j--],FS.back());
        FS.pop_back();
      }
    }
    int nw=sz(FS);
    rep(j,0,nw){
      F f=FS[j];
#define C(a, b, c) if (E(a,b).cnt() != 2) mf(f.a, f.b, i, f.c);
      C(a,b,c);C(a,c,b);C(b,c,a);
    }
  }
  for(F&it:FS)if((A[it.b]-A[it.a]).cross(
    A[it.c]-A[it.a]).dot(it.q)<=0)swap(it.c,it.b);
  return FS;
};
```

## sphericalDistance.h

**Description:** 返回半径为 radius 的球面上两点间的最短 (球面) 距离. 两点的方位角 (经度) 为 f1 ($\phi_1$) 和 f2 ($\phi_2$), 从 x 轴量起; 天顶角 (纬度) 为 t1 ($\theta_1$) 和 t2 ($\theta_2$), 从 z 轴量起 (0 为北极). 角度均为弧度. 算法先把球坐标转为直角坐标, 若已有直角坐标, 只需用最后两行. 此时 dx*radius 为两点在 x 方向上的差, d*radius 为两点间的直线距离.
**Usage:** `double d = sphericalDistance(f1, t1, f2, t2, radius);`

611f07, 8 lines

```
double sphericalDistance(double f1,double t1,
    double f2,double t2,double radius){
  double dx=sin(t2)*cos(f2)-sin(t1)*cos(f1);
  double dy=sin(t2)*sin(f2)-sin(t1)*sin(f1);
  double dz=cos(t2)-cos(t1);
  double d=sqrt(dx*dx+dy*dy+dz*dz);
  return radius*2*asin(d/2);
}
```

# Strings (10)

## KMP.h

**Description:** pi[x] 为 s 的最长前缀中以 x 结尾者的长度, 不算 s[0...x] 本身 (即 s[0...x] 的最长真 border; abacaba 得 0010123). match 找出 pat 在 s 中的所有出现位置.
**Usage:** `vi p = pi(s);`
`vi occ = match(s, pat); // 每次出现的起始下标`
**Time:** $\mathcal{O}(n)$

d4375c, 16 lines

```
vi pi(const string&s){
  vi p(sz(s));
  rep(i,1,sz(s)){
    int g=p[i-1];
    while(g&&s[i]!=s[g])g=p[g-1];
    p[i]=g+(s[i]==s[g]);
  }
  return p;
}

vi match(const string&s,const string&pat){
  vi p=pi(pat+'\0'+s),res;
  rep(i,sz(p)-sz(s),sz(p))
    if(p[i]==sz(pat))res.push_back(i-2*sz(pat));
  return res;
}
```

## Zfunc.h

**Description:** z[i] 为 s[i:] 与 s 的最长公共前缀长度, 但 z[0] = 0. (abacaba 得 0010301)
**Usage:** `vi z = Z(s);`
**Time:** $\mathcal{O}(n)$

ee09e2, 12 lines

```
vi Z(const string&S){
  vi z(sz(S));
  int l=-1,r=-1;
  rep(i,1,sz(S)){
    z[i]=i>=r?0:min(r-i,z[i-l]);
    while(i+z[i]<sz(S)&&S[i+z[i]]==S[z[i]])
      z[i]++;
    if(i+z[i]>r)
      l=i,r=i+z[i];
  }
  return z;
}
```

## Manacher.h

**Description:** 对串的每个位置求: p[0][i] = 以 i-1 和 i 之间为中心的最长偶回文的半长, p[1][i] = 以 i 为中心的最长奇回文的半长 (向下取整).
**Usage:** `array<vi, 2> p = manacher(s); // p[0] 长 n+1, p[1] 长 n`
**Time:** $\mathcal{O}(N)$

e7ad79, 13 lines

```
array<vi,2>manacher(const string&s){
  int n=sz(s);
  array<vi,2>p={vi(n+1),vi(n)};
  rep(z,0,2)for(int i=0,l=0,r=0;i<n;i++){
    int t=r-i+!z;
    if(i<r)p[z][i]=min(t,p[z][l+t]);
    int L=i-p[z][i],R=i+p[z][i]-!z;
    while(L>=1&&R+1<n&&s[L-1]==s[R+1])
      p[z][i]++,L--,R++;
    if(R>r)l=L,r=R;
  }
  return p;
}
```

## MinRotation.h

**Description:** Finds the lexicographically smallest rotation of a string.
**Usage:** `rotate(v.begin(), v.begin()+minRotation(v), v.end());`
**Time:** $\mathcal{O}(N)$

d07a42, 8 lines

```
int minRotation(string s){
  int a=0,N=sz(s);s+=s;
  rep(b,0,N)rep(k,0,N){
    if(a+k==b||s[a+k]<s[b+k]){b+=max(0,k-1);break;}
    if(s[a+k]>s[b+k]){a=b;break;}
  }
  return a;
}
```

## SuffixArray.h

**Description:** 构建字符串的后缀数组. `sa[i]` 是排序后第 $i$ 个后缀的起始下标. 返回的数组长 $n+1$, 且 `sa[0] = n`. `lcp` 数组存后缀数组中相邻后缀的最长公共前缀: `lcp[i] = lcp(sa[i], sa[i-1])`, `lcp[0] = 0`. 输入串不能含字符 0.
**Usage:** `SuffixArray S(s); S.sa[i]; S.lcp[i];`
**Time:** $\mathcal{O}(n \log n)$

635552, 22 lines

```
struct SuffixArray{
  vi sa,lcp;
  SuffixArray(string s,int lim=256){ // 或 vector<int>
    s.push_back(0);int n=sz(s),k=0,a,b;
    vi x(all(s)),y(n),ws(max(n,lim));
    sa=lcp=y,iota(all(sa),0);
    for(int j=0,p=0;p<n;j=max(1,j*2),lim=p){
      p=j,iota(all(y),n-j);
      rep(i,0,n)if(sa[i]>=j)y[p++]=sa[i]-j;
      fill(all(ws),0);
      rep(i,0,n)ws[x[i]]++;
      rep(i,1,lim)ws[i]+=ws[i-1];
      for(int i=n;i--;)sa[--ws[x[y[i]]]]=y[i];
      swap(x,y),p=1,x[sa[0]]=0;
      rep(i,1,n)a=sa[i-1],b=sa[i],x[b]=
        (y[a]==y[b]&&y[a+j]==y[b+j])?p-1:p++;
    }
    for(int i=0,j;i<n-1;lcp[x[i++]]=k)
      for(k&&k--,j=sa[x[i]-1];
          s[i+k]==s[j+k];k++);
  }
};
```

## SuffixAutomaton.h

**Description:** 后缀自动机. 节点 0 总是起始节点, 但节点不一定按拓扑序排列. 每个状态有 len (最长串长度), link (后缀链接), next (转移).
**Usage:** `SAM sam; for (char c : s) sam.extend(c);`
`sam.q[v].len; sam.q[v].link; sam.q[v].next[c];`
**Time:** $\mathcal{O}(n \log \Sigma)$

079638, 36 lines

```
struct SAM{
    struct state{
        int len,link;
        map<char,int>next;
    };
    vector<state>q;
    int last;
    SAM():last(0){q.push_back({0,-1});}
    int new_node(){
        q.emplace_back();return sz(q)-1;
    }
    void extend(char c){
        int cur=new_node(),i=last;
        q[cur].len=q[last].len+1;
        while(i!=-1&&!q[i].next.contains(c))
            q[i].next[c]=cur,i=q[i].link;
        if(i==-1)q[cur].link=0;
        else{
            int j=q[i].next[c];
            if(q[i].len+1==q[j].len)
                q[cur].link=j;
            else{
                int clone=new_node();
                q[clone].len=q[i].len+1;
                q[clone].next=q[j].next;
                q[clone].link=q[j].link;
                while(i!=-1&&q[i].next[c]==j){
                    q[i].next[c]=clone;
                    i=q[i].link;
                }
                q[j].link=q[cur].link=clone;
            }
        }
        last=cur;
    }
};
```

## Hashing.h

**Description:** 字符串哈希, 各函数用途见名知意.
**Usage:** `HashInterval hi(s); H h = hi.hashInterval(a, b); // [a, b)`
`vector<H> hs = getHashes(s, len); H g = hashString(s);`

2d2a67, 43 lines

```
// 模 2^64-1 运算. 比模 2^64 慢 2 倍, 代码也更长, 但能防恶意
// 数据, 如 Thue-Morse 串, 长 2^10 的 ABBA... 和 BAAB...
// 模 2^64 哈希相同. 数据随机时可改用 typedef ull H;
// 不担心生日悖论时也可模 10^9+7.
typedef uint64_t ull;
struct H{
  ull x;H(ull x=0):x(x){}
  H operator+(H o){return x+o.x+(x+o.x<x);}
```

```
  H operator-(H o){return *this+~o.x;}
  H operator*(H o){auto m=(__uint128_t)x*o.x;
    return H((ull)m)+(ull)(m>>64);}
  ull get()const{return x+!~x;}
  bool operator==(H o)const{return get()==o.get();}
  bool operator<(H o)const{return get()<o.get();}
};
static const H C=(ll)1e11+3; // 阶约 3e9; 随机也可

struct HashInterval{
  vector<H>ha,pw;
  HashInterval(string&str):ha(sz(str)+1),pw(ha){
    pw[0]=1;
    rep(i,0,sz(str))
      ha[i+1]=ha[i]*C+str[i],
      pw[i+1]=pw[i]*C;
  }
  H hashInterval(int a,int b){ // [a, b) 的哈希
    return ha[b]-ha[a]*pw[b-a];
  }
};

vector<H>getHashes(string&str,int length){
  if(sz(str)<length)return {};
  H h=0,pw=1;
  rep(i,0,length)
    h=h*C+str[i],pw=pw*C;
  vector<H>ret={h};
  rep(i,length,sz(str)){
    ret.push_back(h=h*C+str[i]-pw*str[i-length]);
  }
  return ret;
}

H hashString(string&s){H h{};for(char c:s)h=h*C+c;return h;}
```

### AhoCorasick.h

**Description:** AC 自动机 (Aho-Corasick), 用于多模式串匹配. 起始节点下标为 0. 默认字母表为大写字母, 按需修改 alpha 和 first. find(word) 对每个位置返回在该处结尾的最长模式串的下标, 没有则为 -1. findAll(−, word) 对每个位置找出从该处开始的所有模式串 (短的在前; 若无重复模式串, 总数至多 $N\sqrt{N}$). 允许重复模式串, 不允许空串. 要求从每个位置开始的最长模式串, 就把输入全部反转. 字母表很大时, 把每个符号拆成若干块, 并用哨兵位标记符号边界.

**Usage:** `AhoCorasick ac(pat); // pat 为 vector<string>`
`vi r = ac.find(word); vector<vi> res = ac.findAll(pat, word);`

**Time:** 构建 $\mathcal{O}(26N)$, 其中 $N$ = 模式串总长. find(x) 为 $\mathcal{O}(N)$, 其中 N = x 的长度. findAll 为 $\mathcal{O}(NM)$.

f35677, 66 lines

```
struct AhoCorasick{
  enum{alpha=26,first='A'}; // 按需修改!
  struct Node{
    // (nmatches 可选)
    int back,next[alpha],start=-1,end=-1,nmatches=0;
    Node(int v){memset(next,v,sizeof(next));}
  };
  vector<Node>N;
  vi backp;
  void insert(string&s,int j){
    assert(!s.empty());
    int n=0;
    for(char c:s){
      int&m=N[n].next[c-first];
      if(m==-1){n=m=sz(N);N.emplace_back(-1);}
      else n=m;
    }
    if(N[n].end==-1)N[n].start=j;
    backp.push_back(N[n].end);
    N[n].end=j;
    N[n].nmatches++;
  }
  AhoCorasick(vector<string>&pat):N(1,-1){
    rep(i,0,sz(pat))insert(pat[i],i);
    N[0].back=sz(N);
    N.emplace_back(0);

    queue<int>q;
    for(q.push(0);!q.empty();q.pop()){
      int n=q.front(),prev=N[n].back;
      rep(i,0,alpha){
        int&ed=N[n].next[i],y=N[prev].next[i];
        if(ed==-1)ed=y;
        else{
          N[ed].back=y;
          (N[ed].end==-1?N[ed].end:backp[N[ed].start])
            =N[y].end;
          N[ed].nmatches+=N[y].nmatches;
          q.push(ed);
        }
      }
    }
  }
  vi find(string word){
    int n=0;
    vi res; // ll count = 0;
    for(char c:word){
      n=N[n].next[c-first];
      res.push_back(N[n].end);
      // count += N[n].nmatches;
    }
    return res;
  }
  vector<vi>findAll(vector<string>&pat,string word){
    vi r=find(word);
    vector<vi>res(sz(word));
    rep(i,0,sz(word)){
      int ind=r[i];
      while(ind!=-1){
        res[i-sz(pat[ind])+1].push_back(ind);
        ind=backp[ind];
      }
    }
    return res;
  }
};
```

# Various (11)

## 11.1 Intervals

### IntervalContainer.h

**Description:** 在一组互不相交的区间中添加或删除区间。 添加时会与集合中所有重叠的区间合并。区间均为左闭右开 $[L, R)$。

**Usage:** `set<pii> is; auto it = addInterval(is, L, R);`
`removeInterval(is, L, R); // it 指向合并后的区间`

**Time:** $\mathcal{O}(\log N)$

edce47, 23 lines

```
set<pii>::iterator addInterval(set<pii>&is,int L,int R){
  if(L==R)return is.end();
  auto it=is.lower_bound({L,R}),before=it;
  while(it!=is.end()&&it->first<=R){
    R=max(R,it->second);
    before=it=is.erase(it);
  }
  if(it!=is.begin()&&(--it)->second>=L){
    L=min(L,it->first);
    R=max(R,it->second);
    is.erase(it);
  }
  return is.insert(before,{L,R});
}

void removeInterval(set<pii>&is,int L,int R){
  if(L==R)return;
  auto it=addInterval(is,L,R);
  auto r2=it->second;
  if(it->first==L)is.erase(it);
  else(int&)it->second=L;
  if(R!=r2)is.emplace(R,r2);
}
```

### IntervalCover.h

**Description:** 求覆盖区间 G 所需最少区间的下标集合。 区间均为左闭右开。若要支持闭区间，将 (A) 处的条件加上 `|| R.empty()`。 失败时（或 G 为空时）返回空集。

**Usage:** `vector<pii> I; vi r = cover(pii(L, R), I);`
`// r 为所选区间在 I 中的下标 , 失败时为空`

**Time:** $\mathcal{O}(N \log N)$

9e9d8d, 19 lines

```
template<class T>
vi cover(pair<T,T>G,vector<pair<T,T>>I){
  vi S(sz(I)),R;
  iota(all(S),0);
  sort(all(S),[&](int a,int b){return I[a]<I[b];});
  T cur=G.first;
  int at=0;
  while(cur<G.second){ // (A)
    pair<T,int>mx=make_pair(cur,-1);
    while(at<sz(I)&&I[S[at]].first<=cur){
      mx=max(mx,make_pair(I[S[at]].second,S[at]));
      at++;
    }
    if(mx.second==-1)return {};
    cur=mx.first;
    R.push_back(mx.second);
  }
  return R;
}
```

### ConstantIntervals.h

**Description:** 将 [from, to) 上的单调函数划分为最少的若干半开区间，使函数在每个区间上取值相同。 对每个这样的区间调用回调 g。

**Usage:** `auto f = [&](int x) { return v[x]; };`
`auto g = [&](int lo, int hi, T val) { ... };`
`constantIntervals(0, sz(v), f, g);`
`// 对每段 [lo, hi) 调用 g, 段内 f 的值均为 val`

**Time:** $\mathcal{O}\left(k \log \frac{n}{k}\right)$

753a4c, 19 lines

```
template<class F,class G,class T>
void rec(int from,int to,F&f,G&g,int&i,T&p,T q){
  if(p==q)return;
  if(from==to){
    g(i,to,p);
    i=to;p=q;
  }else{
    int mid=(from+to)>>1;
    rec(from,mid,f,g,i,p,f(mid));
    rec(mid+1,to,f,g,i,p,q);
  }
}
template<class F,class G>
void constantIntervals(int from,int to,F f,G g){
  if(to<=from)return;
  int i=from;auto p=f(i),q=f(to-1);
  rec(from,to-1,f,g,i,p,q);
```

```
  g(i,to,q);
}
```

## 11.2 Misc. algorithms

### TernarySearch.h

**Description:** 在 $[a,b]$ 中找使 $f(i)$ 最大的最小的 i，前提是 $f(a) < \ldots < f(i) \geq \cdots \geq f(b)$。若要让另一侧允许非严格不等，把 (A) 处的 $<$ 改为 $<=$，并把 (B) 处的循环反向。若要求最小值，把它改为 $>$，(B) 处同样修改。
**Usage:** `int ind = ternSearch(0,n-1,[&](int i){return a[i];});`
**Time:** $\mathcal{O}(\log(b-a))$

9155b4, 11 lines

```
template<class F>
int ternSearch(int a,int b,F f){
  assert(a<=b);
  while(b-a>=5){
    int mid=(a+b)/2;
    if(f(mid)<f(mid+1))a=mid; // (A)
    else b=mid+1;
  }
  rep(i,a+1,b+1)if(f(a)<f(i))a=i; // (B)
  return a;
}
```

### LIS.h

**Description:** 求最长严格上升子序列，返回其各元素的下标。
**Usage:** `vi idx = lis(S); // S 为 vector，返回递增的下标序列`
**Time:** $\mathcal{O}(N \log N)$

2932a0, 17 lines

```
template<class I>vi lis(const vector<I>&S){
  if(S.empty())return {};
  vi prev(sz(S));
  typedef pair<I,int>p;
  vector<p>res;
  rep(i,0,sz(S)){
    // 求最长不降子序列时把 0 改为 i
    auto it=lower_bound(all(res),p{S[i],0});
    if(it==res.end())res.emplace_back(),it=res.end()-1;
    *it={S[i],i};
    prev[i]=it==res.begin()?0:(it-1)->second;
  }
  int L=sz(res),cur=res.back().second;
  vi ans(L);
  while(L--)ans[L]=cur,cur=prev[cur];
  return ans;
}
```

### FastKnapsack.h

**Description:** 给定 N 个非负整数重量 w 和非负目标 t，求最大的 $S \leq t$，使 S 是某个重量子集之和。
**Usage:** `int S = knapsack(w, t); // w 为 vi`
**Time:** $\mathcal{O}(N \max(w_i))$

b20ccc, 16 lines

```
int knapsack(vi w,int t){
  int a=0,b=0,x;
  while(b<sz(w)&&a+w[b]<=t)a+=w[b++];
  if(b==sz(w))return a;
  int m=*max_element(all(w));
  vi u,v(2*m,-1);
  v[a+m-t]=b;
  rep(i,b,sz(w)){
    u=v;
    rep(x,0,m)v[x+w[i]]=max(v[x+w[i]],u[x]);
    for(x=2*m;--x>m;)rep(j,max(0,u[x]),v[x])
      v[x-w[j]]=max(v[x-w[j]],j);
  }
  for(a=t;v[a+m-t]<0;a--);
  return a;
}
```

## 11.3 Dynamic programming

### KnuthDP.h

**Description:** 区间 DP $a[i][j] = \min_{i<k<j}(a[i][k] + a[k][j]) + f(i,j)$ 中，若（最小的）最优 $k$ 随 $i$ 和 $j$ 都单调不减，则可按区间长度递增的顺序求解，且求 $a[i][j]$ 时只需在 $p[i][j-1]$ 与 $p[i+1][j]$ 之间搜索 $k = p[i][j]$。这称为 Knuth DP。充分条件为：对所有 $a \leq b \leq c \leq d$，有 $f(b,c) \leq f(a,d)$ 且 $f(a,c) + f(b,d) \leq f(a,d) + f(b,c)$。另可考虑：LineContainer（数据结构一章）、单调队列、三分搜索。
**Usage:** `rep(len,2,n+1) rep(i,0,n-len+1) { int j = i+len; rep(k, max(i+1, p[i][j-1]), min(j-1, p[i+1][j]) + 1) 用 k 更新 a[i][j] 和 p[i][j]; } // 初始 p[i][i+1] = i`
**Time:** $\mathcal{O}(N^2)$

```
}
```

### DivideAndConquerDP.h

**Description:** 给定 $a[i] = \min_{lo(i)\leq k<hi(i)}(f(i,k))$，其中（最小的）最优 $k$ 随 $i$ 单调不减，对 $i = L..R-1$ 求出 $a[i]$。
**Usage:** `按需改写 lo, hi, f, store; DP().solve(L, R); // 每个 i 求出最优 k 后调用 store(i, k, a[i])`
**Time:** $\mathcal{O}((N + (hi - lo))\log N)$

d38d2b, 18 lines

```
struct DP{ // 按需修改
  int lo(int ind){return 0;}
  int hi(int ind){return ind;}
  ll f(int ind,int k){return dp[ind][k];}
  void store(int ind,int k,ll v){res[ind]=pii(k,v);}

  void rec(int L,int R,int LO,int HI){
    if(L>=R)return;
    int mid=(L+R)>>1;
    pair<ll,int>best(LLONG_MAX,LO);
    rep(k,max(LO,lo(mid)),min(HI,hi(mid)))
      best=min(best,make_pair(f(mid,k),k));
    store(mid,best.second,best.first);
    rec(L,mid,LO,best.second+1);
    rec(mid+1,R,best.second,HI);
  }
  void solve(int L,int R){rec(L,R,INT_MIN,INT_MAX);}
};
```

## 11.4 Debugging tricks

- `signal(SIGSEGV, [](int) { _Exit(0); });` converts segfaults into Wrong Answers. Similarly one can catch SIGABRT (assertion failures) and SIGFPE (zero divisions). `_GLIBCXX_DEBUG` failures generate SIGABRT (or SIGSEGV on gcc 5.4.0 apparently).
- `feenableexcept(29);` kills the program on NaNs (1), 0-divs (4), infinities (8) and denormals (16).

## 11.5 Optimization tricks

`__builtin_ia32_ldmxcsr(40896);` disables denormals (which make floats 20x slower near their minimum value).

### 11.5.1 Bit hacks

- `x & -x` is the least bit in x.
- `for (int x = m; x; ) { --x &= m; ... }` loops over all subset masks of `m` (except `m` itself).
- `c = x&-x, r = x+c; (((r^x) >> 2)/c) | r` is the next number after x with the same number of bits set.
- `rep(b,0,K) rep(i,0,(1 << K)) if (i & 1 << b) D[i] += D[i^(1 << b)];` computes all sums of subsets.

### 11.5.2 Pragmas for bitset (put before bits/stdc++.h)

- `#pragma GCC optimize("O3,unroll-loops")`
- `#include <bits/allocator.h>`
- `#pragma GCC target("avx2,bmi,bmi2,popcnt,lzcnt")`

### FastMod.h

**Description:** 计算 $a\%b$，比普通取模快约 5 倍，适用于 $b$ 为常量但编译期未知的情形。返回值与 $a$ 模 $b$ 同余（$a \pmod b$），范围为 $[0, 2b)$。
**Usage:** `FastMod F(b); ull r = F.reduce(a); // r 在 [0, 2b) 内`

751a02, 8 lines

```
typedef unsigned long long ull;
struct FastMod{
  ull b,m;
  FastMod(ull b):b(b),m(-1ULL/b){}
  ull reduce(ull a){ // a % b + (0 或 b)
    return a-(ull)((__uint128_t(m)*a)>>64)*b;
  }
};
```

### UniformRandom.h

**Description:** 高质量的均匀整数随机数生成器。
**Usage:** `int x = urand(a, b); // [a, b] 内的均匀随机整数`
**Time:** 生成 1e8 个数约 400ms。

39a02b, 3 lines

```
// 64 位整数可用 mt19937_64
mt19937 rng(chrono::steady_clock::now().time_since_epoch().
    count());
int urand(int a,int b){return uniform_int_distribution<int>(a,b
    )(rng);}
```

### StressTest.sh

24 lines

```
# Usage: bash stress.sh [fast] [slow] [gen] [# iters]
# Example: bash stress.sh main main2 generator 1000
# remove this and add -O2 if you want, cc8b47
flags="-std=c++20 -g -fsanitize=address,undefined"
g++ $flags $1.cpp -o $1
g++ $flags $2.cpp -o $2
g++ $flags $3.cpp -o $3
for ((t=1; t<=$4; t++)) do
  ./$3 > input
  ./$2 < input > outslow
  ./$1 < input > outwrong
  if [[ "$?" -ne 0 ]] || ! (cmp -s "outwrong" "outslow") then
    echo error found
    echo input:
    cat input
```

```
    echo wrong output:
    cat outwrong
    echo slow output:
    cat outslow
    exit
  fi
  echo passed $t tests
done
echo passed all tests
```

## ExpressionTree.h

**Description:** 为左结合运算构建表达式树。 按需修改 N 与 prec()。
**Usage:** `Val *root = parseExpression(s); // 之后在树上 dp...`
**Time:** $\mathcal{O}(N)$

9842ea, 53 lines

```
const int N=2e6; // 树的最大节点数取 |S| 即可
struct Val{ // 每个节点 32 字节
    int v; // 仅当 op = 0 时使用
    char op;Val*l,*r; // 仅当 op 非 0 时使用
    Val(int v=0):v(v),op(0),l(0),r(0){}
    Val(char op,Val*l,Val*r):op(op),l(l),r(r){}
};
Val arr[N];int last=0;

Val*parseExpression(string&s){
    using C=array<int,3>; // 优先级比较键
    vector<pair<C,char>>ops;
    vector<Val*>vals;
    int depth=0;

    auto prec=[&](char c,int i)->C{ // 改成你的运算符
        int r;
        if(c=='+'||c=='-')r=0;
        else if(c=='*'||c=='/'||c=='_')r=1;
        else assert(0); // 右结合用 i，左结合用 -i
        return {depth,r,c=='_'?i:-i};
    };
    auto clear=[&](C minp){
        while(!ops.empty()){
            auto[p,c]=ops.back();
            if(p<minp)break;
            ops.pop_back();
            auto b=vals.back();vals.pop_back();
            if(c=='_'){ // 一元运算
                vals.pb(&(arr[last++]=Val(c,b,0)));
            }else{ // 二元运算
                auto a=vals.back();vals.pop_back();
                vals.pb(&(arr[last++]=Val(c,a,b)));
            }
        }
    };
    for(int i=-1;char c:s){i++;
        bool pdigit=i>=1&&'0'<=s[i-1]&&s[i-1]<='9';
        if(c=='(')depth++;
        else if(c==')')depth--;
        else if('0'<=c&&c<='9'){
            if(pdigit)vals.back()->v=10*vals.back()->v+c-'0';
            else vals.pb(&(arr[last++]=Val(c-'0')));
        }else{
            // 一元运算
            if(c=='-'&&!pdigit
                &&!(i>=1&&s[i-1]==')'))c='_';
            clear(prec(c,i));ops.pb({prec(c,i),c});
        }
    }
    clear({-1});
    return vals[0];
}
```